

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт по лабораторным работам №1–5

По дисциплине: «Теория графов»

На тему: «Алгоритмы на графах»

Вариант №44

Выполнил студент
группы 5130201/40002

_____ Семенов И. А.

Проверил
преподаватель

_____ Востров А. В.

«_____» _____ 2026г.

Содержание

Введение	3
1 Постановка задачи	4
2 Математическое описание	7
2.1 Генерация графа и основные понятия	7
2.2 Связность и точки сочленения	12
2.3 Максимальный поток и поток минимальной стоимости . .	16
2.4 Остовы, код Прюфера и независимые множества рёбер . .	19
2.5 Эйлеровы циклы и фундаментальная система разрезов . .	22
3 Особенности реализации	24
3.1 Лабораторная работа № 1	24
3.2 Лабораторная работа № 2	28
3.3 Лабораторная работа № 3	32
3.4 Лабораторная работа № 4	38
3.5 Лабораторная работа № 5	51
4 Результаты работы	58
4.1 Лабораторная работа № 1	58
4.2 Лабораторная работа № 2	60
4.3 Лабораторная работа № 3	62
4.4 Лабораторная работа № 4	65
4.5 Лабораторная работа № 5	68
Заключение	71
Список литературы	74
Приложение. Исходный код программы	75

Введение

Теория графов — раздел дискретной математики, изучающий объекты и связи между ними с помощью графов, состоящих из вершин и рёбер. В рамках теории графов рассматриваются задачи анализа связности, поиска путей, построения остовов, исследования циклов, потоков и паросочетаний. Графовые модели применяются при описании транспортных и компьютерных сетей, социальных связей, структур данных и других сложных систем.

В отчёте представлено описание выполнения цикла лабораторных работ по дисциплине «Теория графов». Целью работ является изучение способов представления графов и практическая реализация алгоритмов для их генерации, анализа и преобразования.

Разработанная консольная программа на языке C++ объединяет результаты пяти лабораторных работ. Программа выполняет случайную генерацию графа и весовых матриц, вычисляет эксцентриситеты вершин, выделяет центр графа и диаметральные вершины, ищет маршруты, кратчайшие пути, точки сочленения и потоки, строит минимальный остов, кодирует его кодом Прюфера, находит максимальное независимое множество рёбер, строит эйлеров цикл и фундаментальную систему разрезов. Исходный граф, сформированный в первой работе, затем используется в последующих работах.

В отчёте рассмотрены математическое описание использованных алгоритмов, особенности их программной реализации и результаты работы программы на сгенерированных графах.

1 Постановка задачи

Цель работы: разработать единый программный комплекс с текстовым пользовательским интерфейсом для генерации, анализа и преобразования графов, объединяющий алгоритмы теории графов, реализованные в ходе выполнения пяти лабораторных работ.

Задачи.

Первая лабораторная работа:

1. сформировать случайным образом связный ациклический ориентированный граф в соответствии со смещённым распределением Вейбулла с заданным пользователем количеством вершин;
2. реализовать генерацию весовой матрицы в соответствии со смещённым распределением Вейбулла с положительными, отрицательными или смешанными весами при заданном пользователем количестве рёбер;
3. посчитать эксцентриситеты вершин;
4. выделить центр графа;
5. определить диаметральные вершины;
6. реализовать метод Шимбелла для сгенерированной весовой матрицы при заданном пользователем количестве рёбер;
7. выводить максимальный или минимальный путь, полученный методом Шимбелла, в виде весовой матрицы;
8. определить возможность построения маршрута между двумя заданными пользователем вершинами и посчитать количество таких маршрутов;
9. реализовать пользовательское меню;

Вторая лабораторная работа:

10. найти точки сочленения в сгенерированном графе;
11. найти кратчайший путь между выбранными вершинами с помощью алгоритма Дейкстры для отрицательных весов, вывести расстояние, вектор расстояний и последовательность вершин пути;

12. сравнить скорость работы алгоритмов по количеству итераций;

Третья лабораторная работа:

13. сформировать случайным образом матрицу пропускных способностей в соответствии со смещённым распределением Вейбулла на основе сгенерированного ориентированного графа;
14. сформировать случайным образом матрицу стоимостей в соответствии со смещённым распределением Вейбулла на основе сгенерированного ориентированного графа;
15. найти максимальный поток для полученной матрицы пропускных способностей по алгоритму Форда–Фалкерсона;
16. вычислить поток минимальной стоимости величины $\lfloor 2/3 \cdot \max \rfloor$, здесь $\max$ — найденный максимальный поток;

Четвёртая лабораторная работа:

17. найти число остовных деревьев неориентированного графа с помощью матричной теоремы Кирхгофа;
18. построить минимальный по весу остов для неориентированного взвешенного графа алгоритмом Краскала;
19. закодировать остов кодом Прюфера с сохранением весов рёбер и декодировать его;
20. найти максимальное независимое множество рёбер на исходном графе и полученном остове по выбору пользователя;

Пятая лабораторная работа:

21. проверить, является ли неориентированный граф эйлеровым;
22. при необходимости модифицировать граф с сохранением связности и вывести сведения о выполненных изменениях;
23. построить эйлеров цикл;
24. построить минимальный остов алгоритмом Краскала;
25. получить фундаментальную систему разрезов и вывести её на экран;

26. получить остальные разрезы из фундаментальных с помощью операции симметрической разности для выбранных пользователем разрезов.

2 Математическое описание

2.1 Генерация графа и основные понятия

Графом $G(V, E)$ называется совокупность двух множеств:

- непустое множество V — множество вершин;
- E , множество двуэлементных подмножеств множества V , — множество рёбер.

Ребро можно трактовать не только как множество $\{v_1, v_2\}$, но и как пару вершин (v_1, v_2) .

$$p \stackrel{\text{Def}}{=} p(G) \stackrel{\text{Def}}{=} |V|, \quad q \stackrel{\text{Def}}{=} q(G) \stackrel{\text{Def}}{=} |E|.$$

Если рёбра задаются упорядоченными парами, то граф называется ориентированным. В противном случае — неориентированным. Степенью вершины $d(v)$ называется количество рёбер, инцидентных этой вершине.

Для неориентированного графа справедлива лемма о рукопожатиях. Сумма степеней вершин графа равна удвоенному количеству рёбер.

$$\sum_{v \in V} d(v) = 2q.$$

Маршрутом в графе называется чередующаяся последовательность вершин и рёбер, начинающаяся и заканчивающаяся вершиной, в которой любые два соседних элемента инцидентны.

- Если все рёбра различны, то маршрут называется цепью.
- Если все вершины различны, то маршрут называется простой цепью.

Если $v_0 = V_k$, то маршрут замкнут, иначе — открыт.

Цепь, соединяющая вершины u и v , обозначается $\langle u, v \rangle$. Говорят, что две вершины в графе связны, если существует соединяющая их цепь. *Граф, в котором все вершины связны, называется связным.*

Замкнутая цепь называется циклом, замкнутая простая цепь называется простым циклом. Число циклов в графе обозначается $z(G)$. *Граф без циклов называется ациклическим.*

В первой лабораторной работе связный ациклический граф генерируется случайным образом на основе смещённого распределения Вейбулла. Распределение задаётся функцией, приведённой в справочнике Р. Н. Вадзинского:

$$F(y) = 1 - \exp \left(- \left(\frac{y - y_0}{a} \right)^c \right),$$

где y — случайная величина, a — параметр масштаба, c — параметр формы ($c > 0$), y_0 — параметр смещения.

Для генерации случайных чисел по смещённому распределению Вейбулла используется формула

$$y = y_0 + a(-\ln r)^{1/c}.$$

Здесь r — равномерно распределённая на интервале $(0, 1)$ случайная величина.

На рис. 1 показана функция риска распределения Вейбулла при параметрах $a = 1$, $c = 0,5; 1,5; 2,5$, $y_0 = 0$.

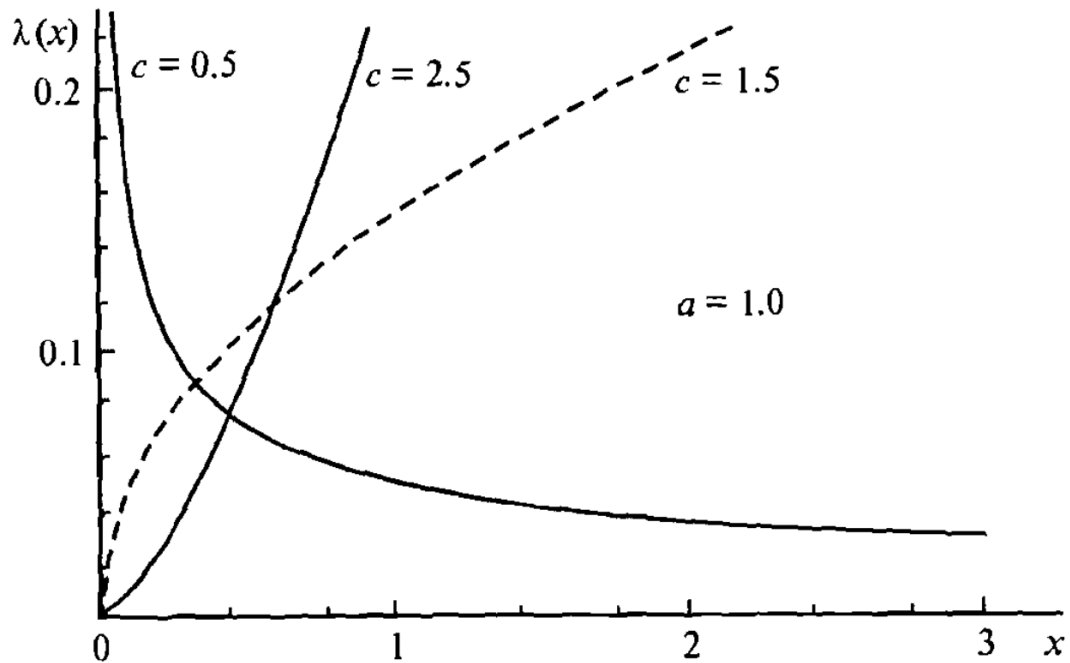


Рис. 1: Функция риска распределения Вейбулла

На рис. 2 приведены гистограммы 1000 сгенерированных значений для тех же параметров. При $c < 1$ большая часть значений сосредоточена возле нуля. При увеличении c значения становятся более сконцентрированными, а максимум гистограммы смещается вправо.

Смещённое распределение Вейбулла при различных значениях c

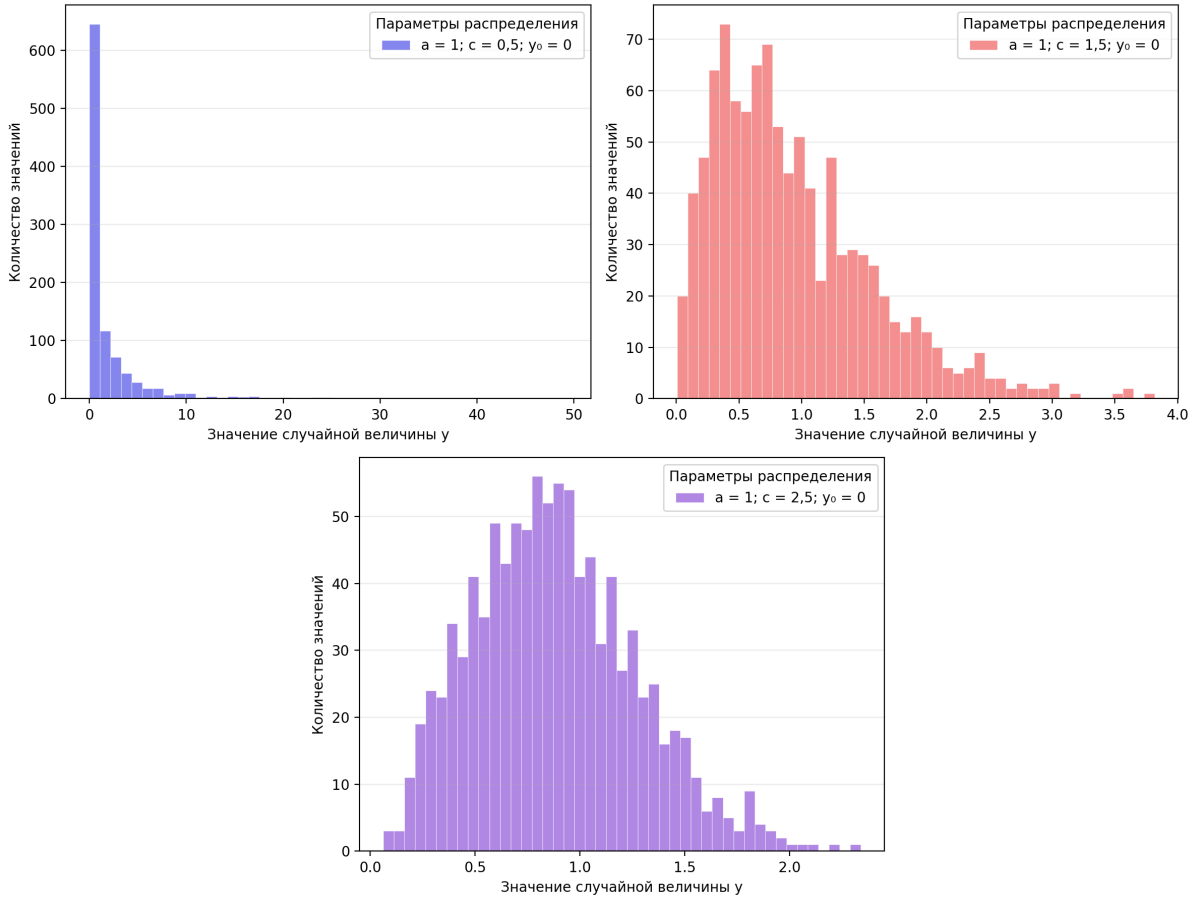


Рис. 2: Гистограммы смещённого распределения Вейбулла при различных значениях c

Для генерации матрицы смежности использовались параметры $a = 0.17$, $c = 2.0$, $y_0 = 0.05$. При формировании графа значение степени i -й вершины вычисляется по формуле

$$d_i = \text{round}((p-1)y_i), \quad i = 1, \dots, p,$$

где p — количество вершин графа, y_i — значение случайной величины, полученное по смещённому распределению Вейбулла. Для графа из 10 вершин формула принимает вид

$$d_i = \text{round}(9y_i).$$

Граф строится по полученной последовательности степеней вершин при выполнении условия

$$1 \leq d_i \leq p-1 \quad \text{для всех } i = 1, \dots, p.$$

Если хотя бы одна степень не удовлетворяет этому условию, последовательность генерируется заново. Такой набор параметров подобран

экспериментально. Он обеспечивает умеренные степени вершин и не приводит к формированию почти полного графа.

Гистограмма 1000 значений распределения с выбранными параметрами приведена на рис. 3. На том же рисунке показаны рассчитанные значения степеней для графа из 10 вершин.

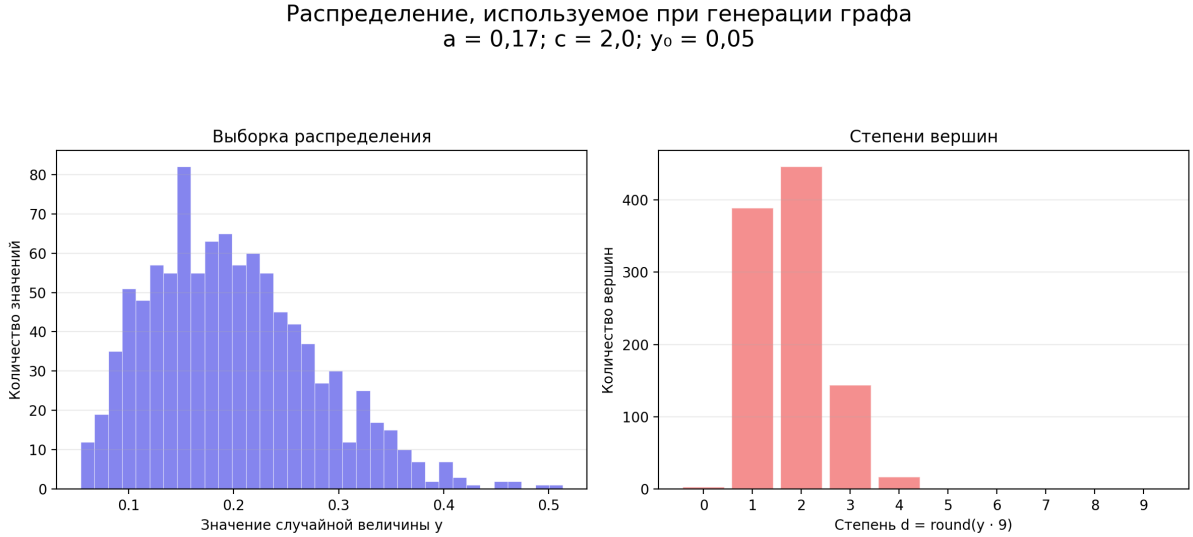


Рис. 3: Распределение Вейбулла и значения степеней при $p = 10$

В рамках работы смещённое распределение Вейбулла используется также для генерации весов рёбер и сети потоков. Для весовой матрицы пользователь может выбрать знак генерируемых весов: положительные, отрицательные или смешанные. При генерации со смешанными весами знак выбирается случайно.

Расстоянием между вершинами u и v (обозначается $d(u, v)$) называется длина кратчайшей цепи $\langle u, v \rangle$, а сама кратчайшая цепь называется геодезической.

$$d(u, v) \stackrel{\text{Def}}{=} \min_{\langle u, v \rangle} \langle u, v \rangle.$$

$$\text{Если } \nexists \langle u, v \rangle, \text{ то } d(u, v) \stackrel{\text{Def}}{=} +\infty.$$

Диаметром графа G называется длиннейшая геодезическая. Длина диаметра обозначается $D(G)$:

$$D(G) \stackrel{\text{Def}}{=} \max_{u, v \in V} d(u, v).$$

Эксцентриситетом $e(v)$ вершины v в связном графе $G(V, E)$ называется максимальное расстояние от v до любой другой вершины графа G :

$$e(v) \stackrel{\text{Def}}{=} \max_{u \in V} d(u, v).$$

Радиусом $R(G)$ графа G называется наименьший из эксцентриситетов вершин:

$$R(G) \stackrel{\text{Def}}{=} \min_{v \in V} e(v).$$

Вершина v называется центральной, если её эксцентриситет совпадает с радиусом графа, т.е. $e(v) = R(G)$.

Множество центральных вершин называется центром графа и обозначается $C(G)$:

$$C(G) \stackrel{\text{Def}}{=} \{v \in V \mid e(v) = R(G)\}.$$

Метод Шимбелла использует степени весовой матрицы для нахождения весов маршрутов с фиксированным количеством рёбер. В данной работе для каждой пары вершин определяется минимальный или максимальный суммарный вес маршрута, содержащего ровно k рёбер.

Пусть $W = (w_{ij})$ — весовая матрица графа. При Шимбелловом умножении произведение элементов заменяется сложением весов, а сложение полученных значений — выбором минимума или максимума. Правила выполнения операций приведены на рис. 4.

1. Операция умножения двух величин a и b при возведении матрицы в степень соответствует их алгебраической сумме, то есть

$$\begin{cases} a \cdot b = b \cdot a \Rightarrow a + b = b + a, \\ a \cdot 0 = 0 \cdot a = 0 \Rightarrow a + 0 = 0. \end{cases} \quad (3.8.1)$$

2. Операция сложения двух величин a и b заменяется выбором из этих величин минимального (максимального) элемента, то есть

$$a + b = b + a = \min(\max) \{a, b\} \quad (3.8.2)$$

нули при этом игнорируются. Минимальный или максимальный элемент выбирается из ненулевых элементов. Нуль в результате операции (3.8.2) может быть получен лишь тогда, когда все элементы из выбираемых — нулевые.

$$\omega_{ij}^{(2)} = \min(\max) \{(\omega_{i1}^{(1)} + \omega_{1j}^{(1)}), (\omega_{i2}^{(1)} + \omega_{2j}^{(1)}), \dots, (\omega_{in}^{(1)} + \omega_{nj}^{(1)})\}$$

Рис. 4: Матричные операции метода Шимбелла

Для произвольного количества рёбер вычисления продолжаютс я рекуррентно. Произведение матриц в следующей формуле является Шимбелловым умножением и выполняется по правилам, приведённым на рисунке 4:

$$W^{(1)} = W, \quad W^{(k)} = W^{(k-1)} \cdot W.$$

Следовательно, элемент $w_{ij}^{(k)}$ результирующей матрицы содержит минимальный или максимальный вес маршрута из v_i в v_j , состоящего ровно из k рёбер. При $k = 0$ диагональные элементы матрицы равны нулю, а остальные элементы соответствуют отсутствующим маршрутам.

2.2 Связность и точки сочленения

Отношение связности вершин называется эквивалентностью. *Классы эквивалентности по отношению связности называются компонентами связности графа.*

Вершина графа называется точкой сочленения, если её удаление увеличивает число компонент связности. Мостом называется ребро, удаление которого увеличивает число компонент связности.

Для поиска точек сочленения граф обходится в глубину. Для каждой вершины определяется, существует ли из её поддерева путь к ранее посещённым вершинам в обход неё. Если такого пути нет, удаление этой вершины разделит граф, поэтому она является точкой сочленения. Для корневой вершины отдельно проверяется количество независимых ветвей обхода. В реализации ориентированный граф рассматривается как неориентированный.

Псевдокод обхода в глубину с вычислением нижних меток приведён на рис. 5.

```

1.  procedure BiComp(v);
2.  begin
3.    num[v] := i; L[v] := i; i := i + 1;
4.    for u ∈ list[v] do
5.      if num[u] = 0 then
6.        begin
7.          SE ← vu; father[u] := v; BiComp(u)
8.          L[v] := min(L[v], L[u]);
9.          if L[u] ≥ num[v] then
10.           {получить новый блок; для этого из
               стека SE надо вытолкнуть все ребра
11.         end
12.       else if num[u] < num[v] and u ≠ father[v] then
13.         begin
14.           SE ← vu; L[v] := min(L[v], num[u]);
15.         end
16.     end;
17.   begin
18.     i := 1; SE := nil; father[v0] := ∅;
19.     for v ∈ V do num[v] := 0;
20.     BiComp(v0)
21.   end.

```

Рис. 5: Псевдокод поиска точек сочленения

Стандартный алгоритм Дейкстры находит кратчайшие пути от начальной вершины s до остальных вершин графа с неотрицательными весами. На каждой итерации выбирается вершина с минимальным текущим весом пути. Затем веса путей до соседних вершин пересчитываются в соответствии с неравенством треугольника. После этого выбранная вершина считается обработанной окончательно. Псевдокод стандартного алгоритма приведён на рис. 6.

Вход: орграф $G(V, E)$, заданный матрицей длин дуг $C : \text{array } [1..p, 1..p] \text{ of real}$;
 s и t — вершины графа.

Выход: векторы $T : \text{array } [1..p] \text{ of real}$; и $H : \text{array } [1..p] \text{ of } 0..p$. Если вершина v лежит на кратчайшем пути от s к t , то $T[v]$ — длина кратчайшего пути от s к v ; $H[v]$ — вершина, непосредственно предшествующая v на кратчайшем пути.

```

for  $v$  from 1 to  $p$  do
   $T[v] := \infty$  { кратчайший путь неизвестен }
   $X[v] := 0$  { все вершины не отмечены }
end for
 $H[s] := 0$  {  $s$  ничего не предшествует }
 $T[s] := 0$  { кратчайший путь имеет длину 0... }
 $X[s] := 1$  { ... и он известен }
 $v := s$  { текущая вершина }
 $M := \{\}$  { обновление пометок }
for  $u \in \Gamma(v)$  do
  if  $X[u] = 0$  &  $T[u] > T[v] + C[v, u]$  then
     $T[u] := T[v] + C[v, u]$  { найден более короткий путь из  $s$  в  $u$  через  $v$  }
     $H[u] := v$  { запоминаем его }
  end if
end for
 $m := \infty$ ;  $v := 0$ 
{ поиск конца кратчайшего пути }
for  $u$  from 1 to  $p$  do
  if  $X[u] = 0$  &  $T[u] < m$  then
     $v := u$ ;  $m := T[u]$  { вершина  $v$  заканчивает кратчайший путь из  $s$  }
  end if
end for
if  $v = 0$  then
  stop { пет пути из  $s$  в  $t$  }
end if
if  $v = t$  then
  stop { найден кратчайший путь из  $s$  в  $t$  }
end if
 $X[v] := 1$  { найден кратчайший путь из  $s$  в  $v$  }
goto  $M$ 

```

Рис. 6: Псевдокод стандартного алгоритма Дейкстры

Для работы с отрицательными весами алгоритм модифицирован: вершина не фиксируется окончательно и при нахождении пути меньшего веса повторно помещается в очередь с приоритетами. Поэтому её вес может уменьшаться несколько раз. Сравнение стандартного и модифицированного алгоритмов приведено на рис. 7.

- Для графов с отрицательными весами (без циклов с суммарным отрицательным весом) время выполнения алгоритма экспоненциальное.

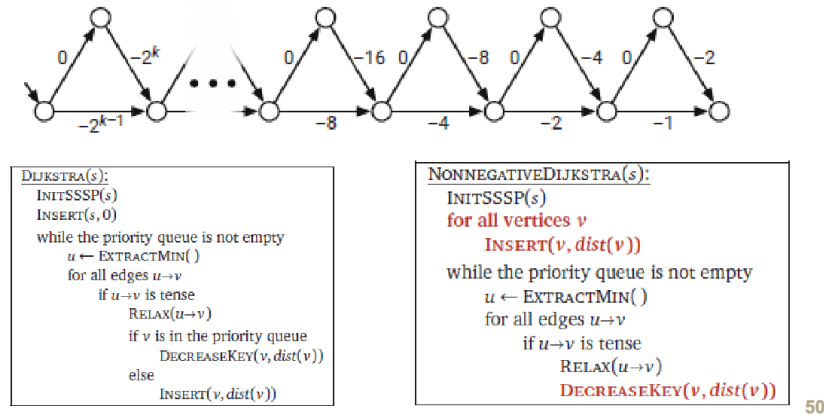


Рис. 7: Сравнение алгоритмов для неотрицательных и отрицательных весов

Для защиты от закливания алгоритм подсчитывает количество уменьшений веса каждой вершины. Если для некоторой вершины оно достигает количества вершин графа p , выполнение прекращается и фиксируется наличие отрицательного цикла. При его отсутствии алгоритм завершается после опустошения очереди. Пример изменения весов приведён на рис. 8 и 9.

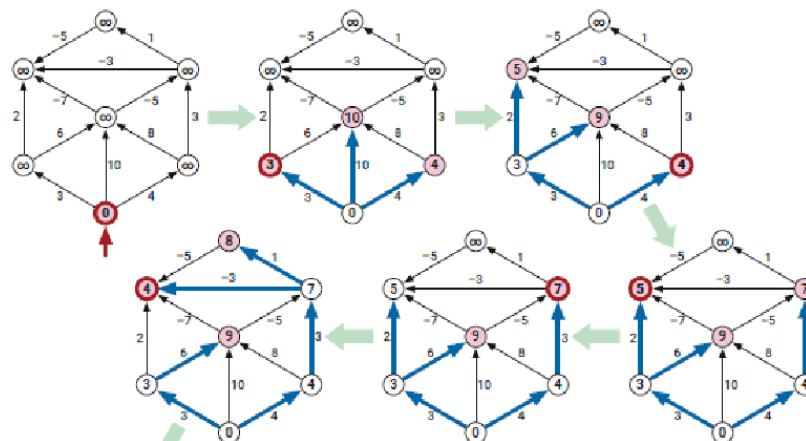


Рис. 8: Последовательное изменение весов при работе алгоритма, часть 1

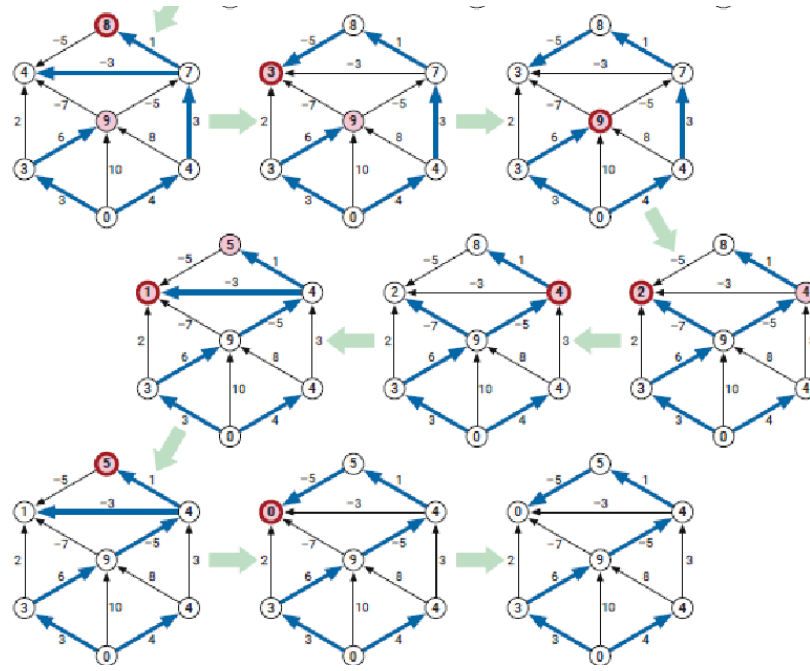


Рис. 9: Последовательное изменение весов при работе алгоритма, часть 2

Результаты сравнения алгоритмов по количеству итераций и соответствующие графики приведены в [разделе «Результаты работы»](#). По результатам запусков поиск точек сочленения оказался эффективнее модифицированного алгоритма Дейкстры.

2.3 Максимальный поток и поток минимальной стоимости

Пусть $G = (V, E)$ — орграф с одним истоком $s \in V$ и одним стоком $t \in V$, где $s \neq t$.

Сетью $S = (G; c)$ называется орграф G с заданной функцией $c : E \rightarrow \mathbb{R}_+$, сопоставляющей каждой дуге $e \in E$ неотрицательное число $c(e)$, называемое пропускной способностью этой дуги e .

Потоком в сети $S = (G; c)$, где $G = (V, E)$, называется такая функция $f : E \rightarrow \mathbb{R}_+$, приписывающая каждой дуге $e \in E$ неотрицательное число $f(e)$, называемое потоком по этой дуге e .

Для поиска максимального потока применяется алгоритм Форда — Фалкерсона. В данной работе он реализован с помощью расстановки меток. Сначала поток по всем дугам полагается равным нулю, а исток получает начальную метку. Далее метки распространяются по дугам, по которым поток можно увеличить, и по обратным дугам, по которым ранее проведённый поток можно уменьшить. Каждая метка хранит предыдущую вершину, направление изменения потока и наименьшую пропускную способность найденной цепи.

Если метка достигает стока, по сохранённым вершинам восстанавливается увеличивающая цепь и поток вдоль неё изменяется на найденную

величину. После этого метки расставляются заново. Когда сток больше нельзя пометить, увеличивающих цепей не остаётся и полученный поток является максимальным. Псевдокод алгоритма приведён на рис. 10 и 11.

Вход: сеть $G(V, E)$ с источником s и стоком t , заданная матрица пропускных способностей $C : \text{array } [1..p, 1..p] \text{ of real}$.
Выход: матрица максимального потока $F : \text{array } [1..p, 1..p] \text{ of real}$.
for $u, v \in V$ **do**
 $F[u, v] := 0$ { вначале поток нулевой }
end for
 $M : \{ \text{итерация увеличения потока} \}$
for $v \in V$ **do**
 $S[v] := 0; N[v] := 0; P[v] := (+, \text{nil}, 0)$ { инициализация }
end for
 $S[s] := 1; P[s] := (+, \text{nil}, \infty)$ { так как $s \in S$ }
repeat
 $a := 0$ { признак расширения S }
 for $v \in V$ **do**
 if $S[v] = 1 \ \& \ N[v] = 0$ **then**
 for $u \in \Gamma(v)$ **do**
 if $S[u] = 0 \ \& \ F[v, u] < C[v, u]$ **then**
 $S[u] := 1; P[u] := (+, v, \min(P[v].\delta, C[v, u] - F[v, u])); a := 1$
 end if
 end for
 end if
 end for
 $N[v] := 1$ { узел v отмечен }
end if
end for
if $S[t]$ **then**
 $x := t$ { текущий узел аугментальной цепи }
 $\delta := P[t].\delta$ { величина изменения потока }
 while $x \neq s$ **do**
 if $P[x].s = +$ **then**
 $F[P[x].n, x] := F[P[x].n, x] + \delta$ { увеличение потока }
 else
 $F[x, P[x].n] := F[x, P[x].n] - \delta$ { увеличение потока }
 end if
 $x := P[x].n$ { предыдущий узел аугментальной цепи }
 end while
 goto M
end if
until $a = 0$

Рис. 10: Псевдокод алгоритма Форда — Фалкерсона, часть 1

end for
for $u \in \Gamma^{-1}(v)$ **do**
 if $S[u] = 0 \ \& \ F[u, v] > 0$ **then**
 $S[u] := 1; P[u] := (-, v, \min(P[v].\delta, F[u, v])); a := 1$
 end if
end for
 $N[v] := 1$ { узел v отмечен }
end if
end for
if $S[t]$ **then**
 $x := t$ { текущий узел аугментальной цепи }
 $\delta := P[t].\delta$ { величина изменения потока }
 while $x \neq s$ **do**
 if $P[x].s = +$ **then**
 $F[P[x].n, x] := F[P[x].n, x] + \delta$ { увеличение потока }
 else
 $F[x, P[x].n] := F[x, P[x].n] - \delta$ { увеличение потока }
 end if
 $x := P[x].n$ { предыдущий узел аугментальной цепи }
 end while
 goto M
end if
until $a = 0$

Рис. 11: Псевдокод алгоритма Форда — Фалкерсона, часть 2

Последовательное увеличение потока на примере сети показано на рис. 12.

Рассмотрим сеть S , и пусть f_0 — нулевой поток.

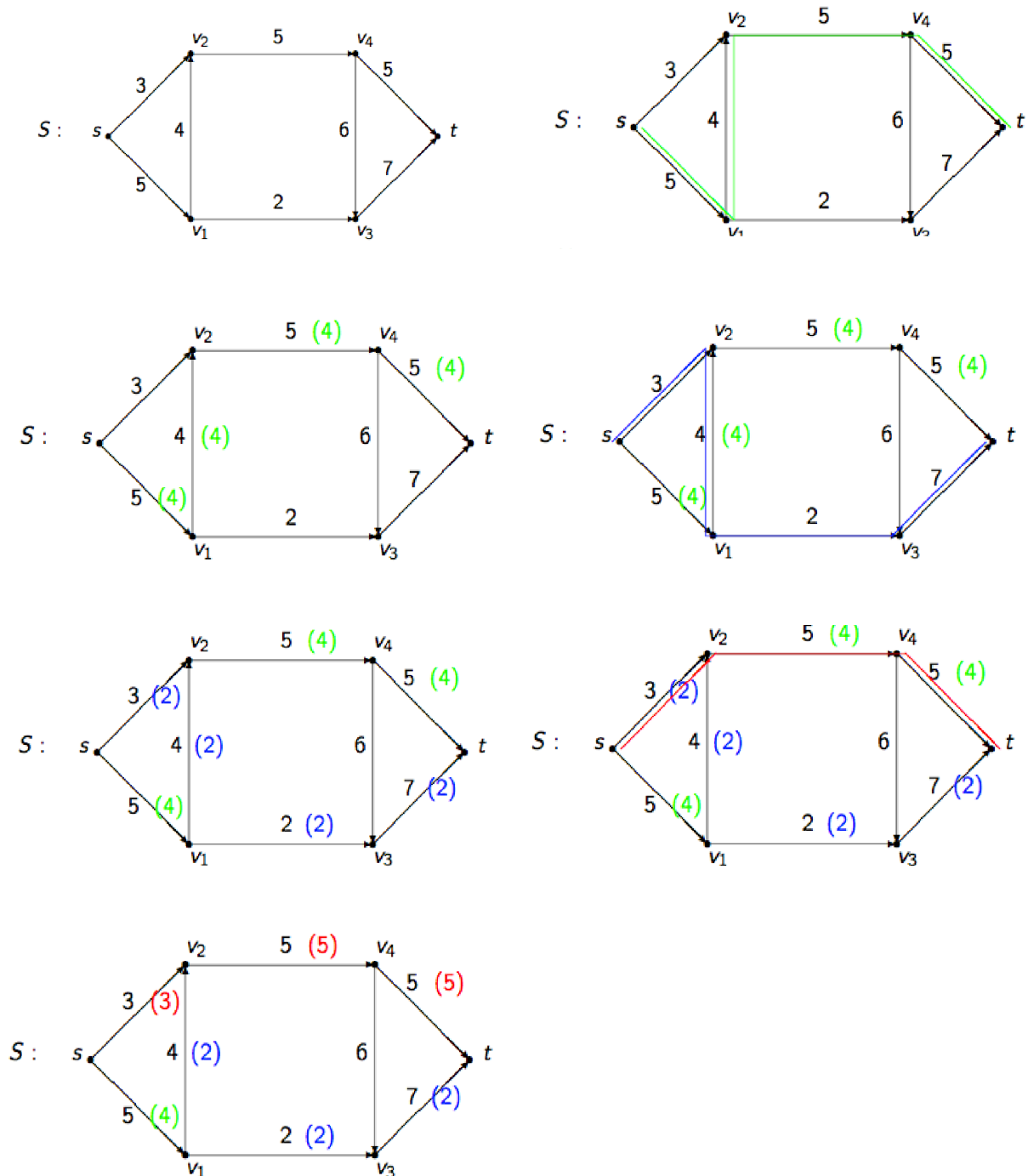


Рис. 12: Последовательное построение максимального потока

Пусть каждой дуге $e \in E$ также поставлена в соответствие стоимость $a(e)$ единицы проходящего по ней потока. Тогда стоимость потока f определяется суммой

$$C(f) = \sum_{e \in E} a(e)f(e).$$

В работе требуется найти поток минимальной стоимости заданной величины

$$F_{\text{req}} = \left\lfloor \frac{2}{3} F_{\text{max}} \right\rfloor,$$

где F_{max} — величина максимального потока, найденная алгоритмом Форда — Фалкерсона. Если $F_{\text{max}} = 1$, принимается $F_{\text{req}} = 1$, чтобы заданная величина потока оставалась положительной.

Поток минимальной стоимости строится в полученной ранее сети. На каждой итерации модифицированным алгоритмом Дейкстры находится путь наименьшей стоимости от истока к стоку. После этого величина потока увеличивается на максимально допустимое значение, определяемое пропускными способностями рёбер найденного пути. Вычисления продолжаются до достижения F_{req} либо до отсутствия такого пути.

2.4 Остовы, код Прюфера и независимые множества рёбер

Граф без циклов называется ациклическим, или лесом. Связный ациклический граф называется свободным деревом.

Пусть $G = (V, E)$ — граф. Остовным подграфом графа G называется подграф, содержащий все вершины графа G . Остовный подграф, являющийся деревом, называется остовом. Поскольку вершины остова совпадают с вершинами исходного графа, остов однозначно задаётся множеством своих рёбер. Несвязный граф не имеет остова, а связный граф может иметь несколько остовов.

Матрицей Кирхгофа графа G называется матрица $B = (b_{ij})$, диагональные элементы которой равны степеням соответствующих вершин, а остальные элементы равны -1 для смежных вершин и нулю в противном случае. По теореме Кирхгофа число остовов связного графа равно алгебраическому дополнению любого элемента матрицы B . Пример вычисления приведён на рис. 13.

- **Теорема Кирхгофа.** Число остовных деревьев в связном графе G порядка $n \geq 2$ равно алгебраическому дополнению любого элемента матрицы Кирхгофа $B(G)$.
- **Н.В.** Все алгебраические дополнения матрицы Кирхгофа равны между собой.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \quad B = \begin{bmatrix} 2 & -1 & 0 & -1 & 0 & 0 \\ -1 & 5 & -1 & -1 & -1 & -1 \\ 0 & -1 & 2 & 0 & -1 & 0 \\ -1 & -1 & 0 & 3 & -1 & 0 \\ 0 & -1 & -1 & -1 & 3 & 0 \\ 0 & -1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_{23} = (-1)^{2+3} \begin{vmatrix} 2 & -1 & -1 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 \\ -1 & -1 & 3 & -1 & 0 \\ 0 & -1 & -1 & 3 & 0 \\ 0 & -1 & 0 & 0 & 1 \end{vmatrix} = (-1)^5 \begin{vmatrix} 2 & -1 & -1 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 \\ -1 & -1 & 3 & -1 & 0 \\ 0 & -1 & -1 & 3 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{vmatrix} = (-1)^{5+5+5} \begin{vmatrix} 2 & -1 & -1 & 0 \\ 0 & -1 & 0 & -1 \\ -1 & -1 & 3 & -1 \\ 0 & -1 & -1 & 3 \end{vmatrix} =$$

$$= (-1)^{15} \begin{vmatrix} 2 & -1 & -1 & 0 \\ 0 & 0 & 0 & -1 \\ -1 & 0 & 3 & -1 \\ 0 & -4 & -1 & 3 \end{vmatrix} = (-1)^{16+4+2} \begin{vmatrix} 2 & -1 & -1 \\ -1 & 0 & 3 \\ 0 & -4 & -1 \end{vmatrix} = (-1)^{22} \begin{vmatrix} 2 & -1 & 5 \\ -1 & 0 & 0 \\ 0 & -4 & -1 \end{vmatrix} = (-1)^{26} \begin{vmatrix} -1 & 5 \\ -4 & -1 \end{vmatrix} = 1 + 20 = 21$$

Рис. 13: Вычисление числа остовов по теореме Кирхгофа

Минимальным остовом взвешенного графа называется остов с наименьшей суммой весов рёбер. Алгоритм Краскала рассматривает рёбра в порядке возрастания их весов. В начале каждая вершина образует отдельное множество. Очередное ребро добавляется в остов, если его концы принадлежат разным множествам, после чего эти множества объединяются. Такая проверка не допускает образования циклов. После выбора $|V| - 1$ ребра получается минимальный остов. Псевдокод алгоритма и пример последовательного построения остова приведены на рис. 14.

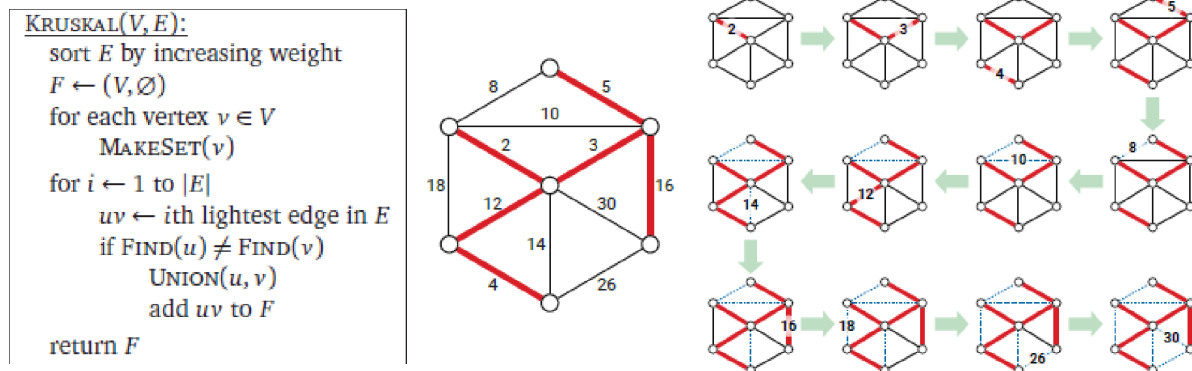


Рис. 14: Алгоритм Краскала и пример его работы

Множество рёбер называется независимым, или паросочетанием, если никакие два его ребра не смежны. Наибольшее число рёбер в независимом множестве рёбер называется рёберным числом независимости и обозначается β_1 .

Вершина, не инцидентная ни одному ребру паросочетания, называется свободной. Увеличивающий путь начинается и заканчивается в свободных вершинах, а его рёбра попеременно не принадлежат и принадлежат паросочетанию. Замена входящих в паросочетание рёбер такого пути на не входящие увеличивает число выбранных рёбер на один.

Алгоритм Эдмондса ищет увеличивающие пути обходом в ширину. Если в дереве поиска обнаруживается цикл нечётной длины, он временно сжимается в одну вершину, после чего поиск продолжается. При нахождении увеличивающего пути сжатые циклы раскрываются, а паросочетание увеличивается. Если увеличивающих путей больше нет, полученное паросочетание является максимальным. Псевдокод алгоритма приведён на рис. 15.

Подробное описание и интерактивная визуализация алгоритма приведены в материале Технического университета Мюнхена [4].

Algorithm status

◀ prev ▶ next ▶▶ fast forward

Explanation Pseudocode

```

BEGIN
  WHILE F != ∅ DO      (*set of free nodes F*)
    pick r ∈ F
    queue.push(r)      (*BFS queue*)
    T ← ∅               (*BFS tree T*)
    T.add(r)
    WHILE queue != ∅
      v ← queue.pop()
      FOR ALL neighbors w of v DO
        IF w ∉ T AND w matched THEN
          T.add(w)
          T.add(mate(w))
          queue.push(mate(w))
        ELSE IF w ∈ T AND even-length
          cycle detected THEN
          CONTINUE
        ELSE IF w ∈ T AND odd-length
          cycle detected THEN
          contract cycle
        ELSE IF w ∈ F THEN
          expand all contracted nodes
          reconstruct augmenting path
          invert augmenting path
      END
    END
  END

```

Рис. 15: Псевдокод алгоритма Эдмондса

2.5 Эйлеровы циклы и фундаментальная система разрезов

Если граф имеет цикл (не обязательно простой), содержащий все рёбра графа, то такой цикл называется эйлеровым циклом, а граф — эйлеровым графом. *Эйлеровым может быть только связный граф.*

Для приведения графа к эйлерову виду сначала находятся вершины нечётной степени. Из них выбираются две вершины u и v . Если они не смежны, ребро (u, v) добавляется. Если ребро существует, оно удаляется только при наличии обходного пути между u и v , что сохраняет связность графа. После каждой операции чётность степеней u и v меняется, поэтому число вершин нечётной степени уменьшается на два. Поскольку таких вершин всегда чётное число, процесс завершается за конечное число шагов.

Для построения эйлерова цикла применяется алгоритм Флёри. Начальная вершина помещается в стек. Из текущей вершины выбирается соседняя вершина, после чего соединяющее их ребро удаляется, а выбранная вершина добавляется в стек. Переходы продолжаются, пока у текущей вершины не останется соседей. Такая вершина добавляется в результат и удаляется из стека, после чего обход продолжается из предыдущей вершины. После опустошения стека получается эйлеров цикл. Материал по построению эйлеровых циклов приведён на сайте Технического университета Мюнхена [5]. Псевдокод алгоритма приведён на рис. 16.

```
Вход: эйлеров граф  $G(V, E)$ , заданный списками смежности ( $\Gamma[v]$  — список вершин, смежных с вершиной  $v$ ).  
Выход: последовательность вершин эйлерова цикла.  
 $S := \emptyset$  { стек для хранения вершин }  
select  $v \in V$  { произвольная вершина }  
 $v \rightarrow S$  { положить  $v$  в стек  $S$  }  
while  $S \neq \emptyset$  do  
   $v := \text{top } S$  {  $v$  — верхний элемент стека }  
  if  $\Gamma[v] = \emptyset$  then  
     $v \leftarrow S$ ; yield  $v$  { очередная вершина эйлерова цикла }  
  else  
    select  $u \in \Gamma[v]$  { взять первую вершину из списка смежности }  
     $u \rightarrow S$  { положить  $u$  в стек }  
     $\Gamma[v] := \Gamma[v] - u$ ;  $\Gamma[u] := \Gamma[u] - v$  { удалить ребро  $(v, u)$  }  
  end if  
end while
```

Если реализовать поиск ребер инцидентных вершине и удаление ребер за $O(1)$, то алгоритм будет работать за $O(p)$.

Рис. 16: Псевдокод алгоритма Флёри

Пусть $T(V, E_T)$ — некоторый остов графа $G(V, E)$. Рассмотрим ребро остова e и определим разрез S_e следующим образом.

Ребро e — мост в дереве T . Следовательно, удаление ребра e разбивает множество вершин V на два непустых подмножества V_1 и V_2 , так

что

$$V_1 \in V, V_1 \neq \emptyset, V_2 \in V, V_2 \neq \emptyset, V_1 \cup V_2 = V, V_1 \cap V_2 = \emptyset.$$

Включим в разрезе S_e ребро e и все хорды остова T , соединяющие вершины множества V_1 с вершинами множества V_2 , тогда S_e — простой разрез:

$$S_e \stackrel{\text{Def}}{=} e + \{(v_1, v_2) \in T \mid v_1 \in V_1 \ \& \ v_2 \in V_2\} = E(V_1, V_2).$$

Система разрезов называется фундаментальной системой разрезов. Разрезы фундаментальной системы называются фундаментальными, а количество разрезов в фундаментальной системе называется коциклическим рангом графа G и обозначается $m^*(G)$.

Рассмотрим векторное пространство над двоичной арифметикой, натянутое на множество рёбер E графа $G(V, E)$. Элемент этого векторного пространства (линейную комбинацию) можно отождествлять с подмножеством рёбер: если коэффициент в линейной комбинации равен 1, то ребро входит в подмножество, если коэффициент равен 0, то не входит.

Сложению векторов соответствует симметрическая разность множеств рёбер. Множество правильных разрезов замкнуто относительно симметрической разности, поэтому оно является подпространством общего пространства рёбер графа.

Множество разрезов называется независимым, если ни один из разрезов не является линейной комбинацией остальных.

Максимальное независимое множество разрезов является базисом подпространства разрезов. Фундаментальная система разрезов является базисом подпространства разрезов. Разрезы любой фундаментальной системы независимы.

Для построения фундаментальной системы сначала алгоритмом Краскала строится остов T . Затем из остова исключается ребро e , в результате чего остов разделяется на две части. В разрез S_e включаются исключённое ребро и все хорды, соединяющие вершины разных частей. Эти действия повторяются для каждого ребра остова, то есть $p - 1$ раз. Полученные разрезы образуют фундаментальную систему.

3 Особенности реализации

Основные алгоритмы реализованы методами класса `GeneratorGraph`. Взаимодействие с пользователем организовано классом `CLI` через общее консольное меню.

Далее через p обозначается количество вершин графа, через q — количество его рёбер, а через k — заданное количество рёбер маршрута.

3.1 Лабораторная работа № 1

Алгоритм генерации графа реализован методом `generate`, приведённым в листинге 1. Он формирует связный ориентированный граф по последовательности степеней, полученной с помощью распределения Вейбулла.

Вход: количество вершин графа.

Выход: матрицы смежности ориентированного и неориентированного графов.

Сложность: $O(p^2)$.

Листинг 1: Генерация графа

```
1 void GeneratorGraph::generate() {
2     m_adjacencyMatrix = Matrix(m_vertexCount, m_vertexCount);
3     auto degrees = m_generateDegreeSequence();
4
5     // гарантирует связность
6     for (int i = 0; i < m_vertexCount - 1; ++i) {
7         m_adjacencyMatrix(i, i + 1) = 1;
8     }
9
10    // добор оставшихся рёбер
11    for (int i = 0; i < m_vertexCount; ++i) {
12        int alreadyAdded = (i < m_vertexCount - 1) ? 1 : 0;
13
14        std::vector<int> candidates;
15        for (int j = i + 1; j < m_vertexCount; ++j) {
16            // если ещё не добавлено
17            if (m_adjacencyMatrix(i, j) == 0) {
18                candidates.push_back(j);
19            }
20        }
21        std::shuffle(candidates.begin(), candidates.end(), m_rng);
22
23        int needToAdd = std::max(0, degrees[i] - alreadyAdded);
24        int edgesAdded = 0;
25        for (int j : candidates) {
26            if (edgesAdded >= needToAdd) break;
27            m_adjacencyMatrix(i, j) = 1;
28            ++edgesAdded;
```



```

29     }
30 }
31
32 m_syncUndirectedAdjacencyMatrix();
33 isMatrixInit.adjacency = true;
34 }

```

Алгоритм вычисления эксцентриситетов реализован методом `computeEccentricities`, приведённым в листинге 2. По степеням матрицы смежности он находит длины кратчайших маршрутов и вычисляет характеристики вершин.

Вход: матрица смежности графа.

Выход: эксцентриситеты, диаметр, центральные и диаметральные вершины.

Сложность: $O(p^4)$, поскольку выполняется до $p - 1$ умножений матриц.

Листинг 2: Вычисление эксцентриситетов, центра и диаметра

```

1 void GeneratorGraph::computeEccentricities() {
2     const Matrix& A = m_adjacencyMatrix;
3
4     const int UNREACHABLE = -1;
5     Matrix dist(m_vertexCount, m_vertexCount, UNREACHABLE);
6     for (int i = 0; i < m_vertexCount; ++i) dist(i, i) = 0;
7
8     Matrix Ak = A;
9     for (int k = 1; k < m_vertexCount; ++k) {
10         for (int i = 0; i < m_vertexCount; ++i) {
11             for (int j = 0; j < m_vertexCount; ++j) {
12                 if (i != j && dist(i, j) == UNREACHABLE && Ak(i, j) > 0)
13                     ↪ {
14                         dist(i, j) = k;
15                     }
16             }
17             if (k < m_vertexCount - 1) Ak = Ak * A;
18         }
19
20         m_eccentricities.assign(m_vertexCount, 0);
21         for (int i = 0; i < m_vertexCount; ++i) {
22             int maxDist = 0;
23             for (int j = 0; j < m_vertexCount; ++j) {
24                 if (i != j && dist(i, j) != UNREACHABLE) {
25                     maxDist = std::max(maxDist, static_cast<int>(dist(i,
26                     ↪ j)));
27                 }
28             }
29             m_eccentricities[i] = maxDist;
30         }
31     }
32 }

```

```

31     m_diameter = *std::max_element(m_eccentricities.begin(),
    ↪ m_eccentricities.end());
32     int radius = *std::min_element(m_eccentricities.begin(),
    ↪ m_eccentricities.end());
33
34     m_centerVertices.clear();
35     m_diametralVertices.clear();
36     for (int i = 0; i < m_vertexCount; ++i) {
37         if (m_eccentricities[i] == radius)
    ↪ m_centerVertices.push_back(i);
38         if (m_eccentricities[i] == m_diameter)
    ↪ m_diametralVertices.push_back(i);
39     }
40 }

```

Метод `generateWeightMatrix`, приведённый в листинге 3, присваивает существующим рёбрам веса в выбранном режиме.

Вход: матрица смежности и режим генерации положительных, отрицательных или смешанных весов.

Выход: весовые матрицы ориентированного и неориентированного графов.

Листинг 3: Генерация весовой матрицы

```

1 void GeneratorGraph::generateWeightMatrix(WeightMode mode) {
2     const double INF = std::numeric_limits<double>::infinity();
3
4     m_weightMatrix = Matrix(m_vertexCount, m_vertexCount, INF);
5
6     std::uniform_int_distribution<int> signDist(0, 1);
7
8     for (int i = 0; i < m_vertexCount; ++i) {
9         m_weightMatrix(i, i) = 0;
10
11         for (int j = 0; j < m_vertexCount; ++j) {
12             if (i == j) continue;
13
14             if (m_adjacencyMatrix(i, j) != 0) {
15                 const int WEIGHT_MATRIX_DIST_VALUE = 10;
16                 double w = std::round(m_sampleWeibull(constants::weight )
    ↪ * WEIGHT_MATRIX_DIST_VALUE);
17
18                 switch (mode) {
19                     case WeightMode::Positive:
20                         m_weightMatrix(i, j) = w;
21                         break;
22
23                     case WeightMode::Negative:
24                         m_weightMatrix(i, j) = -w;
25                         break;
26

```

```

27         case WeightMode::Mixed:
28             m_weightMatrix(i, j) = signDist(m_rng) ? w : -w;
29             break;
30     }
31 }
32 }
33 }
34
35 isMatrixInit.weight = true;
36 m_syncUndirectedWeightMatrix();
37 }

```

Метод Шимбелла реализован методом `shimbell`, приведённым в листинге 4. Он последовательно строит весовую матрицу маршрутов заданной длины.

Вход: матрицы смежности и весов, количество рёбер маршрута и требуемый тип веса: минимальный либо максимальный.

Выход: матрица весов маршрутов заданной длины.

Сложность: $O(kp^3)$.

Листинг 4: Реализация метода Шимбелла

```

1 Matrix GeneratorGraph::shimbell(int steps, bool findMin) const {
2     const double NO_EDGE = findMin
3         ? std::numeric_limits<double>::infinity()
4         : -std::numeric_limits<double>::infinity();
5
6     Matrix W(m_vertexCount, m_vertexCount, NO_EDGE);
7
8     if (steps == 0) {
9         for (int i = 0; i < m_vertexCount; ++i) {
10             W(i, i) = 0;
11         }
12         return W;
13     }
14
15     for (int i = 0; i < m_vertexCount; ++i) {
16         for (int j = 0; j < m_vertexCount; ++j) {
17             if (m_adjacencyMatrix(i, j) != 0) {
18                 W(i, j) = m_weightMatrix(i, j);
19             }
20         }
21     }
22
23     Matrix result = W;
24
25     for (int s = 1; s < steps; ++s) {
26         result = findMin ? result.shimbellMin(W) : result.shimbellMax(W);
27     }
28 }

```

```

29     for (int i = 0; i < m_vertexCount; ++i) {
30         result(i, i) = 0;
31     }
32
33     return result;
34 }

```

Алгоритм исследования маршрутов реализован методами `hasRoute` и `countRoutes`, приведёнными в листинге 5. Методы используют степени матрицы смежности для исследования маршрутов между двумя вершинами.

Вход: матрица смежности, начальная и конечная вершины.

Выход: признак существования маршрута и количество найденных маршрутов.

Сложность: $O(p^4)$.

Листинг 5: Проверка наличия и подсчёт маршрутов

```

1  bool GeneratorGraph::hasRoute(int from, int to) const {
2      if (from == to) return true;
3
4      Matrix Ak = m_adjacencyMatrix;
5      for (int k = 1; k < m_vertexCount; ++k) {
6          if (Ak(from, to) > 0) return true;
7          if (k < m_vertexCount - 1) Ak = Ak * m_adjacencyMatrix;
8      }
9      return false;
10 }
11
12 int GeneratorGraph::countRoutes(int from, int to) const {
13     if (from == to) return 1;
14
15     int total = 0;
16     Matrix Ak = m_adjacencyMatrix;
17     for (int k = 1; k < m_vertexCount; ++k) {
18         total += static_cast<int>(Ak(from, to));
19         if (k < m_vertexCount - 1) Ak = Ak * m_adjacencyMatrix;
20     }
21     return total;
22 }

```

3.2 Лабораторная работа № 2

Алгоритм поиска точек сочленения реализован методами `m_dfsArticulation` и `findArticulationPoints`, приведёнными в листинге 6. Они выполняют обход графа в глубину, вычисляют время входа и нижние метки вершин.

Вход: матрица смежности.

Выход: список точек сочленения и количество итераций алгоритма.

Сложность: $O(p^2)$ при хранении графа в виде матрицы смежности.

Листинг 6: Поиск точек сочленения

```
1 void GeneratorGraph::m_dfsArticulation(int v,
2                                     int parent,
3                                     std::vector<int>& tin,
4                                     std::vector<int>& low,
5                                     std::vector<bool>& visited,
6                                     std::vector<bool>& isAP,
7                                     int& timer,
8                                     int& iterations) const {
9     visited[v] = true;
10    tin[v] = low[v] = timer++;
11
12    int children = 0;
13
14    for (int u = 0; u < m_vertexCount; ++u) {
15        iterations++;
16
17        if (u == v) continue;
18
19        if (m_adjacencyMatrix(v, u) == 0 && m_adjacencyMatrix(u, v) == 0)
20            ↪ {
21                continue;
22            }
23
24        if (u == parent) {
25            continue;
26        }
27
28        if (visited[u]) {
29            low[v] = std::min(low[v], tin[u]);
30        } else {
31            ++children;
32
33            m_dfsArticulation(u, v, tin, low, visited, isAP, timer,
34                            ↪ iterations);
35
36            low[v] = std::min(low[v], low[u]);
37
38            if (parent != -1 && low[u] >= tin[v]) {
39                isAP[v] = true;
40            }
41        }
42    }
43
44    if (parent == -1 && children > 1) {
45        isAP[v] = true;
46    }
```

```

47 // упрощенный упрощённый алгоритм Тарьяна (лек. 4 стр. 11 - полный)
48 // не использует стек рёбер
49 // квадратичная сложность из-за использования матрицы смежности, со
  ↳ списками сложность  $O(V + E)$ 
50 ArticulationPointsResult GeneratorGraph::findArticulationPoints() const {
51     std::vector<int> tin(m_vertexCount, -1);
52     std::vector<int> low(m_vertexCount, -1);
53     std::vector<bool> visited(m_vertexCount, false);
54     std::vector<bool> isAP(m_vertexCount, false);
55
56     int timer = 0;
57     ArticulationPointsResult result;
58
59     for (int v = 0; v < m_vertexCount; ++v) {
60         if (!visited[v]) {
61             m_dfsArticulation(v, -1, tin, low, visited, isAP, timer,
  ↳ result.iterations);
62         }
63     }
64
65     for (int v = 0; v < m_vertexCount; ++v) {
66         if (isAP[v]) {
67             result.points.push_back(v);
68         }
69     }
70
71     return result;
72 }

```

Модифицированный алгоритм Дейкстры реализован методом `dijkstraNegative`, приведённым в листинге 7. Он выбирает из очереди вершину с минимальным текущим весом, допускает её повторную обработку и контролирует число уменьшений веса для обнаружения отрицательного цикла.

Вход: матрица смежности, весовая матрица, начальная и конечная вершины.

Выход: вектор расстояний, предшественники вершин, кратчайший путь, его длина, число итераций и признак отрицательного цикла.

Сложность: $O(e^p)$ в худшем случае.

Листинг 7: Поиск кратчайшего пути при наличии отрицательных весов

```

1 ShortestPathResult GeneratorGraph::dijkstraNegative(
2     int s,
3     int t,
4     const Matrix& adjacency,
5     const Matrix& weight
6 ) const {
7     const double INF = std::numeric_limits<double>::infinity();
8

```

```

9     ShortestPathResult result;
10
11     // T[v] - длина кратчайшего пути от s к v
12     // H[v] - вершина, предшествующая v по кратчайшему пути
13
14     std::vector<double> T(m_vertexCount, INF);
15     std::vector<int> H(m_vertexCount, -1);
16
17     std::vector<int> relaxCount(m_vertexCount, 0);
18
19     // { T[v], v }
20     using QueueItem = std::pair<double, int>;
21
22     // очередь с приоритетами
23     // сначала извлекается вершина с минимальным T[v]
24     std::priority_queue<
25         QueueItem,
26         std::vector<QueueItem>,
27         std::greater<QueueItem>
28     > Q;
29
30     int iterations = 0;
31
32     T[s] = 0;
33     H[s] = -1;
34
35     Q.push({0, s});
36
37     while (!Q.empty()) {
38         double currentDistance = Q.top().first;
39         int v = Q.top().second;
40
41         Q.pop();
42
43         // пропускаем устаревшую запись
44         if (currentDistance != T[v]) {
45             continue;
46         }
47
48         for (int u = 0; u < m_vertexCount; ++u) {
49             ++iterations;
50
51             if (adjacency(v, u) == 0) {
52                 continue;
53             }
54
55             // по нер-ву треугольника
56             if (T[v] != INF && T[u] > T[v] + weight(v, u)) {
57                 T[u] = T[v] + weight(v, u);
58                 H[u] = v;
59
60                 Q.push({T[u], u});
61

```

```

62         ++relaxCount[u];
63
64         // выходим, если пошли по циклу
65         if (relaxCount[u] >= m_vertexCount) {
66             result.hasNegativeCycle = true;
67             break;
68         }
69     }
70 }
71
72     if (result.hasNegativeCycle) {
73         break;
74     }
75 }
76
77 result.T = T;
78 result.H = H;
79
80 result.iterations = iterations;
81
82 result.hasPath = T[t] != INF;
83 result.distance = T[t];
84
85 if (result.hasPath && !result.hasNegativeCycle) {
86     result.path = m_restorePath(s, t, H);
87 }
88
89 return result;
90 }

```

3.3 Лабораторная работа № 3

Метод `generateCapacityAndCostMatrices`, приведённый в листинге 8, назначает каждому ребру исходного ориентированного графа пропускную способность и стоимость.

Вход: матрица смежности ориентированного графа.

Выход: матрицы пропускных способностей и стоимостей.

Листинг 8: Генерация пропускных способностей и стоимостей

```

1 void GeneratorGraph::generateCapacityAndCostMatrices() {
2     m_capacityMatrix = Matrix(m_vertexCount, m_vertexCount, 0);
3     m_costMatrix = Matrix(m_vertexCount, m_vertexCount, 0);
4
5     for (int i = 0; i < m_vertexCount; ++i) {
6         for (int j = 0; j < m_vertexCount; ++j) {
7             if (m_adjacencyMatrix(i, j) != 0) {
8                 double capacityRow =
9                     ↪ m_sampleWeibull(constants::capacity);
10                double costRow = m_sampleWeibull(constants::cost);

```



```

11         int capacity = std::max(1,
    ↪ static_cast<int>(std::round(capacityRaw * 10)));
12         int cost = std::max(1,
    ↪ static_cast<int>(std::round(costRaw * 10)));
13
14         m_capacityMatrix(i, j) = capacity;
15         m_costMatrix(i, j) = cost;
16     }
17 }
18 }
19
20 isMatrixInit.capacity = true;
21 isMatrixInit.cost = true;
22 }

```

Алгоритм Форда — Фалкерсона реализован методом `fordFulkerson`, приведённым в листинге 9. Он расставляет метки, находит увеличивающую цепь и изменяет поток вдоль неё, пока сток может получить метку.

Вход: матрица пропускных способностей, исток и сток.

Выход: величина максимального потока, матрица потоков и количество итераций.

Сложность: $O(Ap^3)$, где A — количество найденных увеличивающих цепей.

Листинг 9: Вычисление максимального потока

```

1 MaxFlowResult GeneratorGraph::fordFulkerson(int source, int sink) const {
2     const int INF = std::numeric_limits<int>::max();
3
4     MaxFlowResult result(m_vertexCount);
5
6     if (!isMatrixInit.capacity) {
7         return result;
8     }
9
10    if (source < 0 || source >= m_vertexCount ||
11        sink < 0 || sink >= m_vertexCount ||
12        source == sink) {
13        return result;
14    }
15
16    struct Mark {
17        char sign = '+';
18        int parent = -1;
19        int delta = 0;
20    };
21
22    while (true) {
23        // S[v] - вершина получила пометку
24        // N[v] - вершина уже обработана

```

```

25      // P[v] - пометка вершины: знак, предок, величина изменения
      ↪ потока
26      std::vector<bool> S(m_vertexCount, false);
27      std::vector<bool> N(m_vertexCount, false);
28      std::vector<Mark> P(m_vertexCount);
29
30      S[source] = true;
31      P[source] = {'+', -1, INF};
32
33      bool expanded = true;
34
35      // расставляем пометки, пока множество S расширяется
36      while (expanded && !S[sink]) {
37          expanded = false;
38
39          for (int v = 0; v < m_vertexCount; ++v) {
40              ++result.iterations;
41
42              if (!S[v] || N[v]) {
43                  continue;
44              }
45
46              // прямые дуги
47              for (int u = 0; u < m_vertexCount; ++u) {
48                  ++result.iterations;
49
50                  if (S[u]) {
51                      continue;
52                  }
53
54                  if (m_capacityMatrix(v, u) <= 0) {
55                      continue;
56                  }
57
58                  int available =
59                      static_cast<int>(m_capacityMatrix(v, u) -
                      ↪ result.flowMatrix(v, u));
60
61                  if (available > 0) {
62                      S[u] = true;
63                      P[u] = {
64                          '+',
65                          v,
66                          std::min(P[v].delta, available)
67                      };
68
69                      expanded = true;
70
71                      if (u == sink) {
72                          break;
73                      }
74                  }
75              }

```

```

76
77         if (S[sink]) {
78             break;
79         }
80
81         // обратные дуги
82         for (int u = 0; u < m_vertexCount; ++u) {
83             ++result.iterations;
84
85             if (S[u]) {
86                 continue;
87             }
88
89             if (result.flowMatrix(u, v) > 0) {
90                 int available =
91                     static_cast<int>(result.flowMatrix(u, v));
92
93                 S[u] = true;
94                 P[u] = {
95                     '-',
96                     v,
97                     std::min(P[v].delta, available)
98                 };
99
100                 expanded = true;
101
102                 if (u == sink) {
103                     break;
104                 }
105             }
106         }
107
108         N[v] = true;
109
110         if (S[sink]) {
111             break;
112         }
113     }
114 }
115
116 // если сток не получил пометку, увеличивающих цепей больше нет
117 if (!S[sink]) {
118     break;
119 }
120
121 int delta = P[sink].delta;
122 int current = sink;
123
124 // изменяем поток по найденной увеличивающей цепи
125 while (current != source) {
126     Mark mark = P[current];
127     int previous = mark.parent;
128

```

```

129         if (mark.sign == '+') {
130             result.flowMatrix(previous, current) += delta;
131         } else {
132             result.flowMatrix(current, previous) -= delta;
133         }
134
135         current = previous;
136     }
137
138     result.maxFlow += delta;
139 }
140
141 return result;
142 }
143
144 // поток минимальной стоимости
145 // заданная величина потока передаётся параметром requiredFlow

```

Алгоритм последовательного нахождения потока минимальной стоимости реализован методом `minCostFlow`, приведённым в листинге 10. Он находит кратчайшие пути в сети и изменяет поток до достижения требуемой величины либо отсутствия подходящего пути.

Вход: матрицы пропускных способностей и стоимостей, исток, сток и требуемая величина потока.

Выход: достигнутая величина и стоимость потока, матрицы потоков и стоимостей потока, число итераций и признак успешного завершения.

Сложность: $O(Ae^p)$ в худшем случае, где A — количество увеличений потока. Экспоненциальная сложность обусловлена использованием модифицированного алгоритма Дейкстры при каждом увеличении потока.

Листинг 10: Вычисление потока минимальной стоимости

```

1  MinCostFlowResult GeneratorGraph::minCostFlow(int source,
2                                              int sink,
3                                              int requiredFlow) const {
4      MinCostFlowResult result(m_vertexCount);
5
6      result.requiredFlow = requiredFlow;
7
8      if (!isMatrixInit.capacity || !isMatrixInit.cost) {
9          return result;
10     }
11
12     if (source < 0 || source >= m_vertexCount ||
13         sink < 0 || sink >= m_vertexCount ||
14         source == sink ||
15         requiredFlow <= 0) {
16         return result;
17     }
18

```

```

19 Matrix residualCapacity = m_capacityMatrix;
20 Matrix residualCost(m_vertexCount, m_vertexCount, 0);
21
22 for (int i = 0; i < m_vertexCount; ++i) {
23     for (int j = 0; j < m_vertexCount; ++j) {
24         if (m_capacityMatrix(i, j) > 0) {
25             residualCost(i, j) = m_costMatrix(i, j);
26             residualCost(j, i) = -m_costMatrix(i, j);
27         }
28     }
29 }
30
31 while (result.achievedFlow < requiredFlow) {
32     Matrix residualAdjacency(m_vertexCount, m_vertexCount, 0);
33
34     for (int i = 0; i < m_vertexCount; ++i) {
35         for (int j = 0; j < m_vertexCount; ++j) {
36             if (residualCapacity(i, j) > 0) {
37                 residualAdjacency(i, j) = 1;
38             }
39         }
40     }
41
42     auto pathResult = dijkstraNegative(
43         source,
44         sink,
45         residualAdjacency,
46         residualCost
47     );
48
49     result.iterations += pathResult.iterations;
50
51     if (!pathResult.hasPath || pathResult.hasNegativeCycle) {
52         break;
53     }
54
55     int addFlow = requiredFlow - result.achievedFlow;
56
57     for (size_t i = 1; i < pathResult.path.size(); ++i) {
58         int from = pathResult.path[i - 1];
59         int to = pathResult.path[i];
60
61         addFlow = std::min(
62             addFlow,
63             static_cast<int>(residualCapacity(from, to))
64         );
65     }
66
67     if (addFlow <= 0) {
68         break;
69     }
70
71     for (size_t i = 1; i < pathResult.path.size(); ++i) {

```

```

72         int from = pathResult.path[i - 1];
73         int to = pathResult.path[i];
74
75         residualCapacity(from, to) -= addFlow;
76         residualCapacity(to, from) += addFlow;
77
78         if (m_capacityMatrix(from, to) > 0) {
79             // идём по исходной дуге
80             result.flowMatrix(from, to) += addFlow;
81             result.totalCost += addFlow *
82                 ↪ static_cast<int>(m_costMatrix(from, to));
83         } else {
84             // идём по обратной дуге остаточной сети
85             // значит уменьшаем поток по исходной дуге to -> from
86             result.flowMatrix(to, from) -= addFlow;
87             result.totalCost -= addFlow *
88                 ↪ static_cast<int>(m_costMatrix(to, from));
89         }
90     }
91     result.achievedFlow += addFlow;
92 }
93 result.success = result.achievedFlow == requiredFlow;
94
95 for (int i = 0; i < m_vertexCount; ++i) {
96     for (int j = 0; j < m_vertexCount; ++j) {
97         result.costFlowMatrix(i, j) =
98             m_costMatrix(i, j) * result.flowMatrix(i, j);
99     }
100 }
101
102 return result;
103 }

```

3.4 Лабораторная работа № 4

Алгоритм подсчёта остовных деревьев по теореме Кирхгофа реализован методом `countSpanningTreesKirchhoff`, приведённым в листинге 11. Он строит матрицу Лапласа и вычисляет определитель её минора.

Вход: матрица смежности неориентированного графа.

Выход: матрица Лапласа, её минор и число остовных деревьев.

Сложность: $O(p!)$, поскольку определитель вычисляется рекурсивным разложением по строке.

Листинг 11: Подсчёт остовных деревьев по теореме Кирхгофа

```

1 SpanningTreesResult GeneratorGraph::countSpanningTreesKirchhoff() const {
2     SpanningTreesResult result(m_vertexCount);
3     const Matrix& adjacency = m_undirectedAdjacencyMatrix;

```

```

4
5     for (int i = 0; i < m_vertexCount; ++i) {
6         int degree = 0;
7
8         for (int j = 0; j < m_vertexCount; ++j) {
9             if (i == j) {
10                 continue;
11             }
12
13             if (adjacency(i, j) != 0) {
14                 ++degree;
15                 result.kirchhoffMatrix(i, j) = -1;
16             }
17         }
18
19         result.kirchhoffMatrix(i, i) = degree;
20     }
21
22     if (m_vertexCount <= 1) {
23         result.count = 1;
24         return result;
25     }
26
27     for (int i = 1; i < m_vertexCount; ++i) {
28         for (int j = 1; j < m_vertexCount; ++j) {
29             result.cofactorMatrix(i - 1, j - 1) =
30                 result.kirchhoffMatrix(i, j);
31         }
32     }
33
34     result.cofactorDeterminant = result.cofactorMatrix.determinant();
35     result.count = std::llround(result.cofactorDeterminant);
36
37     return result;
38 }

```

Определитель вычисляется методом `determinant`, приведённым в листинге 12. Метод проверяет, что матрица является квадратной, и вызывает рекурсивное разложение, приведённое в листинге 13. Для сокращения количества слагаемых выбирается строка или столбец с наибольшим числом нулевых элементов.

Вход: квадратная матрица.

Выход: определитель матрицы.

Сложность: $O(p!)$ в худшем случае.

Листинг 12: Метод вычисления определителя

```

1 double Matrix::determinant() const {
2     if (m_rows != m_cols) {
3         throw std::invalid_argument(
4             "determinant is defined only for square matrices"

```

```

5         );
6     }
7
8     return determinantRecursive(m_data);
9 }
10

```

Листинг 13: Рекурсивное разложение определителя

```

1
2 std::vector<std::vector<double>> buildMinor(
3     const std::vector<std::vector<double>>& data,
4     int excludedRow,
5     int excludedCol
6 ) {
7     const int n = static_cast<int>(data.size());
8     std::vector<std::vector<double>> minor;
9     minor.reserve(n - 1);
10
11     for (int i = 0; i < n; ++i) {
12         if (i == excludedRow) {
13             continue;
14         }
15
16         std::vector<double> row;
17         row.reserve(n - 1);
18
19         for (int j = 0; j < n; ++j) {
20             if (j != excludedCol) {
21                 row.push_back(data[i][j]);
22             }
23         }
24
25         minor.push_back(row);
26     }
27
28     return minor;
29 }
30
31 double determinantRecursive(
32     const std::vector<std::vector<double>>& data
33 ) {
34     const int n = static_cast<int>(data.size());
35
36     if (n == 0) {
37         return 1;
38     }
39
40     if (n == 1) {
41         return data[0][0];
42     }
43
44     if (n == 2) {
45         return data[0][0] * data[1][1]
46             - data[0][1] * data[1][0];
47     }
48
49     bool expandByRow = true;
50     int expandIndex = 0;
51     int maxZeros = -1;
52

```



```

53     for (int i = 0; i < n; ++i) {
54         int zeros = 0;
55
56         for (int j = 0; j < n; ++j) {
57             if (isZero(data[i][j])) {
58                 ++zeros;
59             }
60         }
61
62         if (zeros > maxZeros) {
63             maxZeros = zeros;
64             expandByRow = true;
65             expandIndex = i;
66         }
67     }
68
69     for (int j = 0; j < n; ++j) {
70         int zeros = 0;
71
72         for (int i = 0; i < n; ++i) {
73             if (isZero(data[i][j])) {
74                 ++zeros;
75             }
76         }
77
78         if (zeros > maxZeros) {
79             maxZeros = zeros;
80             expandByRow = false;
81             expandIndex = j;
82         }
83     }
84
85     if (maxZeros == n) {
86         return 0;
87     }
88
89     double determinant = 0;
90
91     if (expandByRow) {
92         const int row = expandIndex;
93
94         for (int col = 0; col < n; ++col) {
95             if (isZero(data[row][col])) {
96                 continue;
97             }
98
99             const double sign = ((row + col) % 2 == 0) ? 1 : -1;
100             determinant += sign
101                 * data[row][col]
102                 * determinantRecursive(buildMinor(data, row, col));
103         }
104     } else {
105         const int col = expandIndex;
106
107         for (int row = 0; row < n; ++row) {
108             if (isZero(data[row][col])) {
109                 continue;
110             }
111
112             const double sign = ((row + col) % 2 == 0) ? 1 : -1;
113             determinant += sign
114                 * data[row][col]

```

```

115         * determinantRecursive(buildMinor(data, row, col));
116     }
117 }
118
119     return determinant;
120 }
121 }

```

Алгоритм Краскала реализован методами `m_kruskalAlgorithm` и `kruskalMinimumSpanningTree`, приведёнными в листинге 14. Они сортируют рёбра, объединяют множества вершин и формируют минимальный остов.

Вход: матрицы смежности и весов неориентированного графа.

Выход: рёбра и вес минимального остова, его весовая матрица и признак успешного построения.

Сложность: $O(p^2 + q \log q)$ с учётом формирования списка рёбер из матрицы смежности.

Листинг 14: Построение минимального остова алгоритмом Краскала

```

1  KruskalResult GeneratorGraph::m_kruskalAlgorithm() const {
2      KruskalResult result(m_vertexCount);
3      std::vector<WeightedEdge> edges;
4
5      for (int i = 0; i < m_vertexCount; ++i) {
6          result.treeMatrix(i, i) = 0;
7      }
8
9      // берем все ребра графа
10     for (int i = 0; i < m_vertexCount; ++i) {
11         for (int j = i + 1; j < m_vertexCount; ++j) {
12             if (m_undirectedAdjacencyMatrix(i, j) != 0 &&
13                 !std::isinf(m_undirectedWeightMatrix(i, j))) {
14                 edges.push_back({
15                     i,
16                     j,
17                     m_undirectedWeightMatrix(i, j)
18                 });
19             }
20         }
21     }
22
23     // сортируем рёбра по возрастанию веса
24     std::sort(
25         edges.begin(),
26         edges.end(),
27         [](const WeightedEdge& lhs, const WeightedEdge& rhs) {
28             if (lhs.weight != rhs.weight) {
29                 return lhs.weight < rhs.weight;
30             }
31         }

```

```

32         if (lhs.from != rhs.from) {
33             return lhs.from < rhs.from;
34         }
35
36         return lhs.to < rhs.to;
37     }
38 );
39
40     std::vector<int> parent(m_vertexCount);
41     std::vector<int> rank(m_vertexCount, 0);
42
43     for (int i = 0; i < m_vertexCount; ++i) {
44         parent[i] = i;
45     }
46
47     std::function<int(int)> findSet = [&](int vertex) {
48         if (parent[vertex] != vertex) {
49             parent[vertex] = findSet(parent[vertex]);
50         }
51
52         return parent[vertex];
53     };
54
55     auto unionSets = [&](int first, int second) {
56         first = findSet(first);
57         second = findSet(second);
58
59         if (first == second) {
60             return false;
61         }
62
63         if (rank[first] < rank[second]) {
64             std::swap(first, second);
65         }
66
67         parent[second] = first;
68
69         if (rank[first] == rank[second]) {
70             ++rank[first];
71         }
72
73         // если ребро соединяет вершины из разных множеств, его можно
74         → взять
75         return true;
76     };
77
78     // идем по ребрам от самого дешевого к самому дорогому
79     // изначально каждая вершина в своем множестве [сверху как раз
80     → сделаны ф. поиска и пересечения «множеств»]
81     for (const auto& edge : edges) {
82         if (unionSets(edge.from, edge.to)) {
83             // добавляем ребро в остов, если оно не образует цикл
84             result.edges.push_back(edge);

```

```

83         result.treeMatrix(edge.from, edge.to) = edge.weight;
84         result.treeMatrix(edge.to, edge.from) = edge.weight;
85         result.totalWeight += edge.weight;
86
87         // останавливаемся, когда выбрали n - 1 ребро
88         if (static_cast<int>(result.edges.size()) == m_vertexCount -
89             ↪ 1) {
89             break;
90         }
91     }
92 }
93
94 result.success =
95     static_cast<int>(result.edges.size()) == m_vertexCount - 1;
96
97 return result;
98 }
99
100 KruskalMinimumSpanningTreeResult
101 ↪ GeneratorGraph::kruskalMinimumSpanningTree() const {
102     KruskalMinimumSpanningTreeResult
103     ↪ result(std::move(m_kruskalAlgorithm()), m_vertexCount);
104
105     for (int i = 0; i < m_vertexCount; ++i) {
106         result.decodedTreeMatrix(i, i) = 0;
107     }
108
109     if (!result.kruskal.success) {
110         return result;
111     }
112
113     result.pruferCode = m_encodePruferCode(result.kruskal.edges);
114     result.decodedTreeMatrix = m_decodePruferCode(result.pruferCode);
115     result.pruferRoundTripSuccess =
116         matricesEqual(result.kruskal.treeMatrix,
117             ↪ result.decodedTreeMatrix);
118
119     return result;
120 }

```

Алгоритмы кодирования и декодирования кода Прюфера реализованы методами `m_encodePruferCode` и `m_decodePruferCode`, приведёнными в листинге 15. Они кодируют минимальный остов и восстанавливают его с сохранением весов рёбер.

Вход: минимальный остов при кодировании или код Прюфера при декодировании.

Выход: код Прюфера либо восстановленная весовая матрица остова.

Сложность: $O(p^2)$ для кодирования и $O(p^2 \log p)$ для декодирования.

Листинг 15: Кодирование и декодирование остова

```

1  std::vector<PruferCodeItem> GeneratorGraph::m_encodePruferCode(
2      const std::vector<WeightedEdge>& treeEdges
3  ) const {
4      // кодируем полученный остов кодом Прюфера с сохранением весов рёбер
5      std::vector<PruferCodeItem> code;
6      std::vector<std::set<std::pair<int, double>>>
7          ↪ adjacency(m_vertexCount);
8
9      for (const auto& edge : treeEdges) {
10         adjacency[edge.from].insert({edge.to, edge.weight});
11         adjacency[edge.to].insert({edge.from, edge.weight});
12     }
13
14     for (int step = 0; step < m_vertexCount - 1; ++step) {
15         int leaf = -1;
16
17         // на каждом шаге берём лист с минимальным номером
18         for (int vertex = 0; vertex < m_vertexCount; ++vertex) {
19             if (adjacency[vertex].size() == 1) {
20                 leaf = vertex;
21                 break;
22             }
23         }
24
25         if (leaf == -1) {
26             break;
27         }
28
29         const auto [neighbor, weight] = *adjacency[leaf].begin();
30         // в код записываем соседа листа, а рядом сохраняем вес
31         ↪ удаляемого ребра
32         code.push_back({neighbor, weight});
33
34         // удаляем лист из текущего дерева
35         adjacency[neighbor].erase({leaf, weight});
36         adjacency[leaf].clear();
37     }
38
39     return code;
40 }
41
42 Matrix GeneratorGraph::m_decodePruferCode(
43     const std::vector<PruferCodeItem>& code
44 ) const {
45     // декодируем код Прюфера обратно в матрицу весов остова
46     Matrix decodedTreeMatrix(
47         m_vertexCount,
48         m_vertexCount,
49         std::numeric_limits<double>::infinity()
50     );
51     std::vector<int> unusedVertices(m_vertexCount);
52
53     for (int i = 0; i < m_vertexCount; ++i) {

```

```

52     decodedTreeMatrix(i, i) = 0;
53     unusedVertices[i] = i;
54 }
55
56     const int steps = std::min(
57         m_vertexCount - 1,
58         static_cast<int>(code.size())
59     );
60
61     for (int i = 0; i < steps; ++i) {
62         std::set<int> remainingCode;
63
64         for (int j = i; j < static_cast<int>(code.size()); ++j) {
65             remainingCode.insert(code[j].vertex);
66         }
67
68         // выбираем минимальную вершину, которой нет в оставшемся коде
69         auto leafIt = std::find_if(
70             unusedVertices.begin(),
71             unusedVertices.end(),
72             [&](int vertex) {
73                 return remainingCode.find(vertex) == remainingCode.end();
74             }
75         );
76
77         if (leafIt == unusedVertices.end()) {
78             break;
79         }
80
81         const int leaf = *leafIt;
82         const int neighbor = code[i].vertex;
83         const double weight = code[i].weight;
84
85         // восстанавливаем ребро с тем же весом, который был сохранён при
86         → кодировании
87         decodedTreeMatrix(leaf, neighbor) = weight;
88         decodedTreeMatrix(neighbor, leaf) = weight;
89
90         unusedVertices.erase(leafIt);
91     }
92     return decodedTreeMatrix;
93 }

```

Алгоритм Эдмондса реализован методами `edmondsBlossom` и `InOriginalGraph`, `edmondsBlossomInMinimumSpanningTree` и `m_edmondsBlossom`, приведёнными в листинге 16. Они выбирают исследуемый граф и находят максимальное независимое множество рёбер, сжимая обнаруженные нечётные циклы.

Вход: матрицы смежности и весов исходного графа либо минимального остова.

Выход: список и матрица рёбер максимального паросочетания.

Сложность: $O(p^3)$.

Листинг 16: Поиск максимального независимого множества рёбер

```
1 EdmondsBlossomResult
2 GeneratorGraph::edmondsBlossomInOriginalGraph() const {
3     const Matrix& weights = isMatrixInit.weight
4         ? m_undirectedWeightMatrix
5         : m_undirectedAdjacencyMatrix;
6
7     return m_edmondsBlossom(
8         m_undirectedAdjacencyMatrix,
9         weights,
10        isMatrixInit.weight
11    );
12 }
13
14 EdmondsBlossomResult
15 GeneratorGraph::edmondsBlossomInMinimumSpanningTree() const {
16     EdmondsBlossomResult result(m_vertexCount);
17     auto kruskalMinimumSpanningTreeResult = kruskalMinimumSpanningTree();
18
19     if (!kruskalMinimumSpanningTreeResult.kruskal.success) {
20         result.success = false;
21         return result;
22     }
23
24     Matrix adjacency(m_vertexCount, m_vertexCount, 0);
25
26     for (const auto& edge :
27         ↪ kruskalMinimumSpanningTreeResult.kruskal.edges) {
28         adjacency(edge.from, edge.to) = 1;
29         adjacency(edge.to, edge.from) = 1;
30     }
31
32     return m_edmondsBlossom(
33         adjacency,
34         kruskalMinimumSpanningTreeResult.kruskal.treeMatrix,
35         true
36     );
37 }
38 EdmondsBlossomResult GeneratorGraph::m_edmondsBlossom(
39     const Matrix& adjacency,
40     const Matrix& weights,
41     bool hasWeights
42 ) const {
43     EdmondsBlossomResult result(m_vertexCount);
44     result.hasWeights = hasWeights;
45
46     if (hasWeights) {
```

```

47     result.edgeMatrix = Matrix(
48         m_vertexCount,
49         m_vertexCount,
50         std::numeric_limits<double>::infinity()
51     );
52
53     for (int i = 0; i < m_vertexCount; ++i) {
54         result.edgeMatrix(i, i) = 0;
55     }
56 }
57
58 const auto graph = m_matrixToAdjacencyList<int>(
59     adjacency,
60     [](int, int to) {
61         return to;
62     }
63 );
64
65 // хотим выбрать как можно больше рёбер, чтобы они не касались друг
66 // => ищем увел. путь, путь от одной вершины до другой, где рёбра
67 //    идут по очереди
68 // BEGIN
69 // а нечётные циклы временно сжимаем.
70 // match[v]: с какой вершиной соединена вершина v
71 std::vector<int> match(m_vertexCount, -1);
72 // parent[v]: предыдущая вершина в дереве поиска увеличивающегося
73 //    пути
74 std::vector<int> parent(m_vertexCount);
75 std::vector<int> base(m_vertexCount);
76 // помечаем, что вершина уже добавлена в очередь
77 std::vector<bool> used(m_vertexCount);
78 // помечаем цикл в графе
79 std::vector<bool> blossom(m_vertexCount);
80 std::queue<int> queue;
81
82 // ищем общую базу нечётного цикла, чтобы сжать к одной вершине
83 auto findLca = [&](int first, int second) {
84     std::vector<bool> usedPath(m_vertexCount, false);
85
86     while (true) {
87         first = base[first];
88         usedPath[first] = true;
89
90         if (match[first] == -1) {
91             break;
92         }
93
94         first = parent[match[first]];
95     }
96
97     while (true) {

```



```

97         second = base[second];
98
99         if (usedPath[second]) {
100             return second;
101         }
102
103         second = parent[match[second]];
104     }
105 };
106
107 auto markPath = [&](int vertex,
108                   int lca,
109                   int child,
110                   auto&& markPathRef) -> void {
111     // помечаем путь от вершины до общей базы
112     while (base[vertex] != lca) {
113         blossom[base[vertex]] = true;
114         blossom[base[match[vertex]]] = true;
115         parent[vertex] = child;
116         child = match[vertex];
117         vertex = parent[match[vertex]];
118     }
119 };
120
121 auto findPath = [&](int root) {
122     // T <- ∅: очищаем BFS-дерево поиска увеличивающего пути
123     std::fill(used.begin(), used.end(), false);
124     std::fill(parent.begin(), parent.end(), -1);
125
126     for (int i = 0; i < m_vertexCount; ++i) {
127         base[i] = i;
128     }
129
130     while (!queue.empty()) {
131         queue.pop();
132     }
133
134     // pick r in F; queue.push(r); T.add(r)
135     queue.push(root);
136     used[root] = true;
137
138     // WHILE queue != ∅
139     while (!queue.empty()) {
140         // v <- queue.pop()
141         int vertex = queue.front();
142         queue.pop();
143
144         // FOR ALL neighbors w of v DO
145         for (int to : graph[vertex]) {
146             if (base[vertex] == base[to] ||
147                 match[vertex] == to) {
148                 continue;
149             }

```

```

150
151         if (to == root ||
152             (match[to] != -1 && parent[match[to]] != -1)) {
153             // ELSE IF w in T AND odd-length cycle detected THEN
154             int lca = findLca(vertex, to);
155             std::fill(blossom.begin(), blossom.end(), false);
156
157             // contract cycle: найден нечётный цикл, сжимаем
158             // → blossom к общей базе
159             markPath(vertex, lca, to, markPath);
160             markPath(to, lca, vertex, markPath);
161
162             for (int i = 0; i < m_vertexCount; ++i) {
163                 if (blossom[base[i]]) {
164                     base[i] = lca;
165
166                     if (!used[i]) {
167                         used[i] = true;
168                         queue.push(i);
169                     }
170                 }
171             }
172             else if (parent[to] == -1) {
173                 // IF w not in T AND w matched THEN T.add(w),
174                 // → T.add(mate(w))
175                 parent[to] = vertex;
176
177                 if (match[to] == -1) {
178                     // ELSE IF w in F THEN свободная вершина
179                     // → завершает увеличивающий путь
180                     return to;
181                 }
182
183                 // queue.push(mate(w))
184                 int next = match[to];
185                 used[next] = true;
186                 queue.push(next);
187             }
188         }
189     }
190
191     return -1;
192 };
193
194 auto augment = [&](int vertex) {
195     // expand all contracted nodes; reconstruct augmenting path;
196     // invert augmenting path
197     while (vertex != -1) {
198         int previous = parent[vertex];
199         int next = (previous == -1) ? -1 : match[previous];
200
201         match[vertex] = previous;
202     }
203 }

```

```

200         if (previous != -1) {
201             match[previous] = vertex;
202         }
203
204         vertex = next;
205     }
206 };
207
208 // WHILE F != ∅ DO: пробуем стартовать из каждой свободной вершины
209 for (int vertex = 0; vertex < m_vertexCount; ++vertex) {
210     if (match[vertex] == -1) {
211         int endpoint = findPath(vertex);
212
213         if (endpoint != -1) {
214             augment(endpoint);
215         }
216     }
217 }
218 // END
219
220 for (int from = 0; from < m_vertexCount; ++from) {
221     int to = match[from];
222
223     if (to == -1 || from > to) {
224         continue;
225     }
226
227     double value = hasWeights ? weights(from, to) : 1;
228
229     result.edges.push_back({from, to, value});
230     result.edgeMatrix(from, to) = value;
231     result.edgeMatrix(to, from) = value;
232 }
233
234 return result;
235 }

```

3.5 Лабораторная работа № 5

Алгоритм приведения графа к эйлерову виду реализован методом `checkIfEulerianGraph`, приведённым в листинге 17. Он проверяет чётность степеней и попарно добавляет либо удаляет рёбра между нечётными вершинами. Перед удалением проверяется наличие обходного пути, поэтому преобразование прекращается, если оно требует удаления моста.

Вход: матрица смежности неориентированного графа.

Выход: изменённая матрица смежности, списки добавленных и удалённых рёбер, признаки исходного состояния и завершения преобразования.

Сложность: в худшем случае $O(p^6)$ с учётом перебора пар нечётных вершин и повторных поисков обходного пути.

Листинг 17: Проверка и преобразование графа к эйлерову виду

```

1  CheckIfEulerianGraphResult GeneratorGraph::checkIfEulerianGraph(
2      bool allowDisconnectedResult
3  ) const {
4      bool isEulerian = true;
5      bool transformationCompleted = true;
6      bool resultIsConnected = true;
7      Matrix adjacency = getUndirectedAdjacencyMatrix();
8      std::vector<Edge> addedEdges;
9      std::vector<Edge> removedEdges;
10     auto getVertexDegree = [&](size_t i) {
11         int degree = 0;
12         for (size_t j = 0; j < adjacency.cols(); ++j) {
13             if (i == j) continue;
14
15             if (adjacency(i, j) != 0) ++degree;
16         }
17
18         return degree;
19     };
20
21     std::vector<int> verticesWithOddDegrees;
22
23     while (true) {
24         verticesWithOddDegrees.clear();
25
26         for (size_t i = 0; i < adjacency.rows(); ++i) {
27             if (getVertexDegree(i) % 2 != 0) {
28                 isEulerian = false;
29                 verticesWithOddDegrees.push_back(i);
30             }
31         }
32
33         if (verticesWithOddDegrees.empty()) {
34             break;
35         }
36
37         bool graphChanged = false;
38
39         for (size_t i = 0; i < verticesWithOddDegrees.size() &&
40             ↪ !graphChanged; ++i) {
41             for (size_t j = i + 1; j < verticesWithOddDegrees.size();
42                 ↪ ++j) {
43                 const auto u = verticesWithOddDegrees[i];
44                 const auto v = verticesWithOddDegrees[j];
45
46                 if (adjacency(u, v) == 0) {
47                     adjacency(u, v) = 1;
48                     adjacency(v, u) = 1;
49                     addedEdges.emplace_back(u, v);
50                     graphChanged = true;
51                     break;

```

```

50         }
51
52         Matrix temp = adjacency;
53         temp(u, v) = 0;
54         temp(v, u) = 0;
55
56         const auto resultPath = dijkstraNegative(u, v, temp);
57
58         if (resultPath.hasPath || allowDisconnectedResult) {
59             adjacency(u, v) = 0;
60             adjacency(v, u) = 0;
61             removedEdges.emplace_back(u, v);
62
63             if (!resultPath.hasPath) {
64                 resultIsConnected = false;
65             }
66
67             graphChanged = true;
68             break;
69         }
70     }
71 }
72
73 if (!graphChanged) {
74     transformationCompleted = false;
75     break;
76 }
77 }
78
79 FleuryResult eulerianCycle;
80
81 if (transformationCompleted && resultIsConnected) {
82     eulerianCycle = m_flueryAlgorithm(adjacency);
83 }
84
85 return CheckIfEulerianGraphResult(
86     isEulerian,
87     transformationCompleted,
88     std::move(addedEdges),
89     std::move(removedEdges),
90     std::move(eulerianCycle),
91     std::move(adjacency)
92 );
93 }

```

Алгоритм Флёрри реализован методом `m_flueryAlgorithm`, приведённым в листинге 18. Он последовательно выбирает соседнюю вершину, удаляет пройденное ребро и добавляет вершину в результат после исчерпания её соседей.

Вход: матрица смежности эйлерова графа.

Выход: последовательность вершин эйлерова цикла.

Сложность: $O(pq)$ в худшем случае.

Листинг 18: Построение эйлерова цикла

```
1 FleuryResult GeneratorGraph::m_flueryAlgorithm(const Matrix& adjacency)
  ↪ const {
2     auto graph = m_matrixToAdjacencyList<int>(
3         adjacency,
4         [](int, int to) {
5             return to;
6         }
7     );
8
9     FleuryResult path;
10    std::stack<int> stack;
11    stack.push(0);
12
13    while (!stack.empty()) {
14        const int node = stack.top();
15
16        if (graph[node].empty()) {
17            path.emplace_back(node);
18            stack.pop();
19
20        } else {
21            const int next = graph[node].front();
22            graph[node].erase(graph[node].begin());
23
24            const auto reverseEdge = std::find(
25                graph[next].begin(),
26                graph[next].end(),
27                node
28            );
29
30            if (reverseEdge == graph[next].end()) {
31                throw std::logic_error(
32                    "Обратное ребро отсутствует в списке смежности"
33                );
34            }
35
36            graph[next].erase(reverseEdge);
37            stack.push(next);
38        }
39    }
40
41    std::reverse(path.begin(), path.end());
42    return path;
43 }
```

Алгоритм построения фундаментальной системы разрезов реализован методом `buildFundamentalCutSystem`, приведённым в листинге 19. Он строит минимальный остов и для каждого его ребра получает фунда-

ментальный разрез.

Вход: матрицы смежности и весов неориентированного графа.

Выход: минимальный остов и фундаментальная система разрезов.

Сложность: $O(p^3 + q \log q)$.

Листинг 19: Построение фундаментальной системы разрезов

```
1 FundamentalCutsResult GeneratorGraph::buildFundamentalCutSystem() const {
2     FundamentalCutsResult result(m_vertexCount);
3
4     if (!isMatrixInit.adjacency || !isMatrixInit.weight) {
5         return result;
6     }
7
8     result.spanningTree = m_kruskalAlgorithm();
9
10    if (!result.spanningTree.success) {
11        return result;
12    }
13
14    AdjacencyList tree(m_vertexCount);
15
16    for (const auto& edge : result.spanningTree.edges) {
17        tree[edge.from].push_back(edge.to);
18        tree[edge.to].push_back(edge.from);
19    }
20
21    for (const auto& removedEdge : result.spanningTree.edges) {
22        std::vector<bool> firstComponent(m_vertexCount, false);
23        std::vector<int> stack = {removedEdge.from};
24        firstComponent[removedEdge.from] = true;
25
26        // Удаление ребра остова делит дерево на две компоненты.
27        while (!stack.empty()) {
28            const int vertex = stack.back();
29            stack.pop_back();
30
31            for (int to : tree[vertex]) {
32                const bool isRemovedEdge =
33                    (vertex == removedEdge.from && to == removedEdge.to)
34                    ||
35                    (vertex == removedEdge.to && to == removedEdge.from);
36
37                if (!isRemovedEdge && !firstComponent[to]) {
38                    firstComponent[to] = true;
39                    stack.push_back(to);
40                }
41            }
42        }
43
44        GraphCut cut;
```

```

45      // В разрез входят все рёбра исходного графа между компонентами.
46      for (int from = 0; from < m_vertexCount; ++from) {
47          for (int to = from + 1; to < m_vertexCount; ++to) {
48              if (m_undirectedAdjacencyMatrix(from, to) != 0 &&
49                  firstComponent[from] != firstComponent[to]) {
50                  cut.emplace(from, to);
51              }
52          }
53      }
54
55      result.fundamentalCuts.push_back({
56          {removedEdge.from, removedEdge.to},
57          std::move(cut)
58      });
59  }
60
61  result.success =
62      static_cast<int>(result.fundamentalCuts.size()) == m_vertexCount
        ↪ - 1;
63  return result;
64  }

```

Алгоритм получения остальных разрезов реализован методами `symmetricDifferenceOfFundamentalCuts` и `m_symmetricDifference`, приведёнными в листинге 20. Они последовательно вычисляют симметрическую разность выбранных фундаментальных разрезов.

Вход: фундаментальная система разрезов и выбранные разрезы.

Выход: множество рёбер результирующего разреза.

Сложность: $O(rq \log q)$, где r — количество выбранных фундаментальных разрезов.

Листинг 20: Получение разреза симметрической разностью

```

1  GraphCut GeneratorGraph::symmetricDifferenceOfFundamentalCuts(
2      const FundamentalCutsResult& system,
3      const std::vector<int>& selectedCutIndices
4  ) const {
5      GraphCut result;
6
7      for (int index : selectedCutIndices) {
8          if (index < 0 || index >=
9              ↪ static_cast<int>(system.fundamentalCuts.size())) {
10             throw std::out_of_range("Номер фундаментального разреза вне
11                 ↪ диапазона");
12         }
13
14         result = m_symmetricDifference(
15             result,
16             system.fundamentalCuts[index].cut
17         );

```



```

16     }
17
18     return result;
19 }
20
21 // этот метод показывает, что используем метод из лекции для вычисления
   ↳ симм. разности
22 // можно найти на слайде 18 «Подпространства циклов и коциклов»
23 GraphCut GeneratorGraph::m_symmetricDifference(
24     const GraphCut& first,
25     const GraphCut& second
26 ) const {
27     GraphCut result;
28     GraphCut edgeCoordinates = first;
29     edgeCoordinates.insert(second.begin(), second.end());
30
31     // разрез рассматриваем как вектор векторного пространства,
   ↳ натянутого на множество рёбер
32     // коэффициент равен 1, если ребро входит в разрез, иначе равен 0
33     for (const auto& edge : edgeCoordinates) {
34         const int firstCoefficient = first.count(edge) == 0 ? 0 : 1;
35         const int secondCoefficient = second.count(edge) == 0 ? 0 : 1;
36         const int resultCoefficient =
37             (firstCoefficient + secondCoefficient) % 2;
38
39         if (resultCoefficient == 1) {
40             result.insert(edge);
41         }
42     }
43
44     return result;
45 }

```

4 Результаты работы

4.1 Лабораторная работа № 1

В результате генерации получен связный ориентированный граф из 10 вершин. Матрица смежности и вычисленные для этого графа эксцентриситеты, диаметр, центральная и диаметральная вершины приведены на рис. 17.

```
Количество вершин (>=2): 10

[*] Граф сгенерирован автоматически.
Матрица смежности:
| 0 1 2 3 4 5 6 7 8 9
--|-----
0 | 0 1 0 0 0 0 0 0 0 0
1 | 0 0 1 0 0 0 0 0 0 0
2 | 0 0 0 1 0 0 0 0 1 0
3 | 0 0 0 0 1 0 1 0 0 0
4 | 0 0 0 0 0 0 1 0 0 1
5 | 0 0 0 0 0 0 1 0 0 0
6 | 0 0 0 0 0 0 0 1 1 1
7 | 0 0 0 0 0 0 0 0 1 0
8 | 0 0 0 0 0 0 0 0 0 1
9 | 0 0 0 0 0 0 0 0 0 0

----- Меню -----
1. Сгенерировать граф
2. Эксцентриситеты, центр, диаметр
3. Сгенерировать весовую матрицу
4. Алгоритм Шимбелла
5. Маршруты между вершинами
6. Показать текущие матрицы
7. Найти точки сочленения
8. Алгоритм Дейкстры для отрицательных весов
9. Сгенерировать матрицы пропускных способностей и стоимостей
10. Максимальный поток
11. Поток минимальной стоимости
12. Число остовных деревьев
13. Минимальный остов: алгоритм Краскала
14. Алгоритм Эдмондса: максимальное независимое множество рёбер
15. Проверка, является ли неориентированный граф эйлеровым, построение эйлерова цикла
16. Фундаментальная система разрезов
0. Выход
>

===== Эксцентриситеты, центр, диаметр =====
Вершина:      0 1 2 3 4 5 6 7 8 9
Эксцентриситет: 5 4 3 2 3 2 1 2 1 0
Диаметр графа: 5
Центральные вершины: 9
Диаметральные вершины: 0

----- Меню -----
1. Сгенерировать граф
2. Эксцентриситеты, центр, диаметр
3. Сгенерировать весовую матрицу
4. Алгоритм Шимбелла
5. Маршруты между вершинами
6. Показать текущие матрицы
7. Найти точки сочленения
8. Алгоритм Дейкстры для отрицательных весов
9. Сгенерировать матрицы пропускных способностей и стоимостей
10. Максимальный поток
11. Поток минимальной стоимости
12. Число остовных деревьев
13. Минимальный остов: алгоритм Краскала
14. Алгоритм Эдмондса: максимальное независимое множество рёбер
15. Проверка, является ли неориентированный граф эйлеровым, построение эйлерова цикла
16. Фундаментальная система разрезов
0. Выход
>
```

Рис. 17: Генерация графа и вычисление его характеристик

Программа проверяет ввод количества вершин и пунктов меню. Примеры сообщений при некорректном вводе показаны на рис. 18.

```
===== Теория графов =====
Количество вершин (>=2): 0
[x] Некорректный ввод.
Количество вершин (>=2):

----- Меню -----
1. Сгенерировать граф
2. Эксцентриситеты, центр, диаметр
3. Сгенерировать весовую матрицу
4. Алгоритм Шимбелла
5. Маршруты между вершинами
6. Показать текущие матрицы
7. Найти точки сочленения
8. Алгоритм Дейкстры для отрицательных весов
9. Сгенерировать матрицы пропускных способностей и стоимостей
10. Максимальный поток
11. Поток минимальной стоимости
12. Число остовных деревьев
13. Минимальный остов: алгоритм Краскала
14. Алгоритм Эдмондса: максимальное независимое множество рёбер
15. Проверка, является ли неориентированный граф эйлеровым, построение эйлерова цикла
16. Фундаментальная система разрезов
0. Выход
> -1
[x] Некорректный ввод.
>
```

Рис. 18: Обработка некорректного ввода

Сгенерированная матрица положительных весов приведена на рис. 19.

```

> 3

===== Весовая матрица =====
Режим весов:
  1 – положительные
  2 – отрицательные
  3 – смешанные
> 1
[*] Весовая матрица сгенерирована.
Матрица весов:

```

	0	1	2	3	4	5	6	7	8	9
0	0	4	inf	inf	inf	inf	inf	inf	inf	inf
1	inf	0	2	inf	inf	inf	inf	inf	inf	inf
2	inf	inf	0	2	inf	inf	inf	3	inf	inf
3	inf	inf	inf	0	4	inf	5	inf	inf	inf
4	inf	inf	inf	inf	0	5	inf	inf	inf	6
5	inf	inf	inf	inf	inf	0	2	inf	inf	inf
6	inf	inf	inf	inf	inf	inf	0	3	4	3
7	inf	inf	inf	inf	inf	inf	inf	0	4	inf
8	inf	inf	inf	inf	inf	inf	inf	inf	0	4
9	inf	inf	inf	inf	inf	inf	inf	inf	inf	0

Рис. 19: Матрица положительных весов

Результат метода Шимбелла для маршрутов из двух рёбер приведён на рис. 20. Для каждой пары вершин получен минимальный вес маршрута заданной длины.

```

===== Алгоритм Шимбелла =====
Количество рёбер в пути [0, 9]: 10
[x] Некорректный ввод.
Количество рёбер в пути [0, 9]: 2
Искать:
  1 – минимальный путь
  2 – максимальный путь
> 1

```

	0	1	2	3	4	5	6	7	8	9
0	0	inf	6	inf	inf	inf	inf	inf	inf	inf
1	inf	0	inf	4	inf	inf	inf	5	inf	inf
2	inf	inf	0	inf	6	inf	7	inf	7	inf
3	inf	inf	inf	0	inf	9	inf	8	9	8
4	inf	inf	inf	inf	0	inf	7	inf	inf	inf
5	inf	inf	inf	inf	inf	0	inf	5	6	5
6	inf	inf	inf	inf	inf	inf	0	inf	7	8
7	inf	inf	inf	inf	inf	inf	inf	0	inf	8
8	inf	inf	inf	inf	inf	inf	inf	inf	0	inf
9	inf	inf	inf	inf	inf	inf	inf	inf	inf	0

Рис. 20: Минимальные веса маршрутов из двух рёбер

При длине девять рёбер существует маршрут из вершины 0 в вершину 9, а при нулевой длине нулевые значения остаются только на диагонали. Соответствующие матрицы приведены на рис. 21.

```
> 4
===== Алгоритм Шимбелла =====
Количество рёбер в пути [0, 9]: 9
Искать:
1 — минимальный путь
2 — максимальный путь
> 1
| 0 1 2 3 4 5 6 7 8 9
-----
0 | 0 inf inf inf inf inf inf inf inf 30
1 | inf 0 inf inf inf inf inf inf inf inf
2 | inf inf 0 inf inf inf inf inf inf inf
3 | inf inf inf 0 inf inf inf inf inf inf
4 | inf inf inf inf 0 inf inf inf inf inf
5 | inf inf inf inf inf 0 inf inf inf inf
6 | inf inf inf inf inf inf 0 inf inf inf
7 | inf inf inf inf inf inf inf 0 inf inf
8 | inf inf inf inf inf inf inf inf 0 inf
9 | inf inf inf inf inf inf inf inf inf 0
```

```
> 4
===== Алгоритм Шимбелла =====
Количество рёбер в пути [0, 9]: 0
Искать:
1 — минимальный путь
2 — максимальный путь
> 1
| 0 1 2 3 4 5 6 7 8 9
-----
0 | 0 inf inf inf inf inf inf inf inf inf
1 | inf 0 inf inf inf inf inf inf inf inf
2 | inf inf 0 inf inf inf inf inf inf inf
3 | inf inf inf 0 inf inf inf inf inf inf
4 | inf inf inf inf 0 inf inf inf inf inf
5 | inf inf inf inf inf 0 inf inf inf inf
6 | inf inf inf inf inf inf 0 inf inf inf
7 | inf inf inf inf inf inf inf 0 inf inf
8 | inf inf inf inf inf inf inf inf 0 inf
9 | inf inf inf inf inf inf inf inf inf 0
```

Рис. 21: Результаты метода Шимбелла для девяти и нуля рёбер

До генерации весовой матрицы выполнение метода Шимбелла не начинается. Также проверяется допустимость введённого количества рёбер. Эти случаи показаны на рис. 22.

```
> 4
===== Алгоритм Шимбелла =====
[x] Весовая матрица не инициализирована.
```

```
> 4
===== Алгоритм Шимбелла =====
Количество рёбер в пути [0, 9]: 10
[x] Некорректный ввод.
Количество рёбер в пути [0, 9]:
```

Рис. 22: Обработка ошибок при запуске метода Шимбелла

Примеры существующего маршрута из вершины 0 в вершину 7 и отсутствующего маршрута из вершины 7 в вершину 0 приведены на рис. 23.

```
> 5
===== Маршруты между вершинами =====
Начальная вершина: 0
Конечная вершина: 7
Маршрут 0 -> 7: существует
Количество маршрутов: 3
```

```
> 5
===== Маршруты между вершинами =====
Начальная вершина: 7
Конечная вершина: 0
Маршрут 7 -> 0: не существует
```

Рис. 23: Проверка маршрутов между вершинами

4.2 Лабораторная работа № 2

Для сгенерированного графа найдены точки сочленения 1 и 2. Модифицированный алгоритм Дейкстры построил маршрут из вершины 0 в вершину 7, определил его вес и сформировал вектор расстояний. Результаты приведены на рис. 24.

```
> 7

===== Точки сочленения =====
Точки сочленения: 1 2
Количество итераций: 100

> 8

===== Алгоритм Дейкстры для отрицательных весов =====
Начальная вершина: 0
Конечная вершина: 7
Кратчайший маршрут из 0 в 7 : (0, 1, 2, 7)
Веса дуг маршрута: (4, 2, 3); sum: 9
Вектор расстояний: (0, 4, 6, 8, 12, 17, 13, 9, 13, 16)
Количество итераций: 100
```

Рис. 24: Результаты поиска точек сочленения и кратчайшего пути

Попытка запустить алгоритм Дейкстры до генерации весовой матрицы завершается сообщением, приведённым на рис. 25.

```
> 8

===== Алгоритм Дейкстры для отрицательных весов =====
[x] Весовая матрица не инициализирована.
```

Рис. 25: Проверка наличия весовой матрицы

Для сравнения было выполнено 100 запусков на случайных графах из 10 вершин. В каждом запуске оба алгоритма работали на одном графе. Количество итераций в отдельных запусках показано на рис. 26.

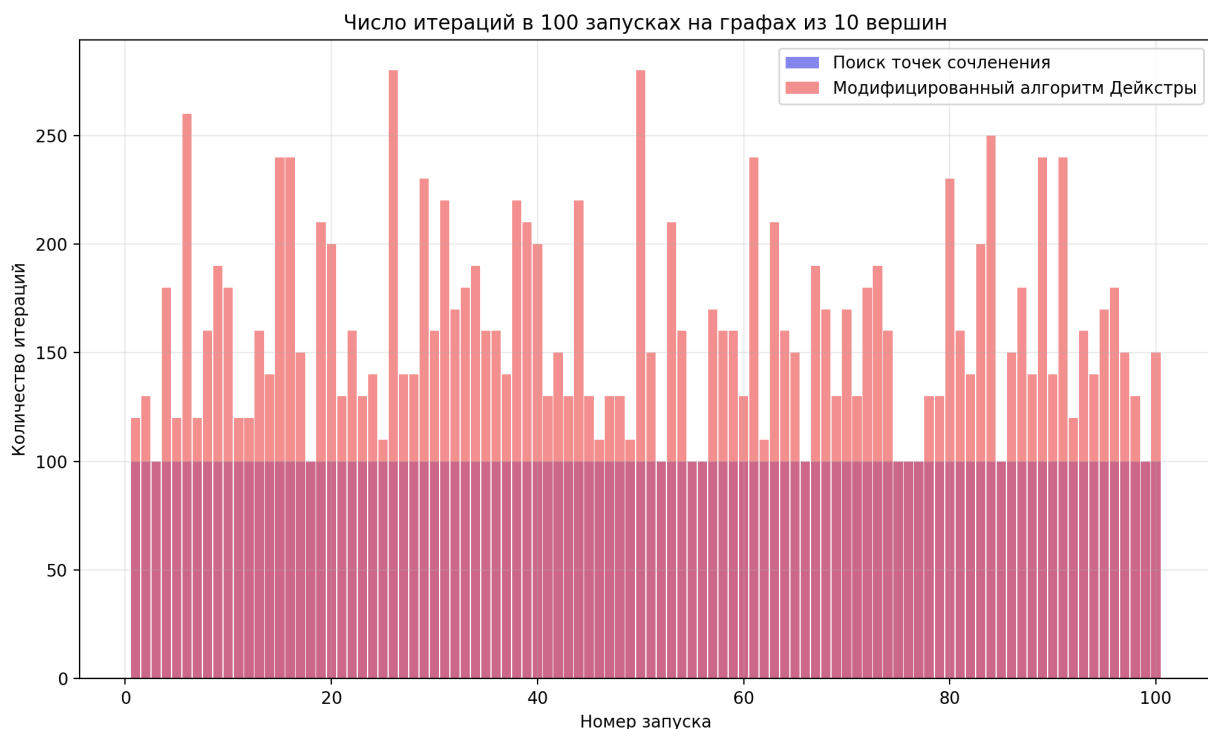


Рис. 26: Количество итераций в каждом запуске

Распределение полученных значений приведено на рис. 27.

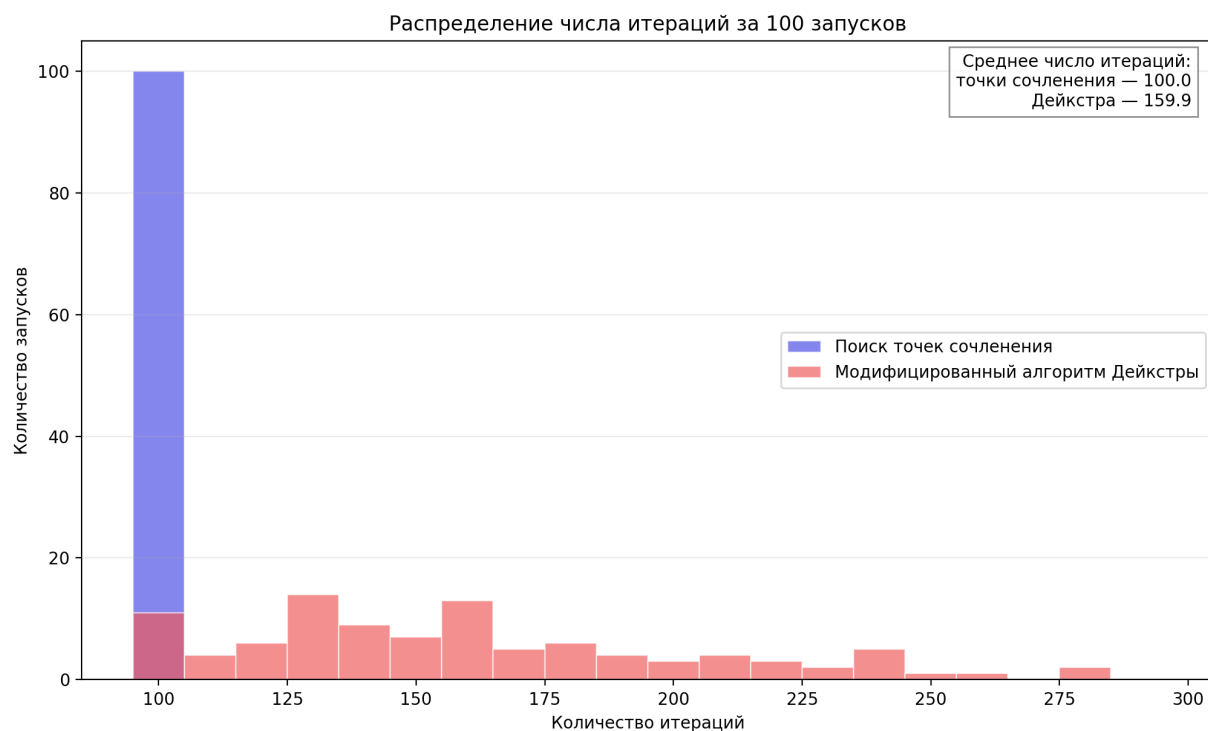


Рис. 27: Распределение количества итераций за 100 запусков

Поиск точек сочленения во всех случаях выполнил 100 итераций. В 89 запусках поиск точек сочленения потребовал меньше итераций, а в 11 запусках значения совпали. Следовательно, на проведённой серии запусков поиск точек сочленения оказался эффективнее по количеству итераций.

4.3 Лабораторная работа № 3

Для ориентированного графа сгенерированы матрицы пропускных способностей и стоимостей, приведённые на рис. 28.

```

> 9

===== Матрицы пропускных способностей и стоимостей =====
[*] Матрицы сгенерированы.

Матрица пропускных способностей:
  | 0 1 2 3 4 5 6 7 8 9
--+-----
0 | 0 6 0 0 0 0 0 0 0 0
1 | 0 0 4 0 0 0 0 0 0 0
2 | 0 0 0 5 0 0 0 5 0 0
3 | 0 0 0 0 5 0 5 0 0 0
4 | 0 0 0 0 0 7 0 0 0 4
5 | 0 0 0 0 0 0 10 0 0 0
6 | 0 0 0 0 0 0 0 3 5 11
7 | 0 0 0 0 0 0 0 0 11 0
8 | 0 0 0 0 0 0 0 0 0 5
9 | 0 0 0 0 0 0 0 0 0 0

Матрица стоимостей:
  | 0 1 2 3 4 5 6 7 8 9
--+-----
0 | 0 2 0 0 0 0 0 0 0 0
1 | 0 0 8 0 0 0 0 0 0 0
2 | 0 0 0 5 0 0 0 6 0 0
3 | 0 0 0 0 3 0 3 0 0 0
4 | 0 0 0 0 0 5 0 0 0 1
5 | 0 0 0 0 0 0 3 0 0 0
6 | 0 0 0 0 0 0 0 4 4 6
7 | 0 0 0 0 0 0 0 0 4 0
8 | 0 0 0 0 0 0 0 0 0 5
9 | 0 0 0 0 0 0 0 0 0 0

```

Рис. 28: Матрицы пропускных способностей и стоимостей

Для истока 0 и стока 9 алгоритм Форда — Фалкерсона получил максимальный поток величины 4. Матрица потоков приведена на рис. [29](#).

```

> 10

===== Максимальный поток =====
Источник:
> 0
Сток:
> 9

Максимальный поток: 4
Количество итераций: 155

Матрица потоков:
  | 0  1  2  3  4  5  6  7  8  9
--+-+-----
0 | 0  4  0  0  0  0  0  0  0  0
1 | 0  0  4  0  0  0  0  0  0  0
2 | 0  0  0  4  0  0  0  0  0  0
3 | 0  0  0  0  4  0  0  0  0  0
4 | 0  0  0  0  0  0  0  0  0  4
5 | 0  0  0  0  0  0  0  0  0  0
6 | 0  0  0  0  0  0  0  0  0  0
7 | 0  0  0  0  0  0  0  0  0  0
8 | 0  0  0  0  0  0  0  0  0  0
9 | 0  0  0  0  0  0  0  0  0  0

```

Рис. 29: Результат вычисления максимального потока

Заданная величина потока составила 2. Поток этой величины построен успешно, его минимальная стоимость равна 38. Результирующие матрицы приведены на рис. [30](#).


```

> 11

===== Поток минимальной стоимости =====
Источник:
> 0
Сток:
> 9
Максимальный поток: 4
Заданный поток [2/3 * max]: 2

Поток минимальной стоимости найден.
Стоимость потока: 38
Количество итераций: 100

Матрица потоков минимальной стоимости:
  | 0  1  2  3  4  5  6  7  8  9
--+--
0 | 0  2  0  0  0  0  0  0  0  0
1 | 0  0  2  0  0  0  0  0  0  0
2 | 0  0  0  2  0  0  0  0  0  0
3 | 0  0  0  0  2  0  0  0  0  0
4 | 0  0  0  0  0  0  0  0  0  2
5 | 0  0  0  0  0  0  0  0  0  0
6 | 0  0  0  0  0  0  0  0  0  0
7 | 0  0  0  0  0  0  0  0  0  0
8 | 0  0  0  0  0  0  0  0  0  0
9 | 0  0  0  0  0  0  0  0  0  0

Матрица стоимостей минимального потока:
  | 0  1  2  3  4  5  6  7  8  9
--+--
0 | 0  4  0  0  0  0  0  0  0  0
1 | 0  0  16  0  0  0  0  0  0  0
2 | 0  0  0  10  0  0  0  0  0  0
3 | 0  0  0  0  6  0  0  0  0  0
4 | 0  0  0  0  0  0  0  0  0  2
5 | 0  0  0  0  0  0  0  0  0  0
6 | 0  0  0  0  0  0  0  0  0  0
7 | 0  0  0  0  0  0  0  0  0  0
8 | 0  0  0  0  0  0  0  0  0  0
9 | 0  0  0  0  0  0  0  0  0  0

```

Рис. 30: Результат вычисления потока минимальной стоимости

При отсутствии матрицы пропускных способностей вычисление максимального потока не начинается, что показано на рис. 31.

```

> 10

===== Максимальный поток =====
[x] Сначала сгенерируйте матрицу пропускных способностей.

```

Рис. 31: Проверка наличия матрицы пропускных способностей

4.4 Лабораторная работа № 4

По матрице Кирхгофа и её алгебраическому дополнению вычислено число остовных деревьев, равное 271. Результат приведён на рис. 32.

```

> 12

==== Число остовных деревьев ====
Матрица Кирхгофа:
  | 0  1  2  3  4  5  6  7  8  9
--|-----
0 | 1 -1  0  0  0  0  0  0  0
1 | -1 2 -1  0  0  0  0  0  0
2 |  0 -1 3 -1  0  0  0 -1  0
3 |  0  0 -1 3 -1  0 -1  0  0
4 |  0  0  0 -1 3 -1  0  0 -1
5 |  0  0  0  0 -1 2 -1  0  0
6 |  0  0  0 -1  0 -1 5 -1 -1
7 |  0  0 -1  0  0  0 -1 3 -1
8 |  0  0  0  0  0  0 -1 -1 3
9 |  0  0  0  0 -1  0 -1  0 -1

Алгебраическое дополнение A11:
  | 0  1  2  3  4  5  6  7  8
--|-----
0 | 2 -1  0  0  0  0  0  0
1 | -1 3 -1  0  0  0 -1  0
2 |  0 -1 3 -1  0 -1  0  0
3 |  0  0 -1 3 -1  0  0 -1
4 |  0  0  0 -1 2 -1  0  0
5 |  0  0 -1  0 -1 5 -1 -1
6 |  0 -1  0  0  0 -1 3 -1
7 |  0  0  0  0  0 -1 -1 3
8 |  0  0  0 -1  0 -1  0 -1

Число остовных деревьев: 271

```

Рис. 32: Вычисление числа остовных деревьев

Алгоритм Краскала построил минимальный остов весом 27. Остов был закодирован кодом Прюфера и затем восстановлен; полученная матрица совпала с исходной матрицей остова. Результат приведён на рис. [33](#).

```

> 13

===== Минимальный остов: алгоритм Краскала =====
Рёбра минимального остова:
1 -- 2 вес: 2
2 -- 3 вес: 2
5 -- 6 вес: 2
2 -- 7 вес: 3
6 -- 7 вес: 3
6 -- 9 вес: 3
0 -- 1 вес: 4
3 -- 4 вес: 4
6 -- 8 вес: 4

Вес минимального остова: 27

Матрица весов минимального остова:
| 0 1 2 3 4 5 6 7 8 9
-----
0 | 0 4 inf inf inf inf inf inf inf inf
1 | 4 0 2 inf inf inf inf inf inf inf
2 | inf 2 0 2 inf inf inf 3 inf inf
3 | inf inf 2 0 4 inf inf inf inf inf
4 | inf inf inf 4 0 inf inf inf inf inf
5 | inf inf inf inf inf 0 2 inf inf inf
6 | inf inf inf inf inf 2 0 3 4 3
7 | inf inf 3 inf inf inf 3 0 inf inf
8 | inf inf inf inf inf inf 4 inf 0 inf
9 | inf inf inf inf inf inf 3 inf inf 0

Код Прюфера: 1, 2, 3, 2, 7, 6, 6, 6, 9
Весы кода Прюфера: 4, 2, 4, 2, 3, 2, 3, 4, 3

Матрица весов декодированного остова:
| 0 1 2 3 4 5 6 7 8 9
-----
0 | 0 4 inf inf inf inf inf inf inf inf
1 | 4 0 2 inf inf inf inf inf inf inf
2 | inf 2 0 2 inf inf inf 3 inf inf
3 | inf inf 2 0 4 inf inf inf inf inf
4 | inf inf inf 4 0 inf inf inf inf inf
5 | inf inf inf inf inf 0 2 inf inf inf
6 | inf inf inf inf inf 2 0 3 4 3
7 | inf inf 3 inf inf inf 3 0 inf inf
8 | inf inf inf inf inf inf 4 inf 0 inf
9 | inf inf inf inf inf inf 3 inf inf 0

Проверка декодирования: остов совпадает

```

Рис. 33: Минимальный остов и проверка кода Прюфера

Алгоритм Эдмондса нашёл пять независимых рёбер в исходном графе и четыре независимых ребра в минимальном остове. Результаты приведены на рис. 34.

```

> 14
==== Алгоритм Эдмондса: максимальное независимое множество рёбер ====
Где искать:
  1 — исходный граф
  2 — минимальный остов
> 1
Размер множества: 5
Рёбра:
0 -- 1 вес: 4
2 -- 3 вес: 2
4 -- 5 вес: 5
6 -- 7 вес: 3
8 -- 9 вес: 4

Матрица независимого множества рёбер:
| 0 1 2 3 4 5 6 7 8 9
-----
0 | 0 4 inf inf inf inf inf inf inf inf
1 | 4 0 inf inf inf inf inf inf inf inf
2 | inf inf 0 2 inf inf inf inf inf inf
3 | inf inf 2 0 inf inf inf inf inf inf
4 | inf inf inf inf 0 5 inf inf inf inf
5 | inf inf inf inf 5 0 inf inf inf inf
6 | inf inf inf inf inf 0 3 inf inf inf
7 | inf inf inf inf inf inf 3 0 inf inf
8 | inf inf inf inf inf inf inf inf 0 4
9 | inf inf inf inf inf inf inf inf 4 0

```

```

> 14
==== Алгоритм Эдмондса: максимальное независимое множество рёбер ====
Где искать:
  1 — исходный граф
  2 — минимальный остов
> 2
Размер множества: 4
Рёбра:
0 -- 1 вес: 4
2 -- 7 вес: 3
3 -- 4 вес: 4
5 -- 6 вес: 2

Матрица независимого множества рёбер:
| 0 1 2 3 4 5 6 7 8 9
-----
0 | 0 4 inf inf inf inf inf inf inf inf
1 | 4 0 inf inf inf inf inf inf inf inf
2 | inf inf 0 inf inf inf inf 3 inf inf
3 | inf inf inf 0 4 inf inf inf inf inf
4 | inf inf inf 4 0 inf inf inf inf inf
5 | inf inf inf inf inf 0 2 inf inf inf
6 | inf inf inf inf inf 2 0 inf inf inf
7 | inf inf 3 inf inf inf inf 0 inf inf
8 | inf inf inf inf inf inf inf 0 inf inf
9 | inf inf inf inf inf inf inf inf 0 inf

```

Рис. 34: Максимальные независимые множества рёбер исходного графа и остова

Для построения минимального остова необходима весовая матрица. Проверка её наличия в алгоритмах Краскала и Эдмондса показана на рис. 35.

```

> 13
==== Минимальный остов: алгоритм Краскала ====
[x] Сначала сгенерируйте весовую матрицу.

```

```

==== Алгоритм Эдмондса: максимальное независимое множество рёбер ====
Где искать:
  1 — исходный граф
  2 — минимальный остов
> 2
[x] Сначала сгенерируйте весовую матрицу для построения минимального остова.

```

Рис. 35: Обработка ошибок при отсутствии весовой матрицы

4.5 Лабораторная работа № 5

Исходный граф не являлся эйлеровым. После удаления и добавления рёбер степени всех вершин стали чётными, после чего был построен эйлеров цикл. Выполненные изменения и результат показаны на рис. 36.

```

> 15

===== Проверка, является ли неориентированный граф эйлеровым, построение эйлерова цикла =====
[*] Неориентированный граф не является эйлеровым.

Изменения графа:
удалены ребра:
3 -- 4
6 -- 7
8 -- 9
добавлены ребра:
0 -- 2

Матрица смежности эйлерова графа:
| 0 1 2 3 4 5 6 7 8 9
-----
0 | 0 1 1 0 0 0 0 0 0 0
1 | 1 0 1 0 0 0 0 0 0 0
2 | 1 1 0 1 0 0 0 1 0 0
3 | 0 0 1 0 0 0 1 0 0 0
4 | 0 0 0 0 0 1 0 0 0 1
5 | 0 0 0 0 1 0 1 0 0 0
6 | 0 0 0 1 0 1 0 0 1 1
7 | 0 0 1 0 0 0 0 0 1 0
8 | 0 0 0 0 0 0 1 1 0 0
9 | 0 0 0 0 1 0 1 0 0 0

Эйлеров цикл: 0 -> 1 -> 2 -> 3 -> 6 -> 5 -> 4 -> 9 -> 6 -> 8 -> 7 -> 2 -> 0

```

Рис. 36: Преобразование графа и построение эйлерова цикла

Для графа из двух вершин единственное ребро является мостом. Поэтому программа прекращает преобразование и сообщает о невозможности сохранить связность. Исходный граф и сообщение приведены на рис. 37.

```

=== Теория графов ===
Количество вершин (>=2): 2

[*] Граф сгенерирован автоматически.
Матрица смежности:
| 0 1
-----
0 | 0 1
1 | 0 0

```

```

> 15

===== Проверка, является ли неориентированный граф эйлеровым, построение эйлерова цикла =====
[*] Невозможно завершить преобразование, не удалив мост и не нарушив связность графа.

```

Рис. 37: Проверка сохранения связности при преобразовании графа

На основе минимального остова получена фундаментальная система из девяти разрезов. Для двух выбранных разрезов вычислена симметрическая разность. Результаты приведены на рис. 38.

```

> 16

===== Фундаментальная система разрезов =====

Матрица весов исходного неориентированного графа:
| 0 1 2 3 4 5 6 7 8 9
-----
0 | 0 4 inf inf inf inf inf inf inf inf
1 | 4 0 2 inf inf inf inf inf inf inf
2 | inf 2 0 2 inf inf inf 3 inf inf
3 | inf inf 2 0 4 inf 5 inf inf inf
4 | inf inf inf 4 0 5 inf inf inf 6
5 | inf inf inf inf 5 0 2 inf inf inf
6 | inf inf inf 5 inf 2 0 3 4 3
7 | inf inf 3 inf inf inf 3 0 4 inf
8 | inf inf inf inf inf inf 4 4 0 4
9 | inf inf inf inf 6 inf 3 inf 4 0

Рёбра минимального остова:
1. 1 -- 2 вес: 2
2. 2 -- 3 вес: 2
3. 5 -- 6 вес: 2
4. 2 -- 7 вес: 3
5. 6 -- 7 вес: 3
6. 6 -- 9 вес: 3
7. 0 -- 1 вес: 4
8. 3 -- 4 вес: 4
9. 6 -- 8 вес: 4

Вес минимального остова: 27

```

```

Матрица весов минимального остова:
| 0 1 2 3 4 5 6 7 8 9
-----
0 | 0 4 inf inf inf inf inf inf inf
1 | 4 0 2 inf inf inf inf inf inf
2 | inf 2 0 2 inf inf inf 3 inf inf
3 | inf inf 2 0 4 inf inf inf inf
4 | inf inf inf 4 0 inf inf inf inf
5 | inf inf inf inf inf 0 2 inf inf
6 | inf inf inf inf inf 2 0 3 4 3
7 | inf inf 3 inf inf inf 3 0 inf inf
8 | inf inf inf inf inf inf 4 inf 0 inf
9 | inf inf inf inf inf inf 3 inf inf 0

Фундаментальные разрезы:
1. Удалено ребро остова 1 -- 2: (1, 2)
2. Удалено ребро остова 2 -- 3: (2, 3) (3, 6) (4, 5) (4, 9)
3. Удалено ребро остова 5 -- 6: (4, 5) (5, 6)
4. Удалено ребро остова 2 -- 7: (2, 7) (3, 6) (4, 5) (4, 9)
5. Удалено ребро остова 6 -- 7: (3, 6) (4, 5) (4, 9) (6, 7) (7, 8)
6. Удалено ребро остова 6 -- 9: (4, 9) (6, 9) (8, 9)
7. Удалено ребро остова 0 -- 1: (0, 1)
8. Удалено ребро остова 3 -- 4: (3, 4) (4, 5) (4, 9)
9. Удалено ребро остова 6 -- 8: (6, 8) (7, 8) (8, 9)

Количество разрезов для симметрической разности: 2
Номер разреза: 1
Номер разреза: 2
Результат симметрической разности: (1, 2) (2, 3) (3, 6) (4, 5) (4, 9)

```

Рис. 38: Построение фундаментальной системы и симметрической разности разрезов

Если весовая матрица не сгенерирована, построение фундаментальных разрезов не выполняется. Соответствующее сообщение показано на рис. 39.

```

> 16

===== Фундаментальная система разрезов =====
[x] Сначала сгенерируйте неориентированный граф и матрицу весов.

```

Рис. 39: Проверка исходных данных для построения разрезов

Заключение

В ходе выполнения лабораторных работ разработана консольная программа для генерации и исследования графов. Реализованные методы объединены общими классами графа и матрицы, а их работа проверена на сгенерированных графах.

В результате выполнены следующие задачи.

Первая лабораторная работа:

1. реализована генерация связного ациклического ориентированного графа в соответствии со смещённым распределением Вейбулла;
2. реализована генерация весовой матрицы с положительными, отрицательными или смешанными весами в соответствии со смещённым распределением Вейбулла;
3. реализовано вычисление эксцентриситетов вершин;
4. реализовано определение центра графа;
5. реализовано определение диаметральных вершин;
6. реализован метод Шимбелла для сгенерированной весовой матрицы;
7. реализован вывод максимальных и минимальных весов маршрутов, полученных методом Шимбелла, в виде весовой матрицы;
8. реализованы проверка возможности построения маршрута между заданными вершинами и подсчёт числа таких маршрутов;
9. реализовано пользовательское меню;

Вторая лабораторная работа:

10. реализован поиск точек сочленения сгенерированного графа;
11. реализован поиск кратчайшего пути между выбранными вершинами модифицированным алгоритмом Дейкстры с выводом расстояния, вектора расстояний и последовательности вершин;
12. выполнено сравнение алгоритмов по числу итераций;

Третья лабораторная работа:

13. реализована генерация матрицы пропускных способностей для ориентированного графа;
14. реализована генерация матрицы стоимостей для ориентированного графа;
15. реализован поиск максимального потока алгоритмом Форда–Фалкерсона;
16. реализовано вычисление потока минимальной стоимости величины $\lfloor 2/3 \cdot \max \rfloor$;

Четвёртая лабораторная работа:

17. реализовано вычисление числа остовных деревьев неориентированного графа по матричной теореме Кирхгофа;
18. реализовано построение минимального по весу остова алгоритмом Краскала;
19. реализованы кодирование остова кодом Прюфера с сохранением весов рёбер и его декодирование;
20. реализован поиск максимального независимого множества рёбер исходного графа и полученного остова;

Пятая лабораторная работа:

21. реализована проверка неориентированного графа на эйлеровость;
22. реализованы модификация графа с сохранением связности и вывод сведений о выполненных изменениях;
23. реализовано построение эйлерова цикла;
24. реализовано построение минимального остова алгоритмом Краскала;
25. реализованы получение и вывод фундаментальной системы разрезов;
26. реализовано получение других разрезов из фундаментальных с помощью операции симметрической разности.

Достоинства реализации

- программа написана в объектно-ориентированном стиле и имеет модульную структуру, поэтому классы графа, матрицы, генератора и консольного интерфейса можно изменять и проверять независимо;
- для каждой лабораторной работы используется единое меню, из которого можно последовательно вызывать все относящиеся к ней методы;
- вместе с итоговыми значениями программа выводит дополнительные данные, например матрицу смежности при построении фундаментальной системы разрезов, что упрощает проверку полученного результата.

Недостатки реализации

- матричное представление требует $O(p^2)$ памяти независимо от числа рёбер, поэтому неэффективно для больших разреженных графов;
- консольное представление матриц и других данных графа усложняет работу с большими графами, содержащими, например, 30 и более вершин;
- параметры распределения заданы в коде как константы, поэтому их нельзя изменить из пользовательского интерфейса.

Масштабируемость

- можно реализовать ввод параметров распределения пользователем, что позволит настраивать характеристики генерируемых графов без изменения кода;
- можно реализовать графический пользовательский интерфейс, например с помощью Qt, для визуализации графов и более удобного выбора операций;
- можно хранить граф в виде списка смежности и адаптировать для такого представления отдельные алгоритмы, что позволит сократить затраты памяти и повысить эффективность их работы на разреженных графах.

Таким образом, поставленные в лабораторных работах задачи выполнены.

Список литературы

- [1] Новиков Ф. А. Дискретная математика для программистов: Учебник для вузов. 3-е изд. — СПб.: Питер, 2009. — 384 с.
- [2] Laaksonen A. Competitive Programmer's Handbook. 2018. 296 p.
- [3] Теория графов: лекционные материалы // Секция «Телематика» СПбПУ [Электронный ресурс]. URL: https://tema.spbstu.ru/tgraph_lect/ (дата обращения: 19.08.2026).
- [4] Feil J., Riedl W. F. Edmonds's Blossom Algorithm // Technische Universität München [Электронный ресурс]. URL: <https://algorithms.discrete.ma.tum.de/matching/blossom/> (дата обращения: 19.08.2026).
- [5] Hierholzer's Algorithm: Computing Eulerian Cycles // Technische Universität München [Электронный ресурс]. URL: <https://algorithms.discrete.ma.tum.de/routing/hierholzer/> (дата обращения: 19.08.2026).

Приложение. Исходный код программы

Листинг 21: Константы программы

```
1  #pragma once
2
3  /*
4   * SCALE -> a > 0
5   * SHAPE -> c > 0
6   * SHIFT -> y0
7   */
8
9  struct WeibullParams {
10     double scale;
11     double shape;
12     double shift;
13 };
14
15 namespace constants {
16
17     inline constexpr WeibullParams graph {
18         .scale = 0.17,
19         .shape = 2.0,
20         .shift = 0.05
21     };
22
23     inline constexpr WeibullParams weight {
24         .scale = 0.3,
25         .shape = 2.0,
26         .shift = 0.1
27     };
28
29     inline constexpr WeibullParams capacity {
30         .scale = 0.5,
31         .shape = 2.0,
32         .shift = 0.2
33     };
34
35     inline constexpr WeibullParams cost {
36         .scale = 0.4,
37         .shape = 2.0,
38         .shift = 0.1
39     };
40
41 }
```

Листинг 22: Структуры данных для работы с потоками

```
1  #pragma once
2
3  #include "matrix.h"
4
5  struct MaxFlowResult {
6     Matrix flowMatrix;
7
8     int maxFlow = 0;
9     int iterations = 0;
10
11     MaxFlowResult(int vertexCount)
12         : flowMatrix(vertexCount, vertexCount, 0)
13     {}
14 };
15
16 struct MinCostFlowResult {
17     Matrix flowMatrix;
18     Matrix costFlowMatrix;
19
20     int requiredFlow = 0;
21     int achievedFlow = 0;
22     int totalCost = 0;
23     int iterations = 0;
24
25     bool success = false;
26
27     MinCostFlowResult(int vertexCount)
28         : flowMatrix(vertexCount, vertexCount, 0)
29         , costFlowMatrix(vertexCount, vertexCount, 0)
30     {}
31
32     };
33 }
```

Листинг 23: Объявление класса матрицы

```
1  #pragma once
2
3  #include <vector>
4  #include <iostream>
5  #include <limits>
6  enum class PrintMode { Integer, Double };
7
8  class Matrix {
9  public:
10     Matrix(int rows, int cols, double fill = 0);
```

```

11
12     int rows() const;
13     int cols() const;
14
15     double& operator()(int i, int j);
16     double operator()(int i, int j) const;
17
18     Matrix operator*(const Matrix& other) const;
19     Matrix multiply(const Matrix& other) const;
20     Matrix shimbellMin(const Matrix& other) const;
21     Matrix shimbellMax(const Matrix& other) const;
22
23     double determinant() const;
24
25     void print(PrintMode mode = PrintMode::Integer) const;
26
27 private:
28     using m_BinaryOp = double (*)(double, double);
29
30     int m_rows, m_cols;
31     std::vector<std::vector<double>> m_data;
32
33     Matrix m_multiply(const Matrix& other,
34                       m_BinaryOp combine,
35                       m_BinaryOp aggregate,
36                       double identity) const;
37 };

```

Листинг 24: Реализация класса матрицы

```

1  #include "matrix.h"
2  #include <iomanip>
3  #include <algorithm>
4  #include <cmath>
5  #include <stdexcept>
6
7  namespace {
8  constexpr double ZERO_EPS = 1e-12;
9
10 bool isZero(double value) {
11     return std::abs(value) < ZERO_EPS;
12 }
13
14 std::vector<std::vector<double>> buildMinor(
15     const std::vector<std::vector<double>>& data,
16     int excludedRow,
17     int excludedCol
18 ) {
19     const int n = static_cast<int>(data.size());
20     std::vector<std::vector<double>> minor;
21     minor.reserve(n - 1);
22
23     for (int i = 0; i < n; ++i) {
24         if (i == excludedRow) {
25             continue;
26         }
27
28         std::vector<double> row;
29         row.reserve(n - 1);
30
31         for (int j = 0; j < n; ++j) {
32             if (j != excludedCol) {
33                 row.push_back(data[i][j]);
34             }
35         }
36
37         minor.push_back(row);
38     }
39
40     return minor;
41 }
42
43 double determinantRecursive(
44     const std::vector<std::vector<double>>& data
45 ) {
46     const int n = static_cast<int>(data.size());
47
48     if (n == 0) {
49         return 1;
50     }
51
52     if (n == 1) {
53         return data[0][0];
54     }
55
56     if (n == 2) {
57         return data[0][0] * data[1][1]
58             - data[0][1] * data[1][0];
59     }
60
61     bool expandByRow = true;
62     int expandIndex = 0;
63     int maxZeros = -1;
64
65     for (int i = 0; i < n; ++i) {
66         int zeros = 0;
67
68         for (int j = 0; j < n; ++j) {
69             if (isZero(data[i][j])) {
70                 ++zeros;
71             }
72         }

```

```

73
74     if (zeros > maxZeros) {
75         maxZeros = zeros;
76         expandByRow = true;
77         expandIndex = i;
78     }
79 }
80
81 for (int j = 0; j < n; ++j) {
82     int zeros = 0;
83
84     for (int i = 0; i < n; ++i) {
85         if (isZero(data[i][j])) {
86             ++zeros;
87         }
88     }
89
90     if (zeros > maxZeros) {
91         maxZeros = zeros;
92         expandByRow = false;
93         expandIndex = j;
94     }
95 }
96
97 if (maxZeros == n) {
98     return 0;
99 }
100
101 double determinant = 0;
102
103 if (expandByRow) {
104     const int row = expandIndex;
105
106     for (int col = 0; col < n; ++col) {
107         if (isZero(data[row][col])) {
108             continue;
109         }
110
111         const double sign = ((row + col) % 2 == 0) ? 1 : -1;
112         determinant += sign
113             * data[row][col]
114             * determinantRecursive(buildMinor(data, row, col));
115     }
116 } else {
117     const int col = expandIndex;
118
119     for (int row = 0; row < n; ++row) {
120         if (isZero(data[row][col])) {
121             continue;
122         }
123
124         const double sign = ((row + col) % 2 == 0) ? 1 : -1;
125         determinant += sign
126             * data[row][col]
127             * determinantRecursive(buildMinor(data, row, col));
128     }
129 }
130
131 return determinant;
132 }
133 }
134
135 Matrix::Matrix(int rows, int cols, double fill)
136 : m_rows(rows), m_cols(cols)
137 , m_data(rows, std::vector<double>(cols, fill))
138 {}
139
140 int Matrix::rows() const { return m_rows; }
141 int Matrix::cols() const { return m_cols; }
142
143 double& Matrix::operator()(int i, int j) { return m_data[i][j]; }
144 double Matrix::operator()(int i, int j) const { return m_data[i][j]; }
145
146 Matrix Matrix::m_multiply(const Matrix& other,
147                           m_BinaryOp combine,
148                           m_BinaryOp aggregate,
149                           double identity) const {
150     Matrix result(m_rows, other.m_cols, identity);
151
152     // в не Шимбелловой матрице identity - нейтральный элемент по сложению (ноль)
153     for (int i = 0; i < m_rows; ++i) {
154         for (int j = 0; j < other.m_cols; ++j) {
155             for (int k = 0; k < m_cols; ++k) {
156                 if (m_data[i][k] != identity && other(k, j) != identity) {
157                     double val = combine(m_data[i][k], other(k, j));
158                     result(i, j) = aggregate(result(i, j), val);
159                 }
160             }
161         }
162     }
163
164     return result;
165 }
166
167 Matrix Matrix::multiply(const Matrix& other) const {
168     return m_multiply(other,
169         [](double a, double b) { return a * b; },
170         [](double a, double b) { return a + b; },
171         0);
172 }
173
174 Matrix Matrix::operator*(const Matrix& other) const {
175     return multiply(other);
176 }
177
178 Matrix Matrix::shimbellMin(const Matrix& other) const {
179     return m_multiply(other,

```

```

180     (double a, double b) { return a + b; },
181     (double a, double b) { return std::min(a, b); },
182     std::numeric_limits<double>::infinity();
183 }
184
185 Matrix Matrix::shimbellMax(const Matrix& other) const {
186     return m_multiply(other,
187         (double a, double b) { return a + b; },
188         (double a, double b) { return std::max(a, b); },
189         -std::numeric_limits<double>::infinity());
190 }
191
192 double Matrix::determinant() const {
193     if (m_rows != m_cols) {
194         throw std::invalid_argument(
195             "determinant is defined only for square matrices"
196         );
197     }
198
199     return determinantRecursive(m_data);
200 }
201
202 void Matrix::print(PrintMode mode) const {
203     const int cellWidth = (mode == PrintMode::Double) ? 9 : 5;
204     const int labelWidth = 3;
205
206     std::cout << std::setw(labelWidth) << "" << " |";
207     for (int j = 0; j < m_cols; ++j) {
208         std::cout << std::setw(cellWidth) << std::defaultfloat << j;
209     }
210     std::cout << "\n";
211
212     std::cout << std::string(labelWidth + 1, '-') << "+"
213         << std::string(cellWidth * m_cols, '-') << "\n";
214
215     for (int i = 0; i < m_rows; ++i) {
216         std::cout << std::setw(labelWidth) << std::defaultfloat << i << " |";
217         for (int j = 0; j < m_cols; ++j) {
218             double val = m_data[i][j];
219             if (std::isinf(val)) {
220                 std::cout << std::setw(cellWidth) << "inf";
221             } else if (mode == PrintMode::Double) {
222                 std::cout << std::setw(cellWidth) << std::fixed << std::setprecision(1) << val;
223             } else {
224                 std::cout << std::setw(cellWidth) << std::defaultfloat << static_cast<int>(val);
225             }
226         }
227         std::cout << "\n";
228     }
229 }

```

Листинг 25: Объявление класса графа

```

1  #pragma once
2
3  #include "matrix.h"
4
5  #include <vector>
6
7  struct WeightedNeighbor {
8      int vertex = 0;
9      double weight = 0;
10 };
11
12 using AdjacencyList = std::vector<std::vector<int>>>;
13 using WeightedAdjacencyList = std::vector<std::vector<WeightedNeighbor>>>;
14
15 class Graph {
16 public:
17     explicit Graph(int vertexCount);
18     virtual ~Graph() = default;
19
20     int getVertexCount() const;
21     const Matrix& getAdjacencyMatrix() const;
22     const Matrix& getWeightMatrix() const;
23     const Matrix& getUndirectedAdjacencyMatrix() const;
24     const Matrix& getUndirectedWeightMatrix() const;
25     AdjacencyList getAdjacencyList() const;
26     WeightedAdjacencyList getWeightedAdjacencyList() const;
27
28     void printAdjacencyMatrix() const;
29     void printWeightMatrix() const;
30     void printUndirectedAdjacencyMatrix() const;
31     void printUndirectedWeightMatrix() const;
32
33     struct isMatrixInit {
34         bool adjacency = false;
35         bool weight = false;
36         bool capacity = false;
37         bool cost = false;
38         bool eulerian = false;
39     } isMatrixInit;
40
41 protected:
42     int m_vertexCount;
43     Matrix m_adjacencyMatrix;
44     Matrix m_weightMatrix;
45     Matrix m_undirectedAdjacencyMatrix;
46     Matrix m_undirectedWeightMatrix;
47
48     void m_resetAdjacencyMatrix();
49     void m_resetWeightMatrix();

```

```

50     void m_syncUndirectedAdjacencyMatrix();
51     void m_syncUndirectedWeightMatrix();
52
53     template<typename Item, typename ItemFactory>
54     std::vector<std::vector<Item>> m_matrixToAdjacencyList(
55         const Matrix& adjacency,
56         ItemFactory createItem
57     ) const {
58         std::vector<std::vector<Item>> result(adjacency.rows());
59
60         for (int from = 0; from < adjacency.rows(); ++from) {
61             for (int to = 0; to < adjacency.cols(); ++to) {
62                 if (from != to && adjacency(from, to) != 0) {
63                     result[from].push_back(createItem(from, to));
64                 }
65             }
66         }
67
68         return result;
69     }
70 };

```

Листинг 26: Реализация класса графа

```

1  #include "graph.h"
2  #include <algorithm>
3  #include <cmath>
4  #include <limits>
5
6  Graph::Graph(int vertexCount)
7      : m_vertexCount(vertexCount)
8      , m_adjacencyMatrix(vertexCount, vertexCount)
9      , m_weightMatrix(vertexCount, vertexCount)
10      , m_undirectedAdjacencyMatrix(vertexCount, vertexCount)
11      , m_undirectedWeightMatrix(vertexCount, vertexCount)
12  {}
13
14  int Graph::getVertexCount() const {
15      return m_vertexCount;
16  }
17
18  const Matrix& Graph::getAdjacencyMatrix() const {
19      return m_adjacencyMatrix;
20  }
21
22  const Matrix& Graph::getWeightMatrix() const {
23      return m_weightMatrix;
24  }
25
26  const Matrix& Graph::getUndirectedAdjacencyMatrix() const {
27      return m_undirectedAdjacencyMatrix;
28  }
29
30  const Matrix& Graph::getUndirectedWeightMatrix() const {
31      return m_undirectedWeightMatrix;
32  }
33
34  AdjacencyList Graph::getAdjacencyList() const {
35      return m_matrixToAdjacencyList<int>(
36          m_adjacencyMatrix,
37          [](int, int to) {
38              return to;
39          }
40      );
41  }
42
43  WeightedAdjacencyList Graph::getWeightedAdjacencyList() const {
44      return m_matrixToAdjacencyList<WeightedNeighbor>(
45          m_adjacencyMatrix,
46          [&](int from, int to) {
47              return WeightedNeighbor{to, m_weightMatrix(from, to)};
48          }
49      );
50  }
51
52  void Graph::printAdjacencyMatrix() const {
53      std::cout << "Матрица смежности:\n";
54      m_adjacencyMatrix.print();
55  }
56
57  void Graph::printWeightMatrix() const {
58      std::cout << "Матрица весов:\n";
59      m_weightMatrix.print();
60  }
61
62  void Graph::printUndirectedAdjacencyMatrix() const {
63      std::cout << "Матрица смежности неориентированного графа:\n";
64      m_undirectedAdjacencyMatrix.print();
65  }
66
67  void Graph::printUndirectedWeightMatrix() const {
68      std::cout << "Матрица весов неориентированного графа:\n";
69      m_undirectedWeightMatrix.print();
70  }
71
72  void Graph::m_resetAdjacencyMatrix() {
73      m_adjacencyMatrix = Matrix(m_vertexCount, m_vertexCount);
74      m_undirectedAdjacencyMatrix = Matrix(m_vertexCount, m_vertexCount);
75      isMatrixInit.adjacency = false;
76  }
77
78  void Graph::m_resetWeightMatrix() {

```

```

79     m_weightMatrix = Matrix(m_vertexCount, m_vertexCount);
80     m_undirectedWeightMatrix = Matrix(m_vertexCount, m_vertexCount);
81     isMatrixInit.weight = false;
82 }
83
84 void Graph::m_syncUndirectedAdjacencyMatrix() {
85     m_undirectedAdjacencyMatrix = Matrix(m_vertexCount, m_vertexCount);
86
87     for (int i = 0; i < m_vertexCount; ++i) {
88         for (int j = i + 1; j < m_vertexCount; ++j) {
89             if (m_adjacencyMatrix(i, j) != 0 ||
90                 m_adjacencyMatrix(j, i) != 0) {
91                 m_undirectedAdjacencyMatrix(i, j) = 1;
92                 m_undirectedAdjacencyMatrix(j, i) = 1;
93             }
94         }
95     }
96 }
97
98 void Graph::m_syncUndirectedWeightMatrix() {
99     const double INF = std::numeric_limits<double>::infinity();
100
101     m_undirectedWeightMatrix = Matrix(m_vertexCount, m_vertexCount, INF);
102
103     for (int i = 0; i < m_vertexCount; ++i) {
104         m_undirectedWeightMatrix(i, i) = 0;
105     }
106
107     for (int i = 0; i < m_vertexCount; ++i) {
108         for (int j = i + 1; j < m_vertexCount; ++j) {
109             double weight = INF;
110
111             if (!std::isinf(m_weightMatrix(i, j))) {
112                 weight = m_weightMatrix(i, j);
113             }
114
115             if (!std::isinf(m_weightMatrix(j, i))) {
116                 weight = std::min(weight, m_weightMatrix(j, i));
117             }
118
119             if (!std::isinf(weight)) {
120                 m_undirectedWeightMatrix(i, j) = weight;
121                 m_undirectedWeightMatrix(j, i) = weight;
122             }
123         }
124     }
125 }

```

Листинг 27: Объявление генератора графа

```

1  #pragma once
2
3  #include "graph.h"
4  #include "constants.h"
5  #include "flows.h"
6  #include <random>
7  #include <vector>
8  #include <queue>
9  #include <limits>
10 #include <algorithm>
11 #include <functional>
12 #include <set>
13
14 enum class WeightMode {
15     Positive,
16     Negative,
17     Mixed
18 };
19
20 struct ShortestPathResult {
21     std::vector<double> T;
22     std::vector<int> H;
23     std::vector<int> path;
24
25     double distance = 0;
26     int iterations = 0;
27
28     bool hasPath = false;
29     bool hasNegativeCycle = false;
30 };
31
32 struct ArticulationPointsResult {
33     std::vector<int> points;
34     int iterations = 0;
35 };
36
37 // lab4
38 struct SpanningTreesResult {
39     Matrix kirchhoffMatrix;
40     Matrix cofactorMatrix;
41
42     double cofactorDeterminant = 0;
43     long long count = 0;
44
45     SpanningTreesResult(int vertexCount)
46         : kirchhoffMatrix(vertexCount, vertexCount, 0)
47         , cofactorMatrix(
48             std::max(0, vertexCount - 1),
49             std::max(0, vertexCount - 1),
50             0
51         )
52     {}

```



```

53 };
54
55 struct Edge {
56     int from = 0;
57     int to = 0;
58
59     Edge() = default;
60     Edge(int from, int to) : from(from), to(to) {};
61
62     bool operator<(const Edge& other) const {
63         return from < other.from ||
64             (from == other.from && to < other.to);
65     }
66
67     bool operator==(const Edge& other) const {
68         return from == other.from && to == other.to;
69     }
70 };
71
72 struct WeightedEdge {
73     int from = 0;
74     int to = 0;
75     double weight = 0;
76 };
77
78 struct PruferCodeItem {
79     int vertex = 0;
80     double weight = 0;
81 };
82
83 struct KruskalResult {
84     std::vector<WeightedEdge> edges;
85     Matrix treeMatrix;
86     double totalWeight = 0;
87     bool success = false;
88
89     KruskalResult(int vertexCount)
90         : treeMatrix(
91             vertexCount,
92             vertexCount,
93             std::numeric_limits<double>::infinity()
94         )
95     {}
96 };
97
98 struct KruskalMinimumSpanningTreeResult {
99     KruskalResult kruskal;
100
101     std::vector<PruferCodeItem> pruferCode;
102     Matrix decodedTreeMatrix;
103     bool pruferRoundTripSuccess = false;
104
105     KruskalMinimumSpanningTreeResult(KruskalResult&& other, int vertexCount)
106         : kruskal(std::move(other)),
107           decodedTreeMatrix(
108               vertexCount,
109               vertexCount,
110               std::numeric_limits<double>::infinity()
111           )
112     {}
113 };
114
115 struct EdmondsBlossomResult {
116     std::vector<WeightedEdge> edges;
117     Matrix edgeMatrix;
118
119     bool success = true;
120     bool hasWeights = false;
121
122     EdmondsBlossomResult(int vertexCount)
123         : edgeMatrix(vertexCount, vertexCount, 0)
124     {}
125 };
126
127 // lab5
128 struct CheckIfEulerianGraphResult {
129     bool isEulerian = false;
130     bool transformationCompleted = true;
131     std::vector<Edge> addedEdges;
132     std::vector<Edge> removedEdges;
133     std::vector<int> eulerianCycle;
134     Matrix eulerianAdjacencyMatrix;
135
136     CheckIfEulerianGraphResult(
137         bool isEulerian,
138         bool transformationCompleted,
139         std::vector<Edge>&& added,
140         std::vector<Edge>&& removed,
141         std::vector<int>&& cycle,
142         Matrix&& adjacency
143     )
144         : isEulerian(isEulerian),
145           transformationCompleted(transformationCompleted),
146           addedEdges(std::move(added)),
147           removedEdges(std::move(removed)),
148           eulerianCycle(std::move(cycle)),
149           eulerianAdjacencyMatrix(std::move(adjacency))
150     {}
151 };
152
153 using FleuryResult = std::vector<int>;
154 using GraphCut = std::set<Edge>;
155
156 struct FundamentalCut {
157     Edge removedTreeEdge;
158     GraphCut cut;
159 };

```

```

160
161 struct FundamentalCutsResult {
162     KruskalResult spanningTree;
163     std::vector<FundamentalCut> fundamentalCuts;
164     bool success = false;
165
166     explicit FundamentalCutsResult(int vertexCount)
167         : spanningTree(vertexCount)
168     {}
169 };
170
171 class GeneratorGraph : public Graph {
172 public:
173     GeneratorGraph(int vertexCount);
174
175     void generate();
176
177     void computeEccentricities();
178     std::vector<double> getEccentricities() const;
179     std::vector<int> getCenterVertices() const;
180     std::vector<int> getDiametralVertices() const;
181     int getDiameter() const;
182
183     void generateWeightMatrix(WeightMode mode);
184     Matrix shimbell(int steps, bool findMin) const;
185
186     bool hasRoute(int from, int to) const;
187     int countRoutes(int from, int to) const;
188     void testDistribution();
189
190     //lab2
191     ArticulationPointsResult findArticulationPoints() const;
192     ShortestPathResult dijkstraNegative(int s, int t) const;
193     ShortestPathResult dijkstraNegative(int s, int t, const Matrix& adjacency) const;
194     ShortestPathResult dijkstraNegative(int s,
195                                         int t,
196                                         const Matrix& adjacency,
197                                         const Matrix& weight) const;
198
199     // lab3
200     void generateCapacityAndCostMatrices();
201
202     const Matrix& getCapacityMatrix() const;
203     const Matrix& getCostMatrix() const;
204
205     void printCapacityMatrix() const;
206     void printCostMatrix() const;
207
208     bool isCapacityMatrixGenerated() const;
209     bool isCostMatrixGenerated() const;
210
211     // Потокковые алгоритмы
212     MaxFlowResult fordFulkerson(int source, int sink) const;
213
214     MinCostFlowResult minCostFlow(
215         int source,
216         int sink,
217         int requiredFlow
218     ) const;
219
220     // lab4
221     SpanningTreesResult countSpanningTreesKirchhoff() const;
222     KruskalMinimumSpanningTreeResult kruskalMinimumSpanningTree() const;
223     EdmondsBlossomResult edmondsBlossomInOriginalGraph() const;
224     EdmondsBlossomResult edmondsBlossomInMinimumSpanningTree() const;
225
226     //lab5
227     CheckIfEulerianGraphResult checkIfEulerianGraph(
228         bool allowDisconnectedResult = false
229     ) const;
230
231     FundamentalCutsResult buildFundamentalCutSystem() const;
232     GraphCut symmetricDifferenceOfFundamentalCuts(
233         const FundamentalCutsResult& system,
234         const std::vector<int>& selectedCutIndices
235     ) const;
236
237 private:
238     std::mt19937 m_rng;
239     std::uniform_real_distribution<double> m_uniformDist;
240
241     std::vector<double> m_eccentricities;
242     std::vector<int> m_centerVertices;
243     std::vector<int> m_diametralVertices;
244     int m_diameter = 0;
245
246     double m_sampleWeibull(const WeibullParams& parameters);
247     std::vector<int> m_generateDegreeSequence();
248
249     //lab2
250     void m_dfsArticulation(int v,
251                             int parent,
252                             std::vector<int>& tin,
253                             std::vector<int>& low,
254                             std::vector<bool>& visited,
255                             std::vector<bool>& isAP,
256                             int& timer,
257                             int& iterations) const;
258     std::vector<int> m_restorePath(int s, int t, const std::vector<int>& H) const;
259
260     // lab3
261     Matrix m_capacityMatrix;
262     Matrix m_costMatrix;
263
264     // lab4
265     KruskalResult m_kruskalAlgorithm() const;
266     EdmondsBlossomResult m_edmondsBlossom(

```

```

267         const Matrix& adjacency,
268         const Matrix& weights,
269         bool hasWeights
270     ) const;
271     std::vector<PruferCodeItem> m_encodePruferCode(
272         const std::vector<WeightedEdge>& treeEdges
273     ) const;
274     Matrix m_decodePruferCode(
275         const std::vector<PruferCodeItem>& code
276     ) const;
277
278     // lab5
279     FleuryResult m_fleuryAlgorithm(const Matrix& adjacency) const;
280     GraphCut m_symmetricDifference(
281         const GraphCut& first,
282         const GraphCut& second
283     ) const;
284 };

```

Листинг 28: Реализация генератора графа

```

1  #include "generator_graph.h"
2  #include <cmath>
3  #include <set>
4  #include <stack>
5  #include <stdexcept>
6  #include <vector>
7
8  namespace {
9      bool matricesEqual(const Matrix& lhs, const Matrix& rhs) {
10         if (lhs.rows() != rhs.rows() || lhs.cols() != rhs.cols()) {
11             return false;
12         }
13
14         for (int i = 0; i < lhs.rows(); ++i) {
15             for (int j = 0; j < lhs.cols(); ++j) {
16                 const double left = lhs(i, j);
17                 const double right = rhs(i, j);
18
19                 if (std::isinf(left) || std::isinf(right)) {
20                     if (!(std::isinf(left) && std::isinf(right))) {
21                         return false;
22                     }
23                 } else if (std::abs(left - right) > 1e-9) {
24                     return false;
25                 }
26             }
27         }
28
29         return true;
30     }
31 }
32
33 GeneratorGraph::GeneratorGraph(int vertexCount)
34     : Graph(vertexCount),
35       m_rng(std::random_device{}()),
36       m_uniformDist(0, 1),
37       m_diameter(0),
38       m_capacityMatrix(vertexCount, vertexCount, 0),
39       m_costMatrix(vertexCount, vertexCount, 0)
40 {}
41
42 double GeneratorGraph::m_sampleWeibull(const WeibullParams& parameters) {
43     double r;
44     do {
45         r = m_uniformDist(m_rng);
46     } while (r >= 1);
47
48     return parameters.shift + parameters.scale * std::pow(-std::log(1 - r), 1 / parameters.shape);
49 }
50
51 std::vector<int> GeneratorGraph::m_generateDegreeSequence() {
52     std::vector<int> degrees(m_vertexCount);
53
54     bool valid;
55     do {
56         valid = true;
57         for (int i = 0; i < m_vertexCount; ++i) {
58             double raw = m_sampleWeibull(constants::graph);
59             degrees[i] = static_cast<int>(std::round(raw * (m_vertexCount - 1)));
60
61             if (degrees[i] > m_vertexCount - 1 || degrees[i] == 0) {
62                 valid = false;
63                 break;
64             }
65         }
66     } while (!valid);
67
68     return degrees;
69 }
70
71 void GeneratorGraph::testDistribution() {
72     std::vector<int> degrees(m_vertexCount);
73
74     for (int i = 0; i < 30; ++i) {
75         std::cout << "Генерация №" << (i + 1) << ":\n";
76         for (int j = 0; j < m_vertexCount; ++j) {
77             degrees = m_generateDegreeSequence();
78         }
79
80         for (auto e : degrees) std::cout << e << " ";
81         std::cout << "\n";

```

```

82     }
83 }
84
85 void GeneratorGraph::generate() {
86     m_adjacencyMatrix = Matrix(m_vertexCount, m_vertexCount);
87     auto degrees = m_generateDegreeSequence();
88
89     // гарантирует связность
90     for (int i = 0; i < m_vertexCount - 1; ++i) {
91         m_adjacencyMatrix(i, i + 1) = 1;
92     }
93
94     // добор оставшихся рёбер
95     for (int i = 0; i < m_vertexCount; ++i) {
96         int alreadyAdded = (i < m_vertexCount - 1) ? 1 : 0;
97
98         std::vector<int> candidates;
99         for (int j = i + 1; j < m_vertexCount; ++j) {
100             // если ещё не добавлено
101             if (m_adjacencyMatrix(i, j) == 0) {
102                 candidates.push_back(j);
103             }
104         }
105         std::shuffle(candidates.begin(), candidates.end(), m_rng);
106
107         int needToAdd = std::max(0, degrees[i] - alreadyAdded);
108         int edgesAdded = 0;
109         for (int j : candidates) {
110             if (edgesAdded >= needToAdd) break;
111             m_adjacencyMatrix(i, j) = 1;
112             ++edgesAdded;
113         }
114     }
115
116     m_syncUndirectedAdjacencyMatrix();
117     isMatrixInit.adjacency = true;
118 }
119
120 void GeneratorGraph::computeEccentricities() {
121     const Matrix& A = m_adjacencyMatrix;
122
123     const int UNREACHABLE = -1;
124     Matrix dist(m_vertexCount, m_vertexCount, UNREACHABLE);
125     for (int i = 0; i < m_vertexCount; ++i) dist(i, i) = 0;
126
127     Matrix Ak = A;
128     for (int k = 1; k < m_vertexCount; ++k) {
129         for (int i = 0; i < m_vertexCount; ++i) {
130             for (int j = 0; j < m_vertexCount; ++j) {
131                 if (i != j && dist(i, j) == UNREACHABLE && Ak(i, j) > 0) {
132                     dist(i, j) = k;
133                 }
134             }
135         }
136         if (k < m_vertexCount - 1) Ak = Ak * A;
137     }
138
139     m_eccentricities.assign(m_vertexCount, 0);
140     for (int i = 0; i < m_vertexCount; ++i) {
141         int maxDist = 0;
142         for (int j = 0; j < m_vertexCount; ++j) {
143             if (i != j && dist(i, j) != UNREACHABLE) {
144                 maxDist = std::max(maxDist, static_cast<int>(dist(i, j)));
145             }
146         }
147         m_eccentricities[i] = maxDist;
148     }
149
150     m_diameter = *std::max_element(m_eccentricities.begin(), m_eccentricities.end());
151     int radius = *std::min_element(m_eccentricities.begin(), m_eccentricities.end());
152
153     m_centerVertices.clear();
154     m_diametralVertices.clear();
155     for (int i = 0; i < m_vertexCount; ++i) {
156         if (m_eccentricities[i] == radius) m_centerVertices.push_back(i);
157         if (m_eccentricities[i] == m_diameter) m_diametralVertices.push_back(i);
158     }
159 }
160
161 std::vector<double> GeneratorGraph::getEccentricities() const { return m_eccentricities; }
162 std::vector<int> GeneratorGraph::getCenterVertices() const { return m_centerVertices; }
163 std::vector<int> GeneratorGraph::getDiametralVertices() const { return m_diametralVertices; }
164 int GeneratorGraph::getDiameter() const { return m_diameter; }
165
166 void GeneratorGraph::generateWeightMatrix(WeightMode mode) {
167     const double INF = std::numeric_limits<double>::infinity();
168
169     m_weightMatrix = Matrix(m_vertexCount, m_vertexCount, INF);
170
171     std::uniform_int_distribution<int> signDist(0, 1);
172
173     for (int i = 0; i < m_vertexCount; ++i) {
174         m_weightMatrix(i, i) = 0;
175
176         for (int j = 0; j < m_vertexCount; ++j) {
177             if (i == j) continue;
178
179             if (m_adjacencyMatrix(i, j) != 0) {
180                 const int WEIGHT_MATRIX_DIST_VALUE = 10;
181                 double w = std::round(m_sampleWeibull(constants::weight) * WEIGHT_MATRIX_DIST_VALUE);
182
183                 switch (mode) {
184                     case WeightMode::Positive:
185                         m_weightMatrix(i, j) = w;
186                         break;
187
188                     case WeightMode::Negative:

```

```

189         m_weightMatrix(i, j) = -w;
190         break;
191     }
192     case WeightMode::Mixed:
193         m_weightMatrix(i, j) = signDist(m_rng) ? w : -w;
194         break;
195     }
196 }
197 }
198 }
199
200 isMatrixInit.weight = true;
201 m_syncUndirectedWeightMatrix();
202 }
203
204 Matrix GeneratorGraph::shimbell(int steps, bool findMin) const {
205     const double NO_EDGE = findMin
206         ? std::numeric_limits<double>::infinity()
207         : -std::numeric_limits<double>::infinity();
208
209     Matrix W(m_vertexCount, m_vertexCount, NO_EDGE);
210
211     if (steps == 0) {
212         for (int i = 0; i < m_vertexCount; ++i) {
213             W(i, i) = 0;
214         }
215         return W;
216     }
217
218     for (int i = 0; i < m_vertexCount; ++i) {
219         for (int j = 0; j < m_vertexCount; ++j) {
220             if (m_adjacencyMatrix(i, j) != 0) {
221                 W(i, j) = m_weightMatrix(i, j);
222             }
223         }
224     }
225
226     Matrix result = W;
227
228     for (int s = 1; s < steps; ++s) {
229         result = findMin ? result.shimbellMin(W) : result.shimbellMax(W);
230     }
231
232     for (int i = 0; i < m_vertexCount; ++i) {
233         result(i, i) = 0;
234     }
235
236     return result;
237 }
238
239 bool GeneratorGraph::hasRoute(int from, int to) const {
240     if (from == to) return true;
241
242     Matrix Ak = m_adjacencyMatrix;
243     for (int k = 1; k < m_vertexCount; ++k) {
244         if (Ak(from, to) > 0) return true;
245         if (k < m_vertexCount - 1) Ak = Ak * m_adjacencyMatrix;
246     }
247     return false;
248 }
249
250 int GeneratorGraph::countRoutes(int from, int to) const {
251     if (from == to) return 1;
252
253     int total = 0;
254     Matrix Ak = m_adjacencyMatrix;
255     for (int k = 1; k < m_vertexCount; ++k) {
256         total += static_cast<int>(Ak(from, to));
257         if (k < m_vertexCount - 1) Ak = Ak * m_adjacencyMatrix;
258     }
259     return total;
260 }
261
262 void GeneratorGraph::m_dfsArticulation(int v,
263     int parent,
264     std::vector<int>& tin,
265     std::vector<int>& low,
266     std::vector<bool>& visited,
267     std::vector<bool>& isAP,
268     int& timer,
269     int& iterations) const {
270     visited[v] = true;
271     tin[v] = low[v] = timer++;
272
273     int children = 0;
274
275     for (int u = 0; u < m_vertexCount; ++u) {
276         iterations++;
277
278         if (u == v) continue;
279
280         if (m_adjacencyMatrix(v, u) == 0 && m_adjacencyMatrix(u, v) == 0) {
281             continue;
282         }
283
284         if (u == parent) {
285             continue;
286         }
287
288         if (visited[u]) {
289             low[v] = std::min(low[v], tin[u]);
290         } else {
291             ++children;
292
293             m_dfsArticulation(u, v, tin, low, visited, isAP, timer, iterations);
294
295             low[v] = std::min(low[v], low[u]);

```

```

296         if (parent != -1 && low[u] >= tin[v]) {
297             isAP[v] = true;
298         }
299     }
300 }
301 }
302
303 if (parent == -1 && children > 1) {
304     isAP[v] = true;
305 }
306 }
307
308 // упрощенный упрощённый алгоритм Тарьяна (лек. 4 стр. 11 - полный)
309 // не использует стек рёбер
310 // квадратичная сложность из-за использования матрицы смежности, со списками сложность  $O(V + E)$ 
311 ArticulationPointsResult GeneratorGraph::findArticulationPoints() const {
312     std::vector<int> tin(m_vertexCount, -1);
313     std::vector<int> low(m_vertexCount, -1);
314     std::vector<bool> visited(m_vertexCount, false);
315     std::vector<bool> isAP(m_vertexCount, false);
316
317     int timer = 0;
318     ArticulationPointsResult result;
319
320     for (int v = 0; v < m_vertexCount; ++v) {
321         if (!visited[v]) {
322             m_dfsArticulation(v, -1, tin, low, visited, isAP, timer, result.iterations);
323         }
324     }
325
326     for (int v = 0; v < m_vertexCount; ++v) {
327         if (isAP[v]) {
328             result.points.push_back(v);
329         }
330     }
331
332     return result;
333 }
334
335 std::vector<int> GeneratorGraph::m_restorePath(int s,
336                                                int t,
337                                                const std::vector<int>& H) const {
338     std::vector<int> path;
339
340     for (int v = t; v != -1; v = H[v]) {
341         path.push_back(v);
342
343         if (v == s) {
344             break;
345         }
346     }
347
348     std::reverse(path.begin(), path.end());
349
350     if (path.empty() || path.front() != s) {
351         path.clear();
352     }
353
354     return path;
355 }
356
357 //TODO: обосновать очередь
358 // 1) обычная очередь работает менее эффективно, т.к. мы можем сделать много лишних циклов, извлекая вершины из очереди
359 // ища оптимальную (в обычной очереди вершины расположены в порядке добавления в очередь, а не отсортированы по весам)
360 // - заменили на очередь с приоритетами (как написано на слайде)
361 //
362 // TODO:
363 // 2) кол-во итераций всегда выводится
364 // - теперь выводится всегда
365 //
366 // TODO:
367 // 3) при перегенерации графа вводить число вершин
368 // - поправка
369 ShortestPathResult GeneratorGraph::dijkstraNegative(
370     int s,
371     int t,
372     const Matrix& adjacency,
373     const Matrix& weight
374 ) const {
375     const double INF = std::numeric_limits<double>::infinity();
376
377     ShortestPathResult result;
378
379     // T[v] - длина кратчайшего пути от s к v
380     // H[v] - вершина, предшествующая v по кратчайшему пути
381
382     std::vector<double> T(m_vertexCount, INF);
383     std::vector<int> H(m_vertexCount, -1);
384
385     std::vector<int> relaxCount(m_vertexCount, 0);
386
387     // { T[v], v }
388     using QueueItem = std::pair<double, int>;
389
390     // очередь с приоритетами
391     // сначала извлекается вершина с минимальным T[v]
392     std::priority_queue<
393         QueueItem,
394         std::vector<QueueItem>,
395         std::greater<QueueItem>
396     > Q;
397
398     int iterations = 0;
399
400     T[s] = 0;
401     H[s] = -1;
402

```

```

403     Q.push({0, s});
404
405     while (!Q.empty()) {
406         double currentDistance = Q.top().first;
407         int v = Q.top().second;
408
409         Q.pop();
410
411         // пропускаем устаревшую запись
412         if (currentDistance != T[v]) {
413             continue;
414         }
415
416         for (int u = 0; u < m_vertexCount; ++u) {
417             ++iterations;
418
419             if (adjacency(v, u) == 0) {
420                 continue;
421             }
422
423             // по нер-ву треугольника
424             if (T[v] != INF && T[u] > T[v] + weight(v, u)) {
425                 T[u] = T[v] + weight(v, u);
426                 H[u] = v;
427
428                 Q.push({T[u], u});
429
430                 ++relaxCount[u];
431
432                 // выходим, если пошли по циклу
433                 if (relaxCount[u] >= m_vertexCount) {
434                     result.hasNegativeCycle = true;
435                     break;
436                 }
437             }
438         }
439
440         if (result.hasNegativeCycle) {
441             break;
442         }
443     }
444
445     result.T = T;
446     result.H = H;
447
448     result.iterations = iterations;
449
450     result.hasPath = T[t] != INF;
451     result.distance = T[t];
452
453     if (result.hasPath && !result.hasNegativeCycle) {
454         result.path = m_restorePath(s, t, H);
455     }
456
457     return result;
458 }
459
460 ShortestPathResult GeneratorGraph::dijkstraNegative(int s, int t) const {
461     return dijkstraNegative(
462         s,
463         t,
464         m_adjacencyMatrix,
465         m_weightMatrix
466     );
467 }
468
469 ShortestPathResult GeneratorGraph::dijkstraNegative(int s, int t, const Matrix& adjacency) const {
470     return dijkstraNegative(
471         s,
472         t,
473         adjacency,
474         adjacency
475     );
476 }
477
478 // lab3
479
480 void GeneratorGraph::generateCapacityAndCostMatrices() {
481     m_capacityMatrix = Matrix(m_vertexCount, m_vertexCount, 0);
482     m_costMatrix = Matrix(m_vertexCount, m_vertexCount, 0);
483
484     for (int i = 0; i < m_vertexCount; ++i) {
485         for (int j = 0; j < m_vertexCount; ++j) {
486             if (m_adjacencyMatrix(i, j) != 0) {
487                 double capacityRow = m_sampleWeibull(constants::capacity);
488                 double costRow = m_sampleWeibull(constants::cost);
489
490                 int capacity = std::max(1, static_cast<int>(std::round(capacityRow * 10)));
491                 int cost = std::max(1, static_cast<int>(std::round(costRow * 10)));
492
493                 m_capacityMatrix(i, j) = capacity;
494                 m_costMatrix(i, j) = cost;
495             }
496         }
497     }
498
499     isMatrixInit.capacity = true;
500     isMatrixInit.cost = true;
501 }
502
503 const Matrix& GeneratorGraph::getCapacityMatrix() const {
504     return m_capacityMatrix;
505 }
506
507 const Matrix& GeneratorGraph::getCostMatrix() const {
508     return m_costMatrix;
509 }

```

```

510
511 void GeneratorGraph::printCapacityMatrix() const {
512     std::cout << "Матрица пропускных способностей:\n";
513     m_capacityMatrix.print();
514 }
515
516 void GeneratorGraph::printCostMatrix() const {
517     std::cout << "Матрица стоимостей:\n";
518     m_costMatrix.print();
519 }
520
521 bool GeneratorGraph::isCapacityMatrixGenerated() const {
522     return isMatrixInit.capacity;
523 }
524
525 bool GeneratorGraph::isCostMatrixGenerated() const {
526     return isMatrixInit.cost;
527 }
528
529 // алгоритм Форда-Фалкерсона
530 // реализован через расстановку пометок
531 MaxFlowResult GeneratorGraph::fordFulkerson(int source, int sink) const {
532     const int INF = std::numeric_limits<int>::max();
533
534     MaxFlowResult result(m_vertexCount);
535
536     if (!isMatrixInit.capacity) {
537         return result;
538     }
539
540     if (source < 0 || source >= m_vertexCount ||
541         sink < 0 || sink >= m_vertexCount ||
542         source == sink) {
543         return result;
544     }
545
546     struct Mark {
547         char sign = '+';
548         int parent = -1;
549         int delta = 0;
550     };
551
552     while (true) {
553         // S[v] - вершина получила пометку
554         // N[v] - вершина уже обработана
555         // P[v] - пометка вершины: знак, предок, величина изменения потока
556         std::vector<bool> S(m_vertexCount, false);
557         std::vector<bool> N(m_vertexCount, false);
558         std::vector<Mark> P(m_vertexCount);
559
560         S[source] = true;
561         P[source] = {'+', -1, INF};
562
563         bool expanded = true;
564
565         // расставляем пометки, пока множество S расширяется
566         while (expanded && !S[sink]) {
567             expanded = false;
568
569             for (int v = 0; v < m_vertexCount; ++v) {
570                 ++result.iterations;
571
572                 if (!S[v] || N[v]) {
573                     continue;
574                 }
575
576                 // прямые дуги
577                 for (int u = 0; u < m_vertexCount; ++u) {
578                     ++result.iterations;
579
580                     if (S[u]) {
581                         continue;
582                     }
583
584                     if (m_capacityMatrix(v, u) <= 0) {
585                         continue;
586                     }
587
588                     int available =
589                         static_cast<int>(m_capacityMatrix(v, u) - result.flowMatrix(v, u));
590
591                     if (available > 0) {
592                         S[u] = true;
593                         P[u] = {
594                             '+',
595                             v,
596                             std::min(P[v].delta, available)
597                         };
598
599                         expanded = true;
600
601                         if (u == sink) {
602                             break;
603                         }
604                     }
605                 }
606
607                 if (S[sink]) {
608                     break;
609                 }
610
611                 // обратные дуги
612                 for (int u = 0; u < m_vertexCount; ++u) {
613                     ++result.iterations;
614
615                     if (S[u]) {
616                         continue;

```



```

617     }
618
619     if (result.flowMatrix(u, v) > 0) {
620         int available =
621             static_cast<int>(result.flowMatrix(u, v));
622
623         S[u] = true;
624         P[u] = {
625             '-',
626             v,
627             std::min(P[v].delta, available)
628         };
629
630         expanded = true;
631
632         if (u == sink) {
633             break;
634         }
635     }
636 }
637
638 N[v] = true;
639
640 if (S[sink]) {
641     break;
642 }
643 }
644 }
645
646 // если сток не получил пометку, увеличивающих цепей больше нет
647 if (!S[sink]) {
648     break;
649 }
650
651 int delta = P[sink].delta;
652 int current = sink;
653
654 // изменяем поток по найденной увеличивающей цепи
655 while (current != source) {
656     Mark mark = P[current];
657     int previous = mark.parent;
658
659     if (mark.sign == '+') {
660         result.flowMatrix(previous, current) += delta;
661     } else {
662         result.flowMatrix(current, previous) -= delta;
663     }
664
665     current = previous;
666 }
667
668 result.maxFlow += delta;
669 }
670
671 return result;
672 }
673
674 // поток минимальной стоимости
675 // заданная величина потока передаётся параметром requiredFlow
676 // на каждой итерации ищем кратчайший путь в остаточной сети
677 MinCostFlowResult GeneratorGraph::minCostFlow(int source,
678                                               int sink,
679                                               int requiredFlow) const {
680     MinCostFlowResult result(m_vertexCount);
681
682     result.requiredFlow = requiredFlow;
683
684     if (!isMatrixInit.capacity || !isMatrixInit.cost) {
685         return result;
686     }
687
688     if (source < 0 || source >= m_vertexCount ||
689         sink < 0 || sink >= m_vertexCount ||
690         source == sink ||
691         requiredFlow <= 0) {
692         return result;
693     }
694
695     Matrix residualCapacity = m_capacityMatrix;
696     Matrix residualCost(m_vertexCount, m_vertexCount, 0);
697
698     for (int i = 0; i < m_vertexCount; ++i) {
699         for (int j = 0; j < m_vertexCount; ++j) {
700             if (m_capacityMatrix(i, j) > 0) {
701                 residualCost(i, j) = m_costMatrix(i, j);
702                 residualCost(j, i) = -m_costMatrix(i, j);
703             }
704         }
705     }
706
707     while (result.achievedFlow < requiredFlow) {
708         Matrix residualAdjacency(m_vertexCount, m_vertexCount, 0);
709
710         for (int i = 0; i < m_vertexCount; ++i) {
711             for (int j = 0; j < m_vertexCount; ++j) {
712                 if (residualCapacity(i, j) > 0) {
713                     residualAdjacency(i, j) = 1;
714                 }
715             }
716         }
717
718         auto pathResult = dijkstraNegative(
719             source,
720             sink,
721             residualAdjacency,
722             residualCost
723         );

```

```

724
725     result.iterations += pathResult.iterations;
726
727     if (!pathResult.hasPath || pathResult.hasNegativeCycle) {
728         break;
729     }
730
731     int addFlow = requiredFlow - result.achievedFlow;
732
733     for (size_t i = 1; i < pathResult.path.size(); ++i) {
734         int from = pathResult.path[i - 1];
735         int to = pathResult.path[i];
736
737         addFlow = std::min(
738             addFlow,
739             static_cast<int>(residualCapacity(from, to))
740         );
741     }
742
743     if (addFlow <= 0) {
744         break;
745     }
746
747     for (size_t i = 1; i < pathResult.path.size(); ++i) {
748         int from = pathResult.path[i - 1];
749         int to = pathResult.path[i];
750
751         residualCapacity(from, to) -= addFlow;
752         residualCapacity(to, from) += addFlow;
753
754         if (m_capacityMatrix(from, to) > 0) {
755             // идем по исходной дуге
756             result.flowMatrix(from, to) += addFlow;
757             result.totalCost += addFlow * static_cast<int>(m_costMatrix(from, to));
758         } else {
759             // идем по обратной дуге остаточной сети
760             // значит уменьшаем поток по исходной дуге to -> from
761             result.flowMatrix(to, from) -= addFlow;
762             result.totalCost -= addFlow * static_cast<int>(m_costMatrix(to, from));
763         }
764     }
765
766     result.achievedFlow += addFlow;
767 }
768
769 result.success = result.achievedFlow == requiredFlow;
770
771 for (int i = 0; i < m_vertexCount; ++i) {
772     for (int j = 0; j < m_vertexCount; ++j) {
773         result.costFlowMatrix(i, j) =
774             m_costMatrix(i, j) * result.flowMatrix(i, j);
775     }
776 }
777
778 return result;
779 }
780
781 // lab4
782 SpanningTreesResult GeneratorGraph::countSpanningTreesKirchhoff() const {
783     SpanningTreesResult result(m_vertexCount);
784     const Matrix& adjacency = m_undirectedAdjacencyMatrix;
785
786     for (int i = 0; i < m_vertexCount; ++i) {
787         int degree = 0;
788
789         for (int j = 0; j < m_vertexCount; ++j) {
790             if (i == j) {
791                 continue;
792             }
793
794             if (adjacency(i, j) != 0) {
795                 ++degree;
796                 result.kirchhoffMatrix(i, j) = -1;
797             }
798         }
799
800         result.kirchhoffMatrix(i, i) = degree;
801     }
802
803     if (m_vertexCount <= 1) {
804         result.count = 1;
805         return result;
806     }
807
808     for (int i = 1; i < m_vertexCount; ++i) {
809         for (int j = 1; j < m_vertexCount; ++j) {
810             result.cofactorMatrix(i - 1, j - 1) =
811                 result.kirchhoffMatrix(i, j);
812         }
813     }
814
815     result.cofactorDeterminant = result.cofactorMatrix.determinant();
816     result.count = std::llround(result.cofactorDeterminant);
817
818     return result;
819 }
820
821 KruskalResult GeneratorGraph::m_kruskalAlgorithm() const {
822     KruskalResult result(m_vertexCount);
823     std::vector<WeightedEdge> edges;
824
825     for (int i = 0; i < m_vertexCount; ++i) {
826         result.treeMatrix(i, i) = 0;
827     }
828
829     // берем все ребра графа
830     for (int i = 0; i < m_vertexCount; ++i) {

```

```

831     for (int j = i + 1; j < m_vertexCount; ++j) {
832         if (m_undirectedAdjacencyMatrix(i, j) != 0 &&
833             !std::isinf(m_undirectedWeightMatrix(i, j))) {
834             edges.push_back({
835                 i,
836                 j,
837                 m_undirectedWeightMatrix(i, j)
838             });
839         }
840     }
841 }
842
843 // сортируем рёбра по возрастанию веса
844 std::sort(
845     edges.begin(),
846     edges.end(),
847     [](const WeightedEdge& lhs, const WeightedEdge& rhs) {
848         if (lhs.weight != rhs.weight) {
849             return lhs.weight < rhs.weight;
850         }
851
852         if (lhs.from != rhs.from) {
853             return lhs.from < rhs.from;
854         }
855
856         return lhs.to < rhs.to;
857     }
858 );
859
860 std::vector<int> parent(m_vertexCount);
861 std::vector<int> rank(m_vertexCount, 0);
862
863 for (int i = 0; i < m_vertexCount; ++i) {
864     parent[i] = i;
865 }
866
867 std::function<int(int)> findSet = [&](int vertex) {
868     if (parent[vertex] != vertex) {
869         parent[vertex] = findSet(parent[vertex]);
870     }
871
872     return parent[vertex];
873 };
874
875 auto unionSets = [&](int first, int second) {
876     first = findSet(first);
877     second = findSet(second);
878
879     if (first == second) {
880         return false;
881     }
882
883     if (rank[first] < rank[second]) {
884         std::swap(first, second);
885     }
886
887     parent[second] = first;
888
889     if (rank[first] == rank[second]) {
890         ++rank[first];
891     }
892
893     // если ребро соединяет вершины из разных множеств, его можно взять
894     return true;
895 };
896
897 // идем по ребрам от самого дешевого к самому дорогому
898 // изначально каждая вершина в своем множестве [сверху как раз сделаны ф. поиска и пересечения «множеств»]
899 for (const auto& edge : edges) {
900     if (unionSets(edge.from, edge.to)) {
901         // добавляем ребро в осто́в, если оно не образует цикла
902         result.edges.push_back(edge);
903         result.treeMatrix(edge.from, edge.to) = edge.weight;
904         result.treeMatrix(edge.to, edge.from) = edge.weight;
905         result.totalWeight += edge.weight;
906
907         // останавливаемся, когда выбрали n - 1 ребро
908         if (static_cast<int>(result.edges.size()) == m_vertexCount - 1) {
909             break;
910         }
911     }
912 }
913
914 result.success =
915     static_cast<int>(result.edges.size()) == m_vertexCount - 1;
916
917 return result;
918 }
919
920 KruskalMinimumSpanningTreeResult GeneratorGraph::kruskalMinimumSpanningTree() const {
921     KruskalMinimumSpanningTreeResult result(std::move(m_kruskalAlgorithm()), m_vertexCount);
922
923     for (int i = 0; i < m_vertexCount; ++i) {
924         result.decodedTreeMatrix(i, i) = 0;
925     }
926
927     if (!result.kruskal.success) {
928         return result;
929     }
930
931     result.pruferCode = m_encodePruferCode(result.kruskal.edges);
932     result.decodedTreeMatrix = m_decodePruferCode(result.pruferCode);
933     result.pruferRoundTripSuccess =
934         matricesEqual(result.kruskal.treeMatrix, result.decodedTreeMatrix);
935
936     return result;
937 }

```

```

938
939 std::vector<PruferCodeItem> GeneratorGraph::m_encodePruferCode(
940     const std::vector<WeightedEdge>& treeEdges
941 ) const {
942     // кодируем полученный остов кодом Прюфера с сохранением весов рёбер
943     std::vector<PruferCodeItem> code;
944     std::vector<std::set<std::pair<int, double>>> adjacency(m_vertexCount);
945
946     for (const auto& edge : treeEdges) {
947         adjacency[edge.from].insert({edge.to, edge.weight});
948         adjacency[edge.to].insert({edge.from, edge.weight});
949     }
950
951     for (int step = 0; step < m_vertexCount - 1; ++step) {
952         int leaf = -1;
953
954         // на каждом шаге берём лист с минимальным номером
955         for (int vertex = 0; vertex < m_vertexCount; ++vertex) {
956             if (adjacency[vertex].size() == 1) {
957                 leaf = vertex;
958                 break;
959             }
960         }
961
962         if (leaf == -1) {
963             break;
964         }
965
966         const auto [neighbor, weight] = *adjacency[leaf].begin();
967         // в код записываем соседа листа, а рядом сохраняем вес удаляемого ребра
968         code.push_back({neighbor, weight});
969
970         // удаляем лист из текущего дерева
971         adjacency[neighbor].erase({leaf, weight});
972         adjacency[leaf].clear();
973     }
974
975     return code;
976 }
977
978 Matrix GeneratorGraph::m_decodePruferCode(
979     const std::vector<PruferCodeItem>& code
980 ) const {
981     // декодируем код Прюфера обратно в матрицу весов остова
982     Matrix decodedTreeMatrix(
983         m_vertexCount,
984         m_vertexCount,
985         std::numeric_limits<double>::infinity()
986     );
987     std::vector<int> unusedVertices(m_vertexCount);
988
989     for (int i = 0; i < m_vertexCount; ++i) {
990         decodedTreeMatrix(i, i) = 0;
991         unusedVertices[i] = i;
992     }
993
994     const int steps = std::min(
995         m_vertexCount - 1,
996         static_cast<int>(code.size())
997     );
998
999     for (int i = 0; i < steps; ++i) {
1000         std::set<int> remainingCode;
1001
1002         for (int j = i; j < static_cast<int>(code.size()); ++j) {
1003             remainingCode.insert(code[j].vertex);
1004         }
1005
1006         // выбираем минимальную вершину, которой нет в оставшемся коде
1007         auto leafIt = std::find_if(
1008             unusedVertices.begin(),
1009             unusedVertices.end(),
1010             [&](int vertex) {
1011                 return remainingCode.find(vertex) == remainingCode.end();
1012             }
1013         );
1014
1015         if (leafIt == unusedVertices.end()) {
1016             break;
1017         }
1018
1019         const int leaf = *leafIt;
1020         const int neighbor = code[i].vertex;
1021         const double weight = code[i].weight;
1022
1023         // восстанавливаем ребро с тем же весом, который был сохранён при кодировании
1024         decodedTreeMatrix(leaf, neighbor) = weight;
1025         decodedTreeMatrix(neighbor, leaf) = weight;
1026
1027         unusedVertices.erase(leafIt);
1028     }
1029
1030     return decodedTreeMatrix;
1031 }
1032
1033 EdmondsBlossomResult
1034 GeneratorGraph::edmondsBlossomInOriginalGraph() const {
1035     const Matrix& weights = isMatrixInit.weight
1036         ? m_undirectedWeightMatrix
1037         : m_undirectedAdjacencyMatrix;
1038
1039     return m_edmondsBlossom(
1040         m_undirectedAdjacencyMatrix,
1041         weights,
1042         isMatrixInit.weight
1043     );
1044 }

```

```

1045
1046 EdmondsBlossomResult
1047 GeneratorGraph::edmondsBlossomInMinimumSpanningTree() const {
1048     EdmondsBlossomResult result(m_vertexCount);
1049     auto kruskalMinimumSpanningTreeResult = kruskalMinimumSpanningTree();
1050
1051     if (!kruskalMinimumSpanningTreeResult.kruskal.success) {
1052         result.success = false;
1053         return result;
1054     }
1055
1056     Matrix adjacency(m_vertexCount, m_vertexCount, 0);
1057
1058     for (const auto& edge : kruskalMinimumSpanningTreeResult.kruskal.edges) {
1059         adjacency(edge.from, edge.to) = 1;
1060         adjacency(edge.to, edge.from) = 1;
1061     }
1062
1063     return m_edmondsBlossom(
1064         adjacency,
1065         kruskalMinimumSpanningTreeResult.kruskal.treeMatrix,
1066         true
1067     );
1068 }
1069
1070 EdmondsBlossomResult GeneratorGraph::m_edmondsBlossom(
1071     const Matrix& adjacency,
1072     const Matrix& weights,
1073     bool hasWeights
1074 ) const {
1075     EdmondsBlossomResult result(m_vertexCount);
1076     result.hasWeights = hasWeights;
1077
1078     if (hasWeights) {
1079         result.edgeMatrix = Matrix(
1080             m_vertexCount,
1081             m_vertexCount,
1082             std::numeric_limits<double>::infinity()
1083         );
1084
1085         for (int i = 0; i < m_vertexCount; ++i) {
1086             result.edgeMatrix(i, i) = 0;
1087         }
1088     }
1089
1090     const auto graph = m_matrixToAdjacencyList<int>(
1091         adjacency,
1092         [](int, int to) {
1093             return to;
1094         }
1095     );
1096
1097     // хотим выбрать как можно больше рёбер, чтобы они не касались друг друга
1098     // => ищем увел. путь, путь от одной вершины до другой, где рёбра идут по очереди
1099
1100     // BEGIN
1101     // а нечётные циклы временно сжимаем.
1102     // match[v]: с какой вершиной соединена вершина v
1103     std::vector<int> match(m_vertexCount, -1);
1104     // parent[v]: предыдущая вершина в дереве поиска увеличивающегося пути
1105     std::vector<int> parent(m_vertexCount);
1106     std::vector<int> base(m_vertexCount);
1107     // помечаем, что вершина уже добавлена в очередь
1108     std::vector<bool> used(m_vertexCount);
1109     // помечаем цикл в графе
1110     std::vector<bool> blossom(m_vertexCount);
1111     std::queue<int> queue;
1112
1113     // ищем общую базу нечётного цикла, чтобы сжать к одной вершине
1114     auto findLca = [&](int first, int second) {
1115         std::vector<bool> usedPath(m_vertexCount, false);
1116
1117         while (true) {
1118             first = base[first];
1119             usedPath[first] = true;
1120
1121             if (match[first] == -1) {
1122                 break;
1123             }
1124
1125             first = parent[match[first]];
1126         }
1127
1128         while (true) {
1129             second = base[second];
1130
1131             if (usedPath[second]) {
1132                 return second;
1133             }
1134
1135             second = parent[match[second]];
1136         }
1137     };
1138
1139     auto markPath = [&](int vertex,
1140         int lca,
1141         int child,
1142         auto&& markPathRef) -> void {
1143         // помечаем путь от вершины до общей базы
1144         while (base[vertex] != lca) {
1145             blossom[base[vertex]] = true;
1146             blossom[base[match[vertex]]] = true;
1147             parent[vertex] = child;
1148             child = match[vertex];
1149             vertex = parent[match[vertex]];
1150         }
1151     };

```

```

1152
1153 auto findPath = [&](int root) {
1154     // T <- ∅: очищаем BFS-дерево поиска увеличивающего пути
1155     std::fill(used.begin(), used.end(), false);
1156     std::fill(parent.begin(), parent.end(), -1);
1157
1158     for (int i = 0; i < m_vertexCount; ++i) {
1159         base[i] = i;
1160     }
1161
1162     while (!queue.empty()) {
1163         queue.pop();
1164     }
1165
1166     // pick r in F; queue.push(r); T.add(r)
1167     queue.push(root);
1168     used[root] = true;
1169
1170     // WHILE queue != ∅
1171     while (!queue.empty()) {
1172         // v <- queue.pop()
1173         int vertex = queue.front();
1174         queue.pop();
1175
1176         // FOR ALL neighbors w of v DO
1177         for (int to : graph[vertex]) {
1178             if (base[vertex] == base[to] ||
1179                 match[vertex] == to) {
1180                 continue;
1181             }
1182
1183             if (to == root ||
1184                 (match[to] != -1 && parent[match[to]] != -1)) {
1185                 // ELSE IF w in T AND odd-length cycle detected THEN
1186                 int lca = findLca(vertex, to);
1187                 std::fill(blossom.begin(), blossom.end(), false);
1188
1189                 // contract cycle: найден нечётный цикл, сжимаем blossom к общей базе
1190                 markPath(vertex, lca, to, markPath);
1191                 markPath(to, lca, vertex, markPath);
1192
1193                 for (int i = 0; i < m_vertexCount; ++i) {
1194                     if (blossom[base[i]]) {
1195                         base[i] = lca;
1196
1197                         if (!used[i]) {
1198                             used[i] = true;
1199                             queue.push(i);
1200                         }
1201                     }
1202                 }
1203             } else if (parent[to] == -1) {
1204                 // IF w not in T AND w matched THEN T.add(w), T.add(mate(w))
1205                 parent[to] = vertex;
1206
1207                 if (match[to] == -1) {
1208                     // ELSE IF w in F THEN свободная вершина завершает увеличивающий путь
1209                     return to;
1210                 }
1211
1212                 // queue.push(mate(w))
1213                 int next = match[to];
1214                 used[next] = true;
1215                 queue.push(next);
1216             }
1217         }
1218     }
1219
1220     return -1;
1221 };
1222
1223 auto augment = [&](int vertex) {
1224     // expand all contracted nodes; reconstruct augmenting path;
1225     // invert augmenting path
1226     while (vertex != -1) {
1227         int previous = parent[vertex];
1228         int next = (previous == -1) ? -1 : match[previous];
1229
1230         match[vertex] = previous;
1231
1232         if (previous != -1) {
1233             match[previous] = vertex;
1234         }
1235
1236         vertex = next;
1237     }
1238 };
1239
1240 // WHILE F != ∅ DO: пробуем стартовать из каждой свободной вершины
1241 for (int vertex = 0; vertex < m_vertexCount; ++vertex) {
1242     if (match[vertex] == -1) {
1243         int endpoint = findPath(vertex);
1244
1245         if (endpoint != -1) {
1246             augment(endpoint);
1247         }
1248     }
1249 }
1250 // END
1251
1252 for (int from = 0; from < m_vertexCount; ++from) {
1253     int to = match[from];
1254
1255     if (to == -1 || from > to) {
1256         continue;
1257     }
1258

```

```

1259         double value = hasWeights ? weights(from, to) : 1;
1260
1261         result.edges.push_back({from, to, value});
1262         result.edgeMatrix(from, to) = value;
1263         result.edgeMatrix(to, from) = value;
1264     }
1265
1266     return result;
1267 }
1268
1269 // lab5
1270 // Алгоритм приведения графа к эйлерову виду:
1271 // 1. Найти все вершины нечётной степени.
1272 // 2. Пока существуют нечётные вершины:
1273 //     * выбрать две вершины нечётной степени (u) и (v);
1274 //     * если ребро ((u,v)) существует, удалить его;
1275 //     * иначе добавить ребро ((u,v)).
1276 // Патч для сохранения связности: перебираем пары нечётных вершин и перед
1277 // удалением ребра проверяем наличие обходного пути. Мост удаляется только
1278 // если пользователь явно разрешил получить несвязный результат.
1279 // После каждой такой операции степени вершин (u) и (v) меняют чётность, поэтому число вершин нечётной степени уменьшается на 2. Поскольку
1280 ↪ количество нечётных вершин в графе всегда чётно, процесс завершится за конечное число шагов.
1281
1282 CheckIfEulerianGraphResult GeneratorGraph::checkIfEulerianGraph(
1283     bool allowDisconnectedResult
1284 ) const {
1285     bool isEulerian = true;
1286     bool transformationCompleted = true;
1287     bool resultIsConnected = true;
1288     Matrix adjacency = getUndirectedAdjacencyMatrix();
1289     std::vector<Edge> addedEdges;
1290     std::vector<Edge> removedEdges;
1291     auto getVertexDegree = [&](size_t i) {
1292         int degree = 0;
1293         for (size_t j = 0; j < adjacency.cols(); ++j) {
1294             if (i == j) continue;
1295             if (adjacency(i, j) != 0) ++degree;
1296         }
1297         return degree;
1298     };
1299
1300     std::vector<int> verticesWithOddDegrees;
1301
1302     while (true) {
1303         verticesWithOddDegrees.clear();
1304
1305         for (size_t i = 0; i < adjacency.rows(); ++i) {
1306             if (getVertexDegree(i) % 2 != 0) {
1307                 isEulerian = false;
1308                 verticesWithOddDegrees.push_back(i);
1309             }
1310         }
1311
1312         if (verticesWithOddDegrees.empty()) {
1313             break;
1314         }
1315
1316         bool graphChanged = false;
1317
1318         for (size_t i = 0; i < verticesWithOddDegrees.size() && !graphChanged; ++i) {
1319             for (size_t j = i + 1; j < verticesWithOddDegrees.size(); ++j) {
1320                 const auto u = verticesWithOddDegrees[i];
1321                 const auto v = verticesWithOddDegrees[j];
1322
1323                 if (adjacency(u, v) == 0) {
1324                     adjacency(u, v) = 1;
1325                     adjacency(v, u) = 1;
1326                     addedEdges.emplace_back(u, v);
1327                     graphChanged = true;
1328                     break;
1329                 }
1330             }
1331
1332             Matrix temp = adjacency;
1333             temp(u, v) = 0;
1334             temp(v, u) = 0;
1335
1336             const auto resultPath = dijkstraNegative(u, v, temp);
1337
1338             if (resultPath.hasPath || allowDisconnectedResult) {
1339                 adjacency(u, v) = 0;
1340                 adjacency(v, u) = 0;
1341                 removedEdges.emplace_back(u, v);
1342
1343                 if (!resultPath.hasPath) {
1344                     resultIsConnected = false;
1345                 }
1346
1347                 graphChanged = true;
1348                 break;
1349             }
1350         }
1351     }
1352
1353     if (!graphChanged) {
1354         transformationCompleted = false;
1355         break;
1356     }
1357 }
1358
1359 FleuryResult eulerianCycle;
1360
1361 if (transformationCompleted && resultIsConnected) {
1362     eulerianCycle = m_fleuryAlgorithm(adjacency);
1363 }
1364

```

```

1365     return CheckIfEulerianGraphResult(
1366         isEulerian,
1367         transformationCompleted,
1368         std::move(addedEdges),
1369         std::move(removedEdges),
1370         std::move(eulerianCycle),
1371         std::move(adjacency)
1372     );
1373 }
1374
1375 FleuryResult GeneratorGraph::m_fleuryAlgorithm(const Matrix& adjacency) const {
1376     auto graph = m_matrixToAdjacencyList<int>(
1377         adjacency,
1378         [](int, int to) {
1379             return to;
1380         }
1381     );
1382
1383     FleuryResult path;
1384     std::stack<int> stack;
1385     stack.push(0);
1386
1387     while (!stack.empty()) {
1388         const int node = stack.top();
1389
1390         if (graph[node].empty()) {
1391             path.emplace_back(node);
1392             stack.pop();
1393         } else {
1394             const int next = graph[node].front();
1395             graph[node].erase(graph[node].begin());
1396
1397             const auto reverseEdge = std::find(
1398                 graph[next].begin(),
1399                 graph[next].end(),
1400                 node
1401             );
1402
1403             if (reverseEdge == graph[next].end()) {
1404                 throw std::logic_error(
1405                     "Обратное ребро отсутствует в списке смежности"
1406                 );
1407             }
1408
1409             graph[next].erase(reverseEdge);
1410             stack.push(next);
1411         }
1412     }
1413
1414     std::reverse(path.begin(), path.end());
1415     return path;
1416 }
1417
1418 FundamentalCutsResult GeneratorGraph::buildFundamentalCutSystem() const {
1419     FundamentalCutsResult result(m_vertexCount);
1420
1421     if (!isMatrixInit.adjacency || !isMatrixInit.weight) {
1422         return result;
1423     }
1424
1425     result.spanningTree = m_kruskalAlgorithm();
1426
1427     if (!result.spanningTree.success) {
1428         return result;
1429     }
1430
1431     AdjacencyList tree(m_vertexCount);
1432
1433     for (const auto& edge : result.spanningTree.edges) {
1434         tree[edge.from].push_back(edge.to);
1435         tree[edge.to].push_back(edge.from);
1436     }
1437
1438     for (const auto& removedEdge : result.spanningTree.edges) {
1439         std::vector<bool> firstComponent(m_vertexCount, false);
1440         std::vector<int> stack = {removedEdge.from};
1441         firstComponent[removedEdge.from] = true;
1442
1443         // Удаление ребра остова делит дерево на две компоненты.
1444         while (!stack.empty()) {
1445             const int vertex = stack.back();
1446             stack.pop_back();
1447
1448             for (int to : tree[vertex]) {
1449                 const bool isRemovedEdge =
1450                     (vertex == removedEdge.from && to == removedEdge.to) ||
1451                     (vertex == removedEdge.to && to == removedEdge.from);
1452
1453                 if (!isRemovedEdge && !firstComponent[to]) {
1454                     firstComponent[to] = true;
1455                     stack.push_back(to);
1456                 }
1457             }
1458         }
1459     }
1460
1461     GraphCut cut;
1462
1463     // В разрез входят все рёбра исходного графа между компонентами.
1464     for (int from = 0; from < m_vertexCount; ++from) {
1465         for (int to = from + 1; to < m_vertexCount; ++to) {
1466             if (m_undirectedAdjacencyMatrix(from, to) != 0 &&
1467                 firstComponent[from] != firstComponent[to]) {
1468                 cut.emplace(from, to);
1469             }
1470         }
1471     }

```



```

1472         result.fundamentalCuts.push_back({
1473             {removedEdge.from, removedEdge.to},
1474             std::move(cut)
1475         });
1476     };
1477 }
1478
1479 result.success =
1480     static_cast<int>(result.fundamentalCuts.size()) == m_vertexCount - 1;
1481 return result;
1482 }
1483
1484 GraphCut GeneratorGraph::symmetricDifferenceOfFundamentalCuts(
1485     const FundamentalCutsResult& system,
1486     const std::vector<int>& selectedCutIndices
1487 ) const {
1488     GraphCut result;
1489
1490     for (int index : selectedCutIndices) {
1491         if (index < 0 || index >= static_cast<int>(system.fundamentalCuts.size())) {
1492             throw std::out_of_range("Номер фундаментального разреза вне диапазона");
1493         }
1494
1495         result = m_symmetricDifference(
1496             result,
1497             system.fundamentalCuts[index].cut
1498         );
1499     }
1500
1501     return result;
1502 }
1503
1504 // этот метод показывает, что используем метод из лекции для вычисления симм. разности
1505 // можно найти на слайде 18 «Подпространства циклов и юциклов»
1506 GraphCut GeneratorGraph::m_symmetricDifference(
1507     const GraphCut& first,
1508     const GraphCut& second
1509 ) const {
1510     GraphCut result;
1511     GraphCut edgeCoordinates = first;
1512     edgeCoordinates.insert(second.begin(), second.end());
1513
1514     // разрез рассматриваем как вектор векторного пространства, натянутого на множество рёбер
1515     // коэффициент равен 1, если ребро входит в разрез, иначе равен 0
1516     for (const auto& edge : edgeCoordinates) {
1517         const int firstCoefficient = first.count(edge) == 0 ? 0 : 1;
1518         const int secondCoefficient = second.count(edge) == 0 ? 0 : 1;
1519         const int resultCoefficient =
1520             (firstCoefficient + secondCoefficient) % 2;
1521
1522         if (resultCoefficient == 1) {
1523             result.insert(edge);
1524         }
1525     }
1526
1527     return result;
1528 }

```

Листинг 29: Объявление интерфейса командной строки

```

1  #pragma once
2
3  #include "../src/generator_graph.h"
4  #include <iostream>
5  #include <iomanip>
6  #include <limits>
7  #include <string>
8
9  class CLI {
10 public:
11     explicit CLI();
12     void run();
13
14 private:
15     int m_vertexCount;
16     std::unique_ptr<GeneratorGraph> m_graph;
17
18     void m_generateGraph();
19     void m_printHeader(const std::string& title) const;
20     void m_printInfo(const std::string& message) const;
21     void m_printWarning(const std::string& message) const;
22     void m_printError(const std::string& message) const;
23     int m_readInt(const std::string& prompt,
24                 int min,
25                 int max) const;
26     void m_printVector(const std::vector<int>& v) const;
27     void m_printMenu() const;
28
29     void m_menuGenerate();
30     void m_menuEccentricities();
31     void m_menuWeights();
32     void m_menuShimbell() const;
33     void m_menuRoutes() const;
34     void m_menuPrintMatrices() const;
35
36     // lab2
37     void m_menuFindArticulationPoints() const;
38     void m_runDijkstraNegative();
39
40     // lab3
41     void m_menuCapacityAndCostMatrices();
42     void m_menuMaxFlow();

```

```

43     void m_menuMinCostFlow();
44
45     // lab4
46     void m_menuSpanningTreesCount();
47     void m_menuKruskalMinimumSpanningTree();
48     void m_menuEdmondsBlossom();
49
50     // lab5
51     void m_menuCheckEulerianGraph() const;
52     void m_menuFundamentalCuts() const;
53 };

```

Листинг 30: Реализация интерфейса командной строки

```

1  #include "cli.h"
2  #include <csddef>
3
4
5  CLI::CLI() : m_vertexCount(0) {}
6
7  void CLI::m_generateGraph() {
8      m_vertexCount = m_readInt(
9          "Количество вершин (>=2): ",
10         2,
11         100
12     );
13
14     m_graph = std::make_unique<GeneratorGraph>(
15         m_vertexCount
16     );
17
18     m_graph->generate();
19 }
20
21 void CLI::m_printHeader(const std::string& title) const {
22     std::cout << "\n==== " << title << "====\n";
23 }
24
25 void CLI::m_printInfo(const std::string& message) const {
26     std::cout << "[*] " << message << "\n";
27 }
28
29 void CLI::m_printWarning(const std::string& message) const {
30     std::cout << "[!] " << message << "\n";
31 }
32
33 void CLI::m_printError(const std::string& message) const {
34     std::cout << "[x] " << message << "\n";
35 }
36
37 int CLI::m_readInt(const std::string& prompt,
38                  int min,
39                  int max) const {
40     while (true) {
41         std::cout << prompt;
42
43         int value;
44
45         if (std::cin >> value &&
46             value >= min &&
47             value <= max) {
48             return value;
49         }
50
51         m_printError("Некорректный ввод.");
52
53         std::cin.clear();
54
55         std::cin.ignore(
56             std::numeric_limits<std::streamsize>::max(),
57             '\n'
58         );
59     }
60 }
61
62 void CLI::m_printVector(const std::vector<int>& v) const {
63     for (size_t i = 0; i < v.size(); ++i) {
64         std::cout << v[i];
65
66         if (i + 1 < v.size()) {
67             std::cout << ", ";
68         }
69     }
70
71     std::cout << "\n";
72 }
73
74 void CLI::m_menuGenerate() {
75     m_printHeader("Генерация графа");
76
77     m_generateGraph();
78
79     m_printInfo("Граф сгенерирован.");
80
81     m_graph->printAdjacencyMatrix();
82 }
83
84 void CLI::m_menuEccentricities() {
85     m_printHeader("Экцентриситеты, центр, диаметр");
86
87     m_graph->computeEccentricities();
88 }

```

```

89     auto ecc = m_graph->getEccentricities();
90
91     const int cellWidth = 4;
92
93     std::cout << "Вершина: ";
94
95     for (size_t i = 0; i < ecc.size(); ++i) {
96         std::cout << std::setw(cellWidth) << i;
97     }
98
99     std::cout << "\n";
100
101     std::cout << "Эксцентриситет: ";
102
103     for (double e : ecc) {
104         std::cout << std::setw(cellWidth)
105             << static_cast<int>(e);
106     }
107
108     std::cout << "\n";
109
110     std::cout << "Диаметр графа: "
111         << m_graph->getDiameter()
112         << "\n";
113
114     std::cout << "Центральные вершины: ";
115     m_printVector(m_graph->getCenterVertices());
116
117     std::cout << "Диаметральные вершины: ";
118     m_printVector(m_graph->getDiametralVertices());
119 }
120
121 void CLI::m_menuWeights() {
122     m_printHeader("Весовая матрица");
123
124     std::cout << "Режим весов:\n"
125         << " 1 - положительные\n"
126         << " 2 - отрицательные\n"
127         << " 3 - смешанные\n";
128
129     int mode = m_readInt("> ", 1, 3);
130
131     WeightMode wm = WeightMode::Positive;
132
133     switch (mode) {
134     case 1:
135         wm = WeightMode::Positive;
136         break;
137
138     case 2:
139         wm = WeightMode::Negative;
140         break;
141
142     case 3:
143         wm = WeightMode::Mixed;
144         break;
145     }
146
147     m_graph->generateWeightMatrix(wm);
148
149     m_printInfo("Весовая матрица сгенерирована.");
150
151     m_graph->printWeightMatrix();
152 }
153
154 void CLI::m_menuShimbell() const {
155     m_printHeader("Алгоритм Шимбелла");
156
157     if (!m_graph->isMatrixInit.weight) {
158         m_printError("Весовая матрица не инициализирована.");
159         return;
160     }
161
162     int n = m_graph->getVertexCount();
163
164     int steps = m_readInt(
165         "Количество рёбер в пути [0, "
166         + std::to_string(n - 1) + "]: ",
167         0,
168         n - 1
169     );
170
171     std::cout << "Искать:\n"
172         << " 1 - минимальный путь\n"
173         << " 2 - максимальный путь\n";
174
175     int mode = m_readInt("> ", 1, 2);
176
177     Matrix result =
178         m_graph->shimbell(steps, mode == 1);
179
180     result.print();
181 }
182
183 void CLI::m_menuRoutes() const {
184     m_printHeader("Маршруты между вершинами");
185
186     int n = m_graph->getVertexCount();
187
188     int from = m_readInt(
189         "Начальная вершина: ",
190         0,
191         n - 1
192     );
193
194     int to = m_readInt(
195         "Конечная вершина: ",

```

```

196         0,
197         n - 1
198     );
199
200     bool exists = m_graph->hasRoute(from, to);
201
202     std::cout << "Маршрут "
203     << from
204     << " -> "
205     << to
206     << ": "
207     << (exists
208         ? "существует"
209         : "не существует")
210     << "\n";
211
212     if (exists) {
213         std::cout << "Количество маршрутов: "
214         << m_graph->countRoutes(from, to)
215         << "\n";
216     }
217 }
218
219 void CLI::m_menuPrintMatrices() const {
220     m_printHeader("Текущие матрицы");
221
222     m_graph->printAdjacencyMatrix();
223
224     std::cout << "\n";
225
226     m_graph->printWeightMatrix();
227 }
228
229 void CLI::m_menuFindArticulationPoints() const {
230     m_printHeader("Точки сочленения");
231
232     auto result =
233         m_graph->findArticulationPoints();
234
235     if (result.points.empty()) {
236         std::cout << "Точки сочленения не найдены.\n";
237     } else {
238         std::cout << "Точки сочленения: ";
239
240         for (int v : result.points) {
241             std::cout << v << " ";
242         }
243
244         std::cout << "\n";
245     }
246
247     std::cout << "Количество итераций: "
248     << result.iterations
249     << "\n";
250 }
251
252 void CLI::m_runDijkstraNegative() {
253     m_printHeader(
254         "Алгоритм Дейкстры для отрицательных весов"
255     );
256
257     if (!m_graph->isMatrixInit.weight) {
258         m_printError("Весовая матрица не инициализирована.");
259         return;
260     }
261
262     int n = m_graph->getVertexCount();
263
264     int s = m_readInt(
265         "Начальная вершина: ",
266         0,
267         n - 1
268     );
269
270     int t = m_readInt(
271         "Конечная вершина: ",
272         0,
273         n - 1
274     );
275
276     auto result =
277         m_graph->dijkstraNegative(s, t);
278
279     if (!result.hasPath) {
280         std::cout << "Путь не найден.\n";
281
282         std::cout
283             << "Количество итераций: "
284             << result.iterations
285             << "\n";
286
287         return;
288     }
289
290     std::string wayString;
291     std::string weightsString;
292
293     double sum = 0;
294
295     for (size_t i = 0; i < result.path.size(); ++i) {
296         if (i > 0) {
297             wayString += ", ";
298         }
299
300         wayString +=
301             std::to_string(result.path[i]);
302     }

```

```

303
304     for (size_t i = 1; i < result.path.size(); ++i) {
305         int from = result.path[i - 1];
306         int to = result.path[i];
307
308         double weight =
309             m_graph->getWeightMatrix()(from, to);
310
311         if (i > 1) {
312             weightsString += ", ";
313         }
314
315         weightsString +=
316             std::to_string(static_cast<int>(weight));
317
318         sum += weight;
319     }
320
321     std::cout
322         << "Кратчайший маршрут из "
323         << s
324         << " в "
325         << t
326         << " : ("
327         << wayString
328         << ")\n";
329
330     std::cout
331         << "Вес дуг маршрута: ("
332         << weightsString
333         << "); sum: "
334         << sum
335         << "\n";
336
337     std::cout
338         << "Вектор расстояний: ("
339
340     for (int i = 0; i < n; ++i) {
341         if (i > 0) {
342             std::cout << ", ";
343         }
344
345         if (result.T[i] ==
346             std::numeric_limits<double>::infinity()) {
347
348             std::cout << "inf";
349
350         } else {
351
352             std::cout
353                 << static_cast<int>(result.T[i]);
354         }
355     }
356
357     std::cout << ")\n";
358
359     std::cout
360         << "Количество итераций: "
361         << result.iterations
362         << "\n";
363 }
364
365 void CLI::m_menuCapacityAndCostMatrices() {
366     m_printHeader("Матрицы пропускных способностей и стоимостей");
367
368     m_graph->generateCapacityAndCostMatrices();
369
370     m_printInfo("Матрицы сгенерированы.");
371     std::cout << "\n";
372
373     m_graph->printCapacityMatrix();
374
375     std::cout << "\n";
376
377     m_graph->printCostMatrix();
378 }
379
380 void CLI::m_menuMaxFlow() {
381     m_printHeader("Максимальный поток");
382
383     if (!m_graph->isCapacityMatrixGenerated()) {
384         m_printError("Сначала сгенерируйте матрицу пропускных способностей.");
385         return;
386     }
387
388     int vertexCount = m_graph->getVertexCount();
389
390     std::cout << "Источник:\n";
391     int source = m_readInt("> ", 0, vertexCount - 1);
392
393     std::cout << "Сток:\n";
394     int sink = m_readInt("> ", 0, vertexCount - 1);
395
396     if (source == sink) {
397         m_printError("Источник и сток должны различаться.");
398         return;
399     }
400
401     auto result = m_graph->fordFulkerson(source, sink);
402
403     std::cout << "\nМаксимальный поток: "
404         << result.maxFlow
405         << "\n";
406
407     std::cout << "Количество итераций: "
408         << result.iterations
409         << "\n\n";

```

```

410
411     std::cout << "Матрица потоков:\n";
412
413     result.flowMatrix.print();
414 }
415
416 void CLI::m_menuMinCostFlow() {
417     m_printHeader("Поток минимальной стоимости");
418
419     if (!m_graph->isCapacityMatrixGenerated()) {
420         m_printError("Сначала сгенерируйте матрицу пропускных способностей.");
421         return;
422     }
423
424     if (!m_graph->isCostMatrixGenerated()) {
425         m_printError("Сначала сгенерируйте матрицу стоимостей.");
426         return;
427     }
428
429     int vertexCount = m_graph->getVertexCount();
430
431     std::cout << "Источник:\n";
432     int source = m_readInt("> ", 0, vertexCount - 1);
433
434     std::cout << "Сток:\n";
435     int sink = m_readInt("> ", 0, vertexCount - 1);
436
437     if (source == sink) {
438         m_printError("Источник и сток должны различаться.");
439         return;
440     }
441
442     auto maxFlowResult = m_graph->fordFulkerson(source, sink);
443
444     int requiredFlow = (2 * maxFlowResult.maxFlow) / 3;
445
446     if (requiredFlow <= 0 && maxFlowResult.maxFlow > 0) {
447         requiredFlow = 1;
448     }
449
450     std::cout << "Максимальный поток: "
451               << maxFlowResult.maxFlow
452               << "\n";
453
454     std::cout << "Заданный поток [2/3 * max]: "
455               << requiredFlow
456               << "\n\n";
457
458     auto minCostResult = m_graph->minCostFlow(
459         source,
460         sink,
461         requiredFlow
462     );
463
464     if (!minCostResult.success) {
465         m_printError("Не удалось построить поток заданной величины.");
466         std::cout << "Достигнутый поток: "
467                   << minCostResult.achievedFlow
468                   << "\n";
469         std::cout << "Количество итераций: "
470                   << minCostResult.iterations
471                   << "\n\n";
472         return;
473     }
474
475     std::cout << "Поток минимальной стоимости найден.\n";
476     std::cout << "Стоимость потока: "
477               << minCostResult.totalCost
478               << "\n";
479
480     std::cout << "Количество итераций: "
481               << minCostResult.iterations
482               << "\n\n";
483
484     std::cout << "Матрица потоков минимальной стоимости:\n";
485     minCostResult.flowMatrix.print();
486
487     std::cout << "\nМатрица стоимостей минимального потока:\n";
488     minCostResult.costFlowMatrix.print();
489 }
490
491 // lab4
492 void CLI::m_menuSpanningTreesCount() {
493     m_printHeader("Число остовных деревьев");
494
495     auto result = m_graph->countSpanningTreesKirchhoff();
496
497     std::cout << "Матрица Кирхгофа:\n";
498     result.kirchhoffMatrix.print();
499
500     std::cout << "\nАлгебраическое дополнение A11:\n";
501     result.cofactorMatrix.print();
502
503     std::cout << "\nЧисло остовных деревьев: "
504               << result.count
505               << "\n";
506 }
507
508 void CLI::m_menuKruskalMinimumSpanningTree() {
509     m_printHeader("Минимальный остов: алгоритм Краскала");
510
511     if (!m_graph->isMatrixInit.weight) {
512         m_printError("Сначала сгенерируйте весовую матрицу.");
513         return;
514     }
515
516     auto result = m_graph->kruskalMinimumSpanningTree();

```

```

517
518 if (!result.kruskal.success) {
519     m_printError("Не удалось построить остов: граф несвязный.");
520     return;
521 }
522
523 std::cout << "Рёбра минимального остова:\n";
524
525 for (const auto& edge : result.kruskal.edges) {
526     std::cout << edge.from
527         << " -- "
528         << edge.to
529         << " вес: "
530         << edge.weight
531         << "\n";
532 }
533
534 std::cout << "\nВес минимального остова: "
535     << result.kruskal.totalWeight
536     << "\n\n";
537
538 std::cout << "Матрица весов минимального остова:\n";
539 result.kruskal.treeMatrix.print();
540
541 std::cout << "\nКод Прюфера: ";
542 for (size_t i = 0; i < result.pruferCode.size(); ++i) {
543     std::cout << result.pruferCode[i].vertex;
544
545     if (i + 1 < result.pruferCode.size()) {
546         std::cout << ", ";
547     }
548 }
549 std::cout << "\n";
550
551 std::cout << "Веса кода Прюфера: ";
552 for (size_t i = 0; i < result.pruferCode.size(); ++i) {
553     std::cout << result.pruferCode[i].weight;
554
555     if (i + 1 < result.pruferCode.size()) {
556         std::cout << ", ";
557     }
558 }
559 std::cout << "\n\n";
560
561 std::cout << "Матрица весов декодированного остова:\n";
562 result.decodedTreeMatrix.print();
563
564 std::cout << "\nПроверка декодирования: "
565     << (result.pruferRoundTripSuccess
566         ? "остов совпадает"
567         : "остов не совпадает")
568     << "\n";
569 }
570
571 void CLI::m_menuEdmondsBlossom() {
572     m_printHeader("Алгоритм Эдмондса: максимальное независимое множество рёбер");
573
574     std::cout << "Где искать:\n"
575         << " 1 - исходный граф\n"
576         << " 2 - минимальный остов\n";
577
578     int target = m_readInt("> ", 1, 2);
579
580     if (target == 2 && !m_graph->isMatrixInit.weight) {
581         m_printError(
582             "Сначала сгенерируйте весовую матрицу "
583             "для построения минимального остова."
584         );
585         return;
586     }
587
588     auto result = (target == 1)
589         ? m_graph->edmondsBlossomInOriginalGraph()
590         : m_graph->edmondsBlossomInMinimumSpanningTree();
591
592     if (!result.success) {
593         m_printError("Не удалось построить множество рёбер.");
594         return;
595     }
596
597     std::cout << "Размер множества: "
598         << result.edges.size()
599         << "\n";
600
601     std::cout << "Рёбра:\n";
602
603     if (result.edges.empty()) {
604         std::cout << " нет рёбер\n";
605     } else {
606         for (const auto& edge : result.edges) {
607             std::cout << edge.from
608                 << " -- "
609                 << edge.to;
610
611             if (result.hasWeights) {
612                 std::cout << " вес: "
613                     << edge.weight;
614             }
615
616             std::cout << "\n";
617         }
618     }
619
620     std::cout << "\nМатрица независимого множества рёбер:\n";
621     result.edgeMatrix.print();
622 }
623

```

```

624 void CLI::m_menuCheckEulerianGraph() const {
625     m_printHeader("Проверка, является ли неориентированный граф эйлеровым, построение эйлерова цикла");
626
627     if (!m_graph->isMatrixInit.adjacency) {
628         m_printError(
629             "Сначала сгенерируйте матрицу смежности "
630             "неориентированного графа."
631         );
632         return;
633     }
634
635     auto result = m_graph->checkIfEulerianGraph();
636
637     if (!result.transformationCompleted) {
638         m_printError(
639             "Невозможно завершить преобразование, не удалив мост "
640             "и не нарушив связность графа."
641         );
642         return;
643     }
644
645     m_printInfo(
646         std::string("Неориентированный граф ")
647         + (result.isEulerian ? "является" : "не является")
648         + " эйлеровым."
649     );
650
651
652
653     if (result.addedEdges.empty() && result.removedEdges.empty()) {
654         m_printInfo("Изменения графа не требуются.");
655     } else {
656         std::cout << "\nИзменения графа:\n";
657
658         if (!result.removedEdges.empty()) {
659             std::cout << "   удалены ребра:\n";
660             for (const auto& edge : result.removedEdges) {
661                 std::cout << edge.from
662                     << " -- "
663                     << edge.to
664                     << "\n";
665             }
666         }
667
668         if (!result.addedEdges.empty()) {
669             std::cout << "   добавлены ребра:\n";
670             for (const auto& edge : result.addedEdges) {
671                 std::cout << edge.from
672                     << " -- "
673                     << edge.to
674                     << "\n";
675             }
676         }
677     }
678
679     std::cout << "\nМатрица смежности эйлерова графа:\n";
680
681     result.eulerianAdjacencyMatrix.print();
682
683     if (!result.eulerianCycle.empty()) {
684         std::cout << "\nЭйлеров цикл: ";
685
686         for (size_t i = 0; i < result.eulerianCycle.size(); ++i) {
687             std::cout << result.eulerianCycle[i];
688
689             if (i + 1 < result.eulerianCycle.size()) {
690                 std::cout << " -> ";
691             }
692         }
693
694         std::cout << "\n";
695     }
696 }
697
698 void CLI::m_menuFundamentalCuts() const {
699     m_printHeader("Фундаментальная система разрезов");
700
701     if (!m_graph->isMatrixInit.adjacency ||
702         !m_graph->isMatrixInit.weight) {
703         m_printError(
704             "Сначала сгенерируйте неориентированный граф и матрицу весов."
705         );
706         return;
707     }
708
709     const auto result = m_graph->buildFundamentalCutSystem();
710
711     if (!result.success) {
712         m_printError("Не удалось построить минимальный остов связного графа.");
713         return;
714     }
715
716     std::cout << "\nМатрица весов исходного неориентированного графа:\n";
717     m_graph->getUndirectedWeightMatrix().print();
718
719     std::cout << "\n";
720     std::cout << "Рёбра минимального остова:\n";
721
722     for (size_t i = 0; i < result.spanningTree.edges.size(); ++i) {
723         const auto& edge = result.spanningTree.edges[i];
724         std::cout << i + 1
725             << ". "
726             << edge.from
727             << " -- "
728             << edge.to
729             << " вес: "
730             << edge.weight

```



```

731         << "\n";
732     }
733
734     std::cout << "\nВес минимального остова: "
735     << result.spanningTree.totalWeight
736     << "\n\nМатрица весов минимального остова:\n";
737     result.spanningTree.treeMatrix.print();
738
739     std::cout << "\nФундаментальные разрезы:\n";
740
741     for (size_t i = 0; i < result.fundamentalCuts.size(); ++i) {
742         const auto& fundamentalCut = result.fundamentalCuts[i];
743         std::cout << i + 1
744         << ". Удалено ребро остова "
745         << fundamentalCut.removedTreeEdge.from
746         << " -- "
747         << fundamentalCut.removedTreeEdge.to
748         << ": ";
749
750         for (const auto& edge : fundamentalCut.cut) {
751             std::cout << "(" << edge.from << ", " << edge.to << ") ";
752         }
753
754         std::cout << "\n";
755     }
756
757     const int count = m_readInt(
758         "\nКоличество разрезов для симметрической разности: ",
759         1,
760         static_cast<int>(result.fundamentalCuts.size())
761     );
762     std::vector<int> selectedCutIndices;
763     selectedCutIndices.reserve(count);
764
765     for (int i = 0; i < count; ++i) {
766         selectedCutIndices.push_back(
767             m_readInt(
768                 "Номер разреза: ",
769                 1,
770                 static_cast<int>(result.fundamentalCuts.size())
771             ) - 1
772         );
773     }
774
775     const GraphCut cut = m_graph->symmetricDifferenceOfFundamentalCuts(
776         result,
777         selectedCutIndices
778     );
779
780     std::cout << "Результат симметрической разности: ";
781
782     if (cut.empty()) {
783         std::cout << "пустой разрез";
784     } else {
785         for (const auto& edge : cut) {
786             std::cout << "(" << edge.from << ", " << edge.to << ") ";
787         }
788     }
789
790     std::cout << "\n";
791 }
792
793 void CLI::m_printMenu() const {
794     std::cout
795     << "\n----- Меню ----- \n"
796     << "1. Сгенерировать граф\n"
797     << "2. Эксцентриситеты, центр, диаметр\n"
798     << "3. Сгенерировать весовую матрицу\n"
799     << "4. Алгоритм Шимбелла\n"
800     << "5. Маршруты между вершинами\n"
801     << "6. Показать текущие матрицы\n"
802     << "7. Найти точки сочленения\n"
803     << "8. Алгоритм Дейкстры для отрицательных весов\n"
804     << "9. Сгенерировать матрицы пропускных способностей и стоимостей\n"
805     << "10. Максимальный поток\n"
806     << "11. Поток минимальной стоимости\n"
807     << "12. Число остовных деревьев\n"
808     << "13. Минимальный остов: алгоритм Краскала\n"
809     << "14. Алгоритм Эдмондса: максимальное независимое множество рёбер\n"
810     << "15. Проверка, является ли неориентированный граф эйлеровым, построение эйлерова цикла\n"
811     << "16. Фундаментальная система разрезов\n"
812     << "0. Выход\n";
813 }
814
815 void CLI::run() {
816     std::cout << "=== Теория графов ===\n";
817
818     m_generateGraph();
819
820     std::cout << "\n";
821     m_printInfo("Граф сгенерирован автоматически.");
822
823     m_graph->printAdjacencyMatrix();
824
825     while (true) {
826         m_printMenu();
827
828         int choice = m_readInt("> ", 0, 16);
829
830         if (choice == 0) {
831             break;
832         }
833
834         switch (choice) {
835             case 1:
836                 m_menuGenerate();
837                 break;

```

```

838
839         case 2:
840             m_menuEccentricities();
841             break;
842
843         case 3:
844             m_menuWeights();
845             break;
846
847         case 4:
848             m_menuShimbell();
849             break;
850
851         case 5:
852             m_menuRoutes();
853             break;
854
855         case 6:
856             m_menuPrintMatrices();
857             break;
858
859         case 7:
860             m_menuFindArticulationPoints();
861             break;
862
863         case 8:
864             m_runDijkstraNegative();
865             break;
866
867         case 9:
868             m_menuCapacityAndCostMatrices();
869             break;
870
871         case 10:
872             m_menuMaxFlow();
873             break;
874
875         case 11:
876             m_menuMinCostFlow();
877             break;
878
879         case 12:
880             m_menuSpanningTreesCount();
881             break;
882
883         case 13:
884             m_menuKruskalMinimumSpanningTree();
885             break;
886
887         case 14:
888             m_menuEdmondsBlossom();
889             break;
890
891         case 15:
892             m_menuCheckEulerianGraph();
893             break;
894
895         case 16:
896             m_menuFundamentalCuts();
897             break;
898     }
899 }
900
901     std::cout << "Выход.\n";
902 }
903
904 int main() {
905     CLI cli;
906     cli.run();
907
908     return 0;
909 }

```